Machine Learning
Lecture Notes

| Bachelor in Applied Mathematics |
IE University

Nacho Molina

Academic Year 2025–2026

Contents

1. Foundations of Machine Learning **1**

1. Introduction to ML & the Learning Problem **2**
1.1. What is Machine Learning? . . . 2
1.2. Why Does Machine Learning Matter? . . . 4
1.3. The Main Paradigms of Machine Learning . . . 5

2. Linear Regression **12**
2.1. Motivation: Predicting Housing Prices . . . 12
2.2. Linear model and assumptions . . . 12
2.3. Loss function: Mean Squared Error (MSE) . . . 13
2.4. Fitting by minimizing the loss . . . 13
2.5. Gradient Descent for Linear Regression . . . 15
2.6. Stochastic Gradient Descent (SGD) . . . 17
2.7. Evaluation Metrics . . . 19
2.8. A Probabilistic View of Linear Regression . . . 21

3. Logistic Regression **24**
3.1. Motivation: Why Linear Regression Fails for Classification . . . 24
3.2. The Logistic (Sigmoid) Function . . . 24
3.3. Model Formulation . . . 25
3.4. Loss Function: Cross-Entropy . . . 27
3.5. Optimization with Gradient Descent . . . 29
3.6. Probabilistic Interpretation . . . 30
3.7. Evaluation Metrics . . . 31
3.8. Extensions . . . 33

4. Dimensionality Reduction and Clustering **36**
4.1. Introduction . . . 36
4.2. Principal Component Analysis (PCA) . . . 37
4.3. k-Means Clustering . . . 45

5. Model Performance, Generalization, and Regularization **50**
5.1. Underfitting, Overfitting, and Generalization . . . 50
5.2. The Bias–Variance Trade-off . . . 51
5.3. Regularization and Model Complexity Control . . . 52

2. Nonlinear and Non-Parametric Models **55**

6. Kernel Methods **56**
6.1. Motivation: From Linear Models to Nonlinear Feature Spaces . . . 56
6.2. Dual Representation of Ridge Regression . . . 58
6.3. The Kernel Trick . . . 61

6.4. Geometric Interpretation of Kernels . . . 63
6.5. Kernel Methods for Classification . . . 64
6.6. Kernel Principal Component Analysis (Kernel PCA) . . . 66
6.7. Discussion and Outlook . . . 68

7. Neural Networks **70**
7.1. Introduction: function approximation and the need for neural networks . . . 70
7.2. Neural networks: definition and basic components . . . 71
7.3. Activation functions . . . 73
7.4. Universal approximation and function representation . . . 74
7.5. Learning neural networks: backpropagation on a toy three-neuron chain . . . 75
7.6. Neural networks for regression and classification . . . 77
7.7. Autoencoders and nonlinear dimensionality reduction . . . 79
7.8. Practical aspects: training stable and generalizing networks . . . 81

8. Tree-Based Methods **85**
8.1. Motivation and Overview . . . 85
8.2. Decision Trees for Regression and Classification . . . 86
8.3. Overfitting and Regularization of Trees . . . 89
8.4. Tree Ensemble Methods: Bagging and Random Forests . . . 92
8.5. Boosting and Gradient Boosting . . . 95
8.6. Practical Guidance and Summary . . . 98

A. Exercises **101**

Part 1.

Foundations of Machine Learning

1. Introduction to ML & the Learning Problem

1.1. What is Machine Learning?

The search for patterns in data has been a central theme of science since its very origins. From the motion of celestial bodies to the spread of epidemics, scientific discovery often begins by identifying regularities in observations and then formulating mathematical laws that capture them.

A classical example is provided by the astronomical observations of Tycho Brahe in the 16th century. Over two decades, Brahe recorded extremely precise measurements of planetary positions. These data enabled Johannes Kepler to infer the empirical laws of planetary motion, which later provided the foundation for Newton's theory of gravitation. This process, i.e. collecting data, finding patterns, and formulating predictive rules, is essentially the same principle underlying modern machine learning.

> **Bishop's definition:** *Machine learning can be understood as the study of algorithms and models that discover and exploit patterns in data, with the goal of making predictions or decisions in situations of uncertainty.*

Core Elements of a Machine Learning Problem

A complementary perspective is offered by Tom Mitchell, who provides a more operational definition of machine learning:

> **Mitchell's definition:** *A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.*

Consider the task of detecting whether an incoming email is spam. The task (T) is to classify each email as spam or not spam. The experience (E) consists of a dataset of past emails already labeled by users. The performance (P) can be measured, for instance, by the accuracy of the classifier on new, unseen emails. In this setting, the machine learning program automatically infers patterns, such as words, phrases, or sender information, that distinguish spam from legitimate messages, without requiring a human to explicitly code the rules.

Mitchell's definition may at first read like a confusing tongue twister, but its strength lies in making explicit the essential components of any machine learning problem:

1. **Model** (Program): the mathematical algorithm that can generate solutions.
2. **Dataset** (Experience): the data from which the model will learn.
3. **Objective** (Task): the specific problem to be solved.
4. **Evaluation** (Performance): a quantitative measure of success.

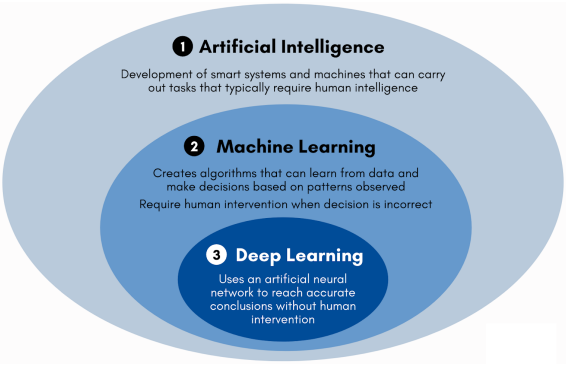


Figure 1.1.: Relationship between artificial intelligence (AI), machine learning (ML), and deep learning (DL). AI is the broadest field, encompassing any method to build intelligent systems. ML is a subfield of AI focused on algorithms that learn from data. DL is a subset of ML based on multi-layer neural networks that automatically learn hierarchical representations, and has driven many recent advances in AI.

An Integrative View

Bringing these perspectives together, we can formulate a more contemporary definition:

> **Integrative definition:** *Machine learning is the discipline concerned with developing mathematical models and algorithms that improve their ability to perform tasks, as measured by well-defined performance criteria, through identifying and exploiting patterns in data.*

This unified definition combines Bishop's emphasis on *pattern recognition* with Mitchell's operational framework of tasks, experience, and performance, providing a foundation for both the theory and applications of modern machine learning.

Machine Learning vs Artificial Intelligence vs Deep Learning

The terms *artificial intelligence* (AI), *machine learning* (ML), and *deep learning* (DL) are often used interchangeably in popular media, but they refer to concepts of different scope. Figure 1.1 illustrates their relationship.

Artificial Intelligence. AI is the broadest term, referring to the design of systems that display behaviors associated with intelligence: reasoning, problem solving, perception, planning, and language understanding. Classical AI includes rule-based systems, search algorithms, and expert systems, many of which do not involve learning from data.

Machine Learning. ML is a subfield of AI focused on algorithms that *learn from data*. Instead of programming explicit rules, we provide data and allow models to infer patterns and make predictions. Regression, decision trees, support vector machines, and probabilistic models all fall within ML. In practice, ML has become the dominant approach to building AI systems today.

Deep Learning. DL is a subset of ML that uses multi-layered artificial neural networks to automatically learn rich, hierarchical representations of data. While classical ML often relies on handcrafted features, deep models learn features directly from raw inputs (e.g., pixels, audio waveforms, text). This has led to breakthroughs in computer vision, speech recognition, and natural language processing.

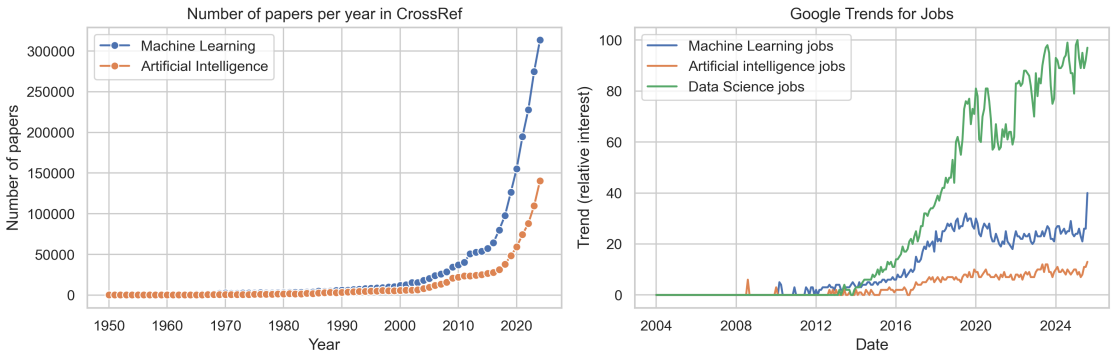


Figure 1.2.: Left: Number of publications per year in Crossref that mention "Machine Learning" or "Artificial Intelligence," showing rapid (approximately exponential) growth in recent years. Right: Google Trends time series for "Machine Learning jobs," "Data Science jobs," and "Artificial Intelligence jobs," used as a proxy for job-market interest. Together, these trends illustrate ML's expanding scientific footprint and economic relevance.

AI is the overarching goal of creating intelligent systems; ML provides the data-driven methodology to achieve this; and DL offers powerful neural-network architectures that have driven many of the most impressive successes in AI in the past decade.

1.2. Why Does Machine Learning Matter?

The scale and complexity of modern data

While early scientists worked with small, carefully curated datasets, today we operate in environments where data are **massive in volume**, **rapidly generated**, **multi-modal**, and often **noisy or biased**. Three domains illustrate the scale:

Biotech: A human genome has $\sim 3 \times 10^9$ base pairs, with routine sequencing yielding $\sim$ 50–200 GB per person; biobanks of 10^5–10^6 genomes reach $\sim$ 10–200 PB. Single-cell atlases span 10^5–10^7 cells with 10^3–10^4 features, generating matrices with 10^9–10^{11} entries. High-throughput microscopy and spatial omics add slides of 10^8–10^9 pixels, producing terabytes daily. These datasets are vast, sparse, and noisy, demanding scalable methods for integration and denoising.

Internet: The web hosts 10^9–10^{10} pages, while search engines process $\sim 10^9$–10^{10} queries each day. Large platforms log 10^{11}–10^{13} daily user interactions, and social networks connect 10^8–10^9 users with 10^{11}–10^{12} edges. Content flows at 10^8–10^9 posts or messages per day. Such data are dynamic, heterogeneous, and networked, requiring ML models that scale and adapt in real time.

Economy and finance: Markets generate millions of trades per second, creating terabytes of tick-level data daily. Payment systems handle billions of transactions each day, while macroeconomic indicators track populations of 10^6–10^8 households. E-commerce platforms list 10^7–10^8 products and log 10^9–10^{10} interactions daily. Economic data are high-frequency, incomplete, and heterogeneous, demanding ML systems that integrate transactional, textual, and aggregated streams to support forecasting and decision-making.

These characteristics push us beyond simple curve-fitting: we need models that can capture non-linear structure, quantify uncertainty, handle high-dimensional dependencies, and scale computationally. This is precisely where modern ML proves essential.



Figure 1.3.: Regression of house prices. Each point is a house; panels illustrate how sale price varies with living area and overall quality. The task is to learn a predictive function from features to price. In the course we will approach this with ordinary and Bayesian linear regression, regularization (ridge and lasso), kernel methods, Gaussian processes, neural networks, and tree-based regressors, evaluating generalization with appropriate validation schemes.

Economic and societal impact of ML

Machine Learning is reshaping the economy and society, and the pace of change is accelerating. One way to see this is through the research pipeline and the labor market. On the *research* side, the number of papers that mention "machine learning" or "artificial intelligence" has grown explosively over the last decades, reflecting rapid methodological progress and diffusion across disciplines. On the *labor* side, interest in ML-related jobs (used here as a rough proxy via Google Trends) shows sustained growth for terms such as "Machine Learning jobs," "Data Science jobs," and "Artificial Intelligence jobs," consistent with expanding opportunities in industry and the public sector. While search activity is an imperfect proxy, together these signals suggest that ML's scientific footprint and its employment impact are both rising sharply.

1.3. The Main Paradigms of Machine Learning

Machine learning problems are often grouped by the type of signal available during training and by the goal of the analysis. In this course we will move back and forth between these paradigms, using a common probabilistic and geometric language to connect them. The figures in this section illustrate each setting on real datasets and anticipate the models that we will study in later modules.

Supervised Learning

In supervised learning we observe input–output pairs $(\mathbf{x}_i, y_i)$ and seek a rule that generalizes from the training data to unseen examples. The central questions are how to represent the rule, how to fit its parameters, and how to assess generalization. We begin with linear models and their probabilistic interpretations, develop classification as inference, and then extend to richer function classes and tree-based ensembles. Model selection, regularization, and cross-validation will be used throughout to balance fit and generalization.

Regression In regression the target y is continuous. Figure 1.3 shows a classical example: predicting house sale price from features such as living area and overall quality. We will start with least-squares and its probabilistic view, then introduce Bayesian linear

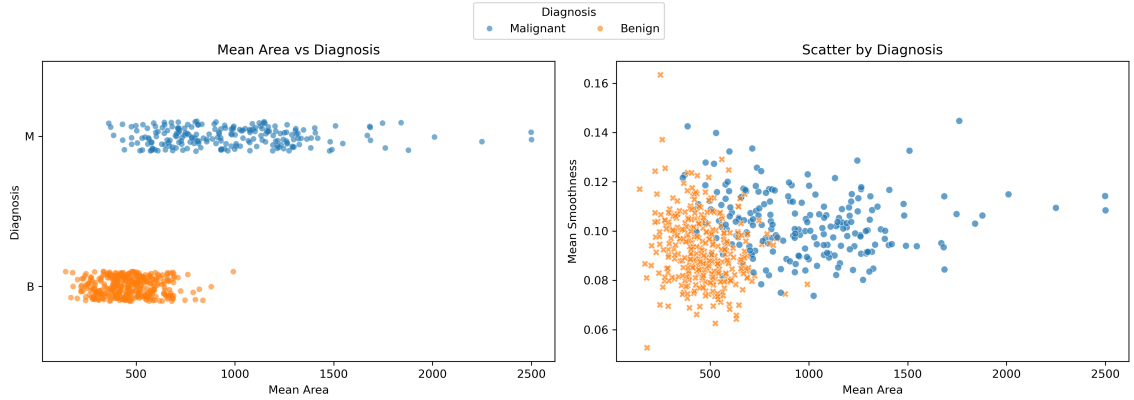


Figure 1.4.: Classification of breast cancer patients. Two views of the feature space show separation between benign and malignant diagnoses. We will build classifiers with logistic regression, LDA/QDA and Naive Bayes, support vector machines with kernels, and tree-based ensembles, and assess them with out-of-sample validation and classification metrics.

regression, ridge and lasso regularization, and model selection. Later we will handle nonlinearity with kernel methods and Gaussian processes, consider generalized linear models for non-Gaussian noise, and compare these with flexible function approximators such as neural networks and with tree-based methods for regression.

Classification In classification the target y takes discrete values. Figure 1.4 uses the Wisconsin Breast Cancer dataset to illustrate how cell measurements can separate benign from malignant cases. We will model class probabilities with logistic regression, study generative linear classifiers such as LDA and QDA (including Naive Bayes), and then develop large-margin methods with SVMs and kernels. We will also compare these to decision trees and ensemble methods (random forests and boosting), and evaluate models with metrics such as accuracy and ROC–AUC using cross-validation.

Unsupervised Learning

Unsupervised learning seeks structure in data without labels. The aims include uncovering low-dimensional organization for visualization or preprocessing, identifying groups of similar observations, and estimating probability distributions. We will treat these tasks with linear algebra, geometry, and probabilistic modeling, and we will see how unsupervised structure often improves downstream supervised performance.

Dimensionality Reduction High-dimensional measurements often lie near a lower-dimensional manifold. Figure 1.5 contrasts a linear projection with a nonlinear embedding of the cancer features, revealing patterns that are not obvious in the raw space. We will study PCA from its variance-maximization and SVD derivations, extend it probabilistically (pPCA) and via Factor Analysis to model covariance structure, and then turn to distance- and graph-based embeddings such as Multidimensional Scaling, Isomap, and Laplacian Eigenmaps for nonlinear geometry. These tools will be used both for interpretation and as a preprocessing step for classification and clustering.

Clustering Clustering aims to discover groups of similar points. Figure 1.6 shows a single-cell transcriptomic atlas (Tabula Sapiens) embedded in two dimensions, where cell types form coherent clusters. We will analyze clustering from geometric and probabilistic viewpoints: k-means and hierarchical clustering for partitioning by geometry, and Gaussian mixture models fit by Expectation–Maximization for density-based clustering with

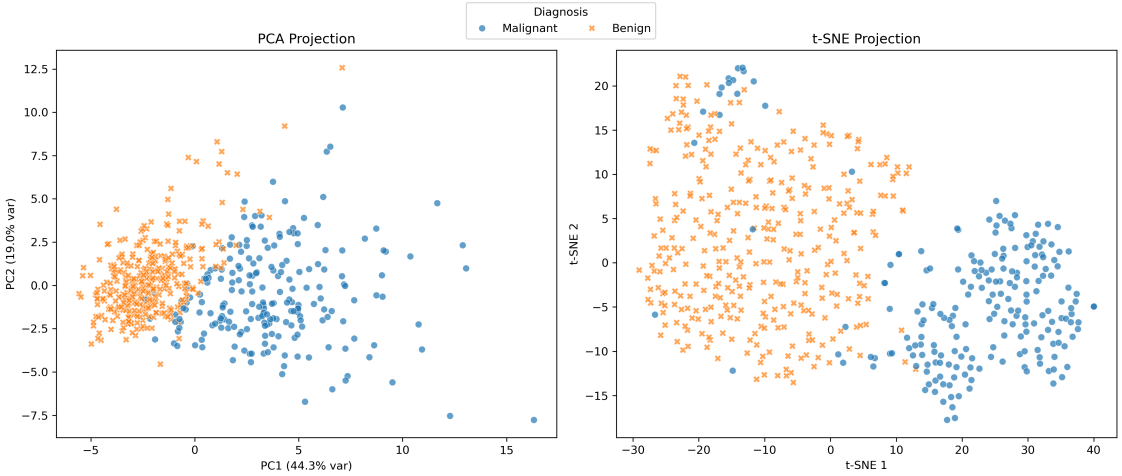


Figure 1.5.: Dimensionality reduction using the cancer data. A linear projection (PCA, left) and a nonlinear embedding (right) reveal low-dimensional structure helpful for visualization and downstream modeling. In class we will develop PCA, probabilistic PCA and Factor Analysis, together with distance- and graph-based methods including MDS, Isomap, and Laplacian Eigenmaps.

model selection. Spectral ideas will appear via graph-based embeddings such as Laplacian Eigenmaps, which often make cluster structure more apparent.

Density estimation Estimating a probability distribution from data underpins many unsupervised tasks. We will use Gaussian mixture models for flexible multimodal densities and connect probabilistic PCA and Factor Analysis to latent-variable density modeling. These models enable tasks such as anomaly detection, soft clustering, and uncertainty-aware preprocessing.

Hybrid Paradigms

Between fully labeled and unlabeled settings there are regimes where a small amount of supervision guides structure learning.

Semi-supervised learning uses unlabeled data to improve predictive models when labels are scarce, often relying on the same regularization and generative ideas we develop for linear models, mixtures, and graph-based embeddings.

Self-supervised learning creates surrogate prediction tasks from the data itself to learn representations that transfer to downstream problems. In this course we will emphasize the underlying probabilistic and geometric tools that make these approaches effective, even when we do not study particular large-scale pipelines in depth.

Generative AI

Modern generative AI systems are trained with unsupervised or self-supervised objectives to model complex data distributions and produce realistic samples. While our focus is not on large generative models, several building blocks recur in this course: probabilistic modeling, latent variables, kernels, and Gaussian processes. Understanding these foundations prepares you to read and reason about contemporary generative methods.

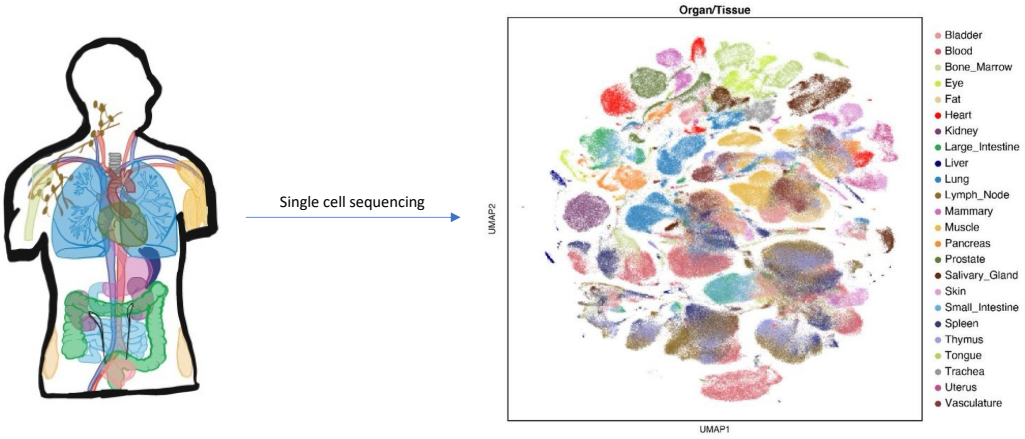


Figure 1.6.: Clustering in the Tabula Sapiens single-cell atlas. A graph-based embedding of thousands of cells arranges transcriptomes so that related cell types cluster together. We will compare geometric clustering (k-means, hierarchical) with probabilistic clustering using Gaussian mixture models and EM, and use spectral embeddings (e.g., Laplacian Eigenmaps) to reveal cluster structure.

Reinforcement Learning

Reinforcement learning studies how agents learn actions by interacting with an environment and receiving rewards. Although we will not develop RL algorithms in detail, the probabilistic mindset, function approximation, and generalization principles established in supervised and unsupervised learning will give you a solid conceptual bridge to this paradigm.

Formalizing the Learning Problem: From Data to Models

Data. Let (X, Y) be random variables defined on a probability space, with values in measurable spaces $\mathcal{X}$ (inputs, or features) and $\mathcal{Y}$ (targets, or labels). We posit an unknown data-generating distribution $\mathbb{P}(\curvearrowleft, \curvearrowright)$ and receive an i.i.d. sample:

$$\mathcal{D} \;=\; \{(x_i, y_i)\}_{i=1}^{n}.$$

The goal of *learning* is to use $\mathcal{D}$ to construct a function (a *predictor*) $f : \mathcal{X} \to \mathcal{Y}$ that performs well on new pairs sampled from $P(x, y)$

Hypothesis spaces. We restrict attention to a family of functions or models (hypothesis class)

$$\mathcal{H} \;\subseteq\; \{f : \mathcal{X} \to \mathcal{Y}\},$$

often parameterized by $\theta \in \Theta \subseteq \mathbb{R}^p$, e.g. linear functions, kernels, trees, or neural networks: $\mathcal{H} = \{f_\theta : \theta \in \Theta\}$. The choice of $\mathcal{H}$ determines what kinds of patterns our model can represent: a very simple $\mathcal{H}$ (such as straight lines) can only capture simple trends, while a richer $\mathcal{H}$ (such as high-degree polynomials or neural networks) can express much more complex relationships.

Loss function A *loss* $\ell : \mathcal{Y} \times \mathcal{Y} \to [0, \infty)$ quantifies prediction error. Common examples are the squared loss $\ell(\hat{y}, y) = (\hat{y} - y)^2$ (regression) and the 0–1 loss $\ell(\hat{y}, y) = \mathbb{1}\{\hat{y} \neq y\}$ (classification), $\ell(\hat{y}, y) = \{1 : y = y, \quad 0 : \hat{y} \neq y\}$. along with smooth surrogates (e.g.

logistic/cross-entropy). The *expected* (or *population*) *risk* of f is

$$R(f) \;=\; \int \ell\big(f(x), y\big) P(x, y) dx dy$$

Because $\mathbb{P}(x, y)$ is unknown, we optimize the *empirical* risk

$$R_n(f) \;=\; \frac{1}{n} \sum_{i=1}^{n} \ell\big(f(x_i), y_i\big).$$

Optimizing the empirical loss: finding the best-fitting f_θ. Once we choose a hypothesis space $\mathcal{H} = \{f_\theta\}$ and a loss ℓ, the learning problem becomes:

$$\hat{\theta} \;=\; \arg\min_{\theta} \; R_n\big(f_\theta\big) \quad \Longleftrightarrow \quad \hat{f} \;=\; f_{\hat{\theta}}.$$

Intuitively, $\hat{\theta}$ is the set of parameters that makes the average error on the observed data as small as possible. In practice we solve this optimization with standard tools (closed-form solutions when available, or numerical methods such as gradient descent).

The Learning Problem in Action

We now illustrate the abstract learning framework with a concrete and societally relevant problem: modeling the evolution of global temperature anomalies over time. Understanding long-term warming trends is one of the most pressing scientific and policy challenges of our time. From the perspective of machine learning, this dataset provides an ideal didactic example: it is a real-world time series, it contains clear structure but also substantial variability, and the task—predicting temperature anomalies from time—is a regression problem that can be approached with increasingly flexible models.

(i) Data and de-seasonalization. Our input variable is time x_t, measured in months between 1880 and 2022. The target variable y_t is the global land temperature anomaly relative to a historical baseline. The raw monthly anomalies (Figure 1.7a) show a pronounced seasonal cycle and strong year-to-year fluctuations. To extract the underlying long-term warming trend, we smooth the series by averaging over 12 months. The resulting yearly anomalies (Figure 1.7b) provide a cleaner signal for modeling.

(ii) Hypothesis space: polynomials. We consider polynomial regression models of increasing degree d:

$$\mathcal{H}_d \;=\; \big\{ f_\theta(x) = \theta_0 + \theta_1 x + \cdots + \theta_d x^d \big\}.$$

Low-degree polynomials correspond to simple trends (linear or quadratic growth), while high-degree polynomials are increasingly expressive and able to follow fine-grained patterns in the data (Figure 1.7c).

(iii) Loss function: mean squared error. We measure the quality of fit using the mean squared error (MSE). For polynomial degree d and parameters θ, the empirical risk on a dataset $\mathcal{D}$ is

$$\mathrm{MSE}_{\mathcal{D}}(\theta; d) \;=\; \frac{1}{|\mathcal{D}|} \sum_{(x_i, y_i) \in \mathcal{D}} \Big(y_i - \theta^\top \phi_d(x_i) \Big)^2,$$

where we define the *polynomial feature map*

$$\phi_d(x) = \big(1,\, x,\, x^2,\, \ldots,\, x^d\big)^\top.$$

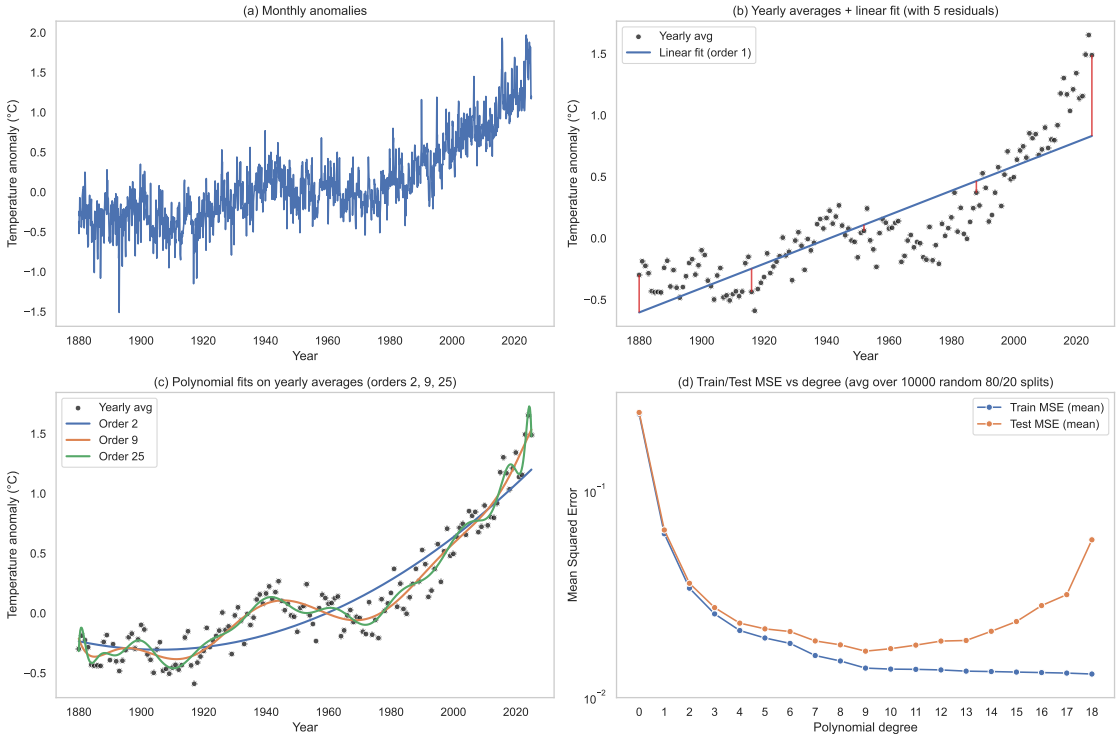


Figure 1.7.: The learning problem in action: modeling the global temperature anomaly over time (1880–2022). (a) Raw monthly anomalies reveal large seasonal and interannual variability. (b) Yearly averages with a linear fit highlight the long-term warming trend; red segments show residuals between the model and five sampled years. (c) Polynomial regressions of orders 2, 9, and 25 on yearly averages demonstrate how increasing model complexity can capture nonlinear patterns but also produce spurious oscillations. (d) Average training and test mean squared errors (MSE) over 10,000 random 80/20 splits as a function of polynomial degree (0–18), illustrating the bias–variance trade-off: error decreases with moderate degrees, but high-degree models overfit and generalization error rises again.

This definition is very convenient: although the model $f(x) = \theta_0 + \theta_1 x + \cdots + \theta_d x^d$ is nonlinear in x, it is *linear in the parameters* θ. By rewriting the problem in terms of the feature vector $\phi_d(x)$, polynomial regression becomes a linear regression problem in the parameters. In fact, it is possible to compute analytically the values of θ that minimize the loss by setting the gradient to zero, something we will derive in detail in the next chapter.

Training and test errors

A key principle in machine learning is to distinguish between the data we use to *fit* the model and the data we use to *evaluate* how well it generalizes. We therefore split the dataset into two parts:

- a *training set* (80% of the data in this example), used to compute $\hat{\theta}_d$ by minimizing the training MSE,
- and a *test set* (the remaining 20%), kept aside and used only to measure predictive performance.

For each polynomial degree $d = 0, 1, 2, \ldots, D_{\max}$ we proceed in two steps:

1. Fit the parameters $\hat{\theta}_d$ by minimizing $\mathrm{MSE}_{\mathrm{train}}(\theta; d)$ on the training set.

2. Evaluate both the *training error*, $\text{MSE}_{\text{train}}(\hat{\theta}_d; d)$, and the *test error*, $\text{MSE}_{\text{test}}(\hat{\theta}_d; d)$.

Repeating this random split many times (here, 10,000 repetitions with 80/20 splits) and averaging the results gives a stable estimate of how training and test errors behave as a function of model complexity. This procedure reveals the characteristic pattern shown in Figure 1.7d: training error decreases monotonically with degree, while test error follows a U-shape, decreasing at first and then rising again as the model begins to overfit (Figure 1.7d

Capacity, Overfitting, and Generalization

This example allows us to visualize three central concepts in machine learning: *capacity*, *overfitting*, and *generalization*.

Capacity. The capacity of a hypothesis space refers to its ability to represent complex functions. As the polynomial degree d increases, the space $\mathcal{H}_d$ becomes richer: linear models (degree 1) capture only straight trends, while higher degrees can fit nonlinear curvature and oscillations.

Overfitting. Because $\mathcal{H}_d \subset \mathcal{H}_{d+1}$, training error always decreases with d. Indeed, a 25th-degree polynomial can nearly interpolate the yearly averages (Figure 1.7c), achieving very low training error. However, such flexibility comes at a cost: the oscillations are not physically meaningful, but artifacts of fitting noise and sampling fluctuations. This is the hallmark of overfitting: low training error but poor predictive performance.

Generalization. The true goal of learning is not to minimize training error, but to perform well on unseen data. Figure 1.7d shows that test error decreases initially as the model gains capacity (degree 2 fits the warming trend better than degree 1), reaches a minimum around moderate degree, and then increases again at high degrees. This U-shaped curve illustrates the bias–variance trade-off, which we will study formally in the next chapter: simple models underfit (high bias, poor fit), while overly complex models overfit (high variance, poor generalization). The sweet spot lies where test error is minimized, a balance between capturing the true signal and avoiding spurious noise.

Takeaway. The temperature example demonstrates the essential structure of all machine learning problems: choosing a hypothesis space (capacity), fitting it to training data (optimization), and evaluating performance on unseen data (generalization). It also foreshadows key techniques we will study later: regularization to control capacity, cross-validation to select models, and probabilistic methods to quantify uncertainty.

2. Linear Regression

Linear models form the cornerstone of machine learning. Despite their apparent simplicity, they provide a powerful and interpretable framework for relating inputs to outputs and serve as the gateway to more complex models. In this chapter we introduce linear regression, motivated by the practical problem of predicting housing prices. We begin by formalizing the linear assumption that each feature contributes additively and proportionally to the prediction, and we derive the least-squares criterion for fitting the model. We then present two main optimization strategies: the *normal equations*, which give a closed-form solution, and *gradient descent*, which iteratively updates the parameters and anticipates methods used for non-linear models. We extend this to *stochastic gradient descent*, where only a small batch of data points is used at each step, making learning scalable to modern large datasets. Finally, we revisit linear regression from a probabilistic perspective, showing how minimizing squared error corresponds to maximum likelihood under Gaussian noise and how this formulation naturally extends to uncertainty quantification and generative modeling. Together, these perspectives establish linear regression not only as a practical predictive tool but also as a foundational paradigm for the broader study of machine learning.

2.1. Motivation: Predicting Housing Prices

A classical and practical use case for linear regression is the prediction of house sale prices. Let $y_i \in \mathbb{R}$ denote the sale price of house i, and let $\mathbf{x}_i = (x_{i1}, \ldots, x_{ip})^\top \in \mathbb{R}^p$ collect p measurable attributes (*features*) of that house, such as living area, number of rooms, age, neighborhood indicators, etc. We will augment the feature vector with a constant $x_{i0} = 1$ to learn an intercept:

$$\tilde{\mathbf{x}}_i \;=\; (1, x_{i1}, \ldots, x_{ip})^\top \in \mathbb{R}^{p+1}.$$

Given a dataset $\{(\tilde{\mathbf{x}}_i, y_i)\}_{i=1}^n$, our goal is to learn a function that predicts y_i from $\tilde{\mathbf{x}}_i$ and generalizes well to unseen houses.

2.2. Linear model and assumptions

In *linear regression* we assume the prediction for house i is a linear combination of its features:

$$\hat{y}_i \;=\; \theta_0 + \theta_1 x_{i1} + \cdots + \theta_p x_{ip} \;=\; \tilde{\mathbf{x}}_i^\top \boldsymbol{\theta}, \qquad \boldsymbol{\theta} = (\theta_0, \theta_1, \ldots, \theta_p)^\top.$$

This carries three important modeling assumptions:

1. **Proportional contribution (linearity in features):** Each feature x_{ij} contributes an amount *proportional* to its value, with proportionality constant θ_j. Increasing x_{ij} by one unit increases the prediction by exactly θ_j, *all else equal*.

2. **Additivity (no interactions):** The total predicted price is the *sum* of the individual contributions $\theta_j x_{ij}$ plus an intercept. There are no interaction terms like $x_{ij}x_{ik}$ unless we explicitly include them as additional features.

3. **Context-independence of effects:** The contribution of feature j does *not* depend on the levels of other features (e.g., the effect of one extra square meter is θ_1 no matter the number of rooms).

It is common to relate the prediction to the observed price via a "noise" model $y_i = \tilde{\mathbf{x}}_i^\top \boldsymbol{\theta} + \varepsilon_i$ with $\mathbb{E}[\varepsilon_i] = 0$. While we will not rely on distributional assumptions here, this motivates the loss we minimize.

2.3. Loss function: Mean Squared Error (MSE)

To fit the model we choose parameters $\boldsymbol{\theta}$ that make predictions close to observed prices. We quantify closeness with the mean squared error:

$$\mathcal{L}(\boldsymbol{\theta}) \;=\; \frac{1}{n}\sum_{i=1}^{n}\Big(y_i - \tilde{\mathbf{x}}_i^\top \boldsymbol{\theta}\Big)^2 \;=\; \frac{1}{n}\sum_{i=1}^{n} e_i(\boldsymbol{\theta})^2, \qquad e_i(\boldsymbol{\theta}) := y_i - \sum_{j=0}^{p}\theta_j x_{ij}.$$

(The factor $1/n$ does not affect the minimizer, but is convenient for interpretation.)

2.4. Fitting by minimizing the loss

The *best* linear model in the least-squares sense is obtained by minimizing $\mathcal{L}(\boldsymbol{\theta})$. Because $\mathcal{L}$ is a convex quadratic function of $\boldsymbol{\theta}$, any stationary point is the global minimizer. Therefore, we find $\hat{\boldsymbol{\theta}}$ by setting the gradient to zero.

Element-wise derivation (normal equations)

Let us compute the partial derivative of $\mathcal{L}$ with respect to a generic parameter θ_k for $k = 0, 1, \ldots, p$. Start from

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n}\big(y_i - \sum_{j=0}^{p}\theta_j x_{ij}\big)^2.$$

Step 1 (chain rule on each term). For a fixed i,

$$\frac{\partial}{\partial\theta_k}\Big(y_i - \sum_{j=0}^{p}\theta_j x_{ij}\Big)^2 = 2\Big(y_i - \sum_{j=0}^{p}\theta_j x_{ij}\Big)\cdot\frac{\partial}{\partial\theta_k}\Big(y_i - \sum_{j=0}^{p}\theta_j x_{ij}\Big).$$

Step 2 (derivative of the inner term). Since y_i does not depend on θ_k,

$$\frac{\partial}{\partial\theta_k}\Big(y_i - \sum_{j=0}^{p}\theta_j x_{ij}\Big) = -\frac{\partial}{\partial\theta_k}\Big(\sum_{j=0}^{p}\theta_j x_{ij}\Big) = -x_{ik}.$$

Step 3 (combine Steps 1–2).

$$\frac{\partial}{\partial\theta_k}\Big(y_i - \sum_{j=0}^{p}\theta_j x_{ij}\Big)^2 = -2\,x_{ik}\Big(y_i - \sum_{j=0}^{p}\theta_j x_{ij}\Big).$$

Step 4 (sum over i and scale by $1/n$). Therefore,

$$\frac{\partial\mathcal{L}}{\partial\theta_k}(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n}\big[-2\,x_{ik}\big(y_i - \sum_{j=0}^{p}\theta_j x_{ij}\big)\big] = -\frac{2}{n}\sum_{i=1}^{n}x_{ik}\Big(y_i - \sum_{j=0}^{p}\theta_j x_{ij}\Big).$$

Step 5 (set the gradient to zero). At the minimizer $\hat{\boldsymbol{\theta}}$ we have

$$\frac{\partial \mathcal{L}}{\partial \theta_k}(\hat{\boldsymbol{\theta}}) = 0 \quad \Longleftrightarrow \quad \sum_{i=1}^{n} x_{ik}\Big(y_i - \sum_{j=0}^{p} \hat{\theta}_j x_{ij}\Big) = 0, \qquad k = 0, 1, \ldots, p.$$

Step 6 (expand the sums).

$$\sum_{i=1}^{n} x_{ik} y_i \; - \; \sum_{i=1}^{n}\sum_{j=0}^{p} x_{ik} x_{ij}\, \hat{\theta}_j \; = \; 0 \quad \Longleftrightarrow \quad \sum_{j=0}^{p}\Big(\sum_{i=1}^{n} x_{ik} x_{ij}\Big)\hat{\theta}_j \; = \; \sum_{i=1}^{n} x_{ik} y_i, \quad k = 0, \ldots, p.$$

This is a linear system of $p{+}1$ equations in the $p{+}1$ unknowns $\{\hat{\theta}_j\}_{j=0}^{p}$.

Matrix form and closed-form solution

Define the design matrix $X \in \mathbb{R}^{n\times(p+1)}$ by $X_{ij} = x_{ij}$ (with $x_{i0} = 1$), the parameter vector $\boldsymbol{\theta} \in \mathbb{R}^{p+1}$, and the response vector $\mathbf{y} = (y_1, \ldots, y_n)^\top$. The element-wise system above can be written compactly as the *normal equations*:

$$X^\top X\, \hat{\boldsymbol{\theta}} \; = \; X^\top \mathbf{y}.$$

If $X^\top X$ is invertible (e.g., no exact multicollinearity and n sufficiently large), the unique least-squares solution is

$$\boxed{\hat{\boldsymbol{\theta}} \; = \; (X^\top X)^{-1} X^\top \mathbf{y}}$$

and predictions for a new house with features $\tilde{\mathbf{x}}_*$ are $\hat{y}_* = \tilde{\mathbf{x}}_*^\top \hat{\boldsymbol{\theta}}$.

Remarks. If $X^\top X$ is singular or nearly singular (e.g., redundant features or $p{+}1 > n$), one can use the Moore–Penrose pseudoinverse $\hat{\boldsymbol{\theta}} = X^{+}\mathbf{y}$ or introduce regularization (ridge, lasso), topics we will develop later.

Normal Equations: Matrix Calculus Derivation

Setup. Let $X \in \mathbb{R}^{n\times(p+1)}$ be the design matrix (first column ones for the intercept), $\theta \in \mathbb{R}^{p+1}$ the parameter vector, and $y \in \mathbb{R}^n$ the response vector. The mean squared error (MSE) used in the notes is

$$L(\theta) \; = \; \frac{1}{n}\,\|y - X\theta\|_2^2 \; = \; \frac{1}{n}\,(y - X\theta)^\top (y - X\theta).$$

Minimizing $L(\theta)$ yields the least–squares estimator.

Step 1: Expand the quadratic (matrix form). Using $\|a\|^2 = a^\top a$ and bilinearity,

$$L(\theta) = \frac{1}{n}\Big(y^\top y - 2\, y^\top X\theta + \theta^\top X^\top X\theta\Big).$$

Step 2: Differentiate term by term with respect to θ. We will use the following standard vector–matrix derivative identities (for constant A, b, c and vector variable θ):

1. $\nabla_\theta\, c = 0$.
2. $\nabla_\theta\,(b^\top\theta) = b$ and $\nabla_\theta\,(\theta^\top b) = b$.

3. $\nabla_\theta\,(\theta^\top A\theta) = (A + A^\top)\,\theta$; in particular, if A is symmetric, $\nabla_\theta\,(\theta^\top A\theta) = 2A\theta$.
4. Linearity of the gradient: $\nabla_\theta(\alpha f + \beta g) = \alpha\nabla_\theta f + \beta\nabla_\theta g$.

Apply these to each term (noting $X^\top X$ is symmetric):

$$\nabla_\theta L(\theta) = \frac{1}{n}\Big(\underbrace{\nabla_\theta\, y^\top y}_{=0} - 2\,\underbrace{\nabla_\theta(y^\top X\theta)}_{=X^\top y} + \underbrace{\nabla_\theta(\theta^\top X^\top X\theta)}_{=2X^\top X\theta}\Big) = \frac{1}{n}\Big(-2X^\top y + 2X^\top X\theta\Big).$$

Thus,

$$\nabla_\theta L(\theta) \;=\; \frac{2}{n}\left(X^\top X\theta - X^\top y\right) \;=\; -\frac{2}{n}\,X^\top(y - X\theta).$$

Step 3: Stationary condition $\Rightarrow$ normal equations. At the minimizer $\hat\theta$, set the gradient to zero:

$$\nabla_\theta L(\hat\theta) = 0 \quad\Longleftrightarrow\quad X^\top X\,\hat\theta \;=\; X^\top y,$$

which are the *normal equations*. If $X^\top X$ is invertible (e.g., X has full column rank), the unique least–squares solution is $\hat\theta = (X^\top X)^{-1}X^\top y$.

Step 4 (optional but informative): Hessian and convexity.

$$\nabla^2_\theta L(\theta) \;=\; \frac{2}{n}\,X^\top X \;\succeq\; 0.$$

Hence L is convex; if X has full column rank, $X^\top X \succ 0$ and the minimizer is unique.

Alternative (trace) derivation. Using $\|a\|^2 = \operatorname{tr}(aa^\top)$ and the identities $\nabla_\theta \operatorname{tr}(A\theta b^\top) = A^\top b$ and $\nabla_\theta \operatorname{tr}(\theta^\top A\theta) = (A + A^\top)\theta$:

$$L(\theta) = \frac{1}{n}\operatorname{tr}\big[(y - X\theta)(y - X\theta)^\top\big] = \frac{1}{n}\Big(\operatorname{tr}(yy^\top) - 2\operatorname{tr}(y\theta^\top X^\top) + \operatorname{tr}(\theta^\top X^\top X\theta)\Big),$$

so

$$\nabla_\theta L(\theta) = \frac{1}{n}\Big(0 - 2X^\top y + 2X^\top X\theta\Big) = \frac{2}{n}\,(X^\top X\theta - X^\top y),$$

leading again to $X^\top X\hat\theta = X^\top y$.

2.5. Gradient Descent for Linear Regression

The closed-form solution derived above is elegant and efficient when $X^\top X$ is moderate in size and invertible. However, in many modern machine learning settings:

- the number of features p can be very large (up to millions),
- the design matrix X may not fit in memory,
- or the model may be non-linear in its parameters (e.g. neural networks),

so that computing $(X^\top X)^{-1}$ is either infeasible or prohibitively expensive. In these cases, we resort to *iterative optimization algorithms*. The most basic and widely used is **gradient descent**.

Idea of gradient descent

Gradient descent is a simple iterative algorithm to minimize a differentiable loss function $\mathcal{L}(\boldsymbol{\theta})$:

1. Start from an initial guess $\boldsymbol{\theta}^{(0)}$.
2. At each iteration t, update the parameters by moving in the negative gradient direction:
$$\boldsymbol{\theta}^{(t+1)} \;=\; \boldsymbol{\theta}^{(t)} - \eta \nabla \mathcal{L}(\boldsymbol{\theta}^{(t)}),$$
where $\eta > 0$ is the *learning rate*.
3. Repeat until convergence (loss stops decreasing or parameters stabilize).

The gradient points in the direction of steepest increase of the loss; moving against it decreases the loss.

Gradient descent for linear regression

Recall the MSE loss:
$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n}\Big(y_i - \tilde{\mathbf{x}}_i^\top \boldsymbol{\theta}\Big)^2.$$

We computed earlier that the gradient is

$$\nabla \mathcal{L}(\boldsymbol{\theta}) = -\frac{2}{n}\sum_{i=1}^{n}\tilde{\mathbf{x}}_i\Big(y_i - \tilde{\mathbf{x}}_i^\top \boldsymbol{\theta}\Big) = -\frac{2}{n}X^\top\big(\mathbf{y} - X\boldsymbol{\theta}\big).$$

Therefore, the gradient descent update rule is

$$\boldsymbol{\theta}^{(t+1)} \;=\; \boldsymbol{\theta}^{(t)} + \frac{2\eta}{n}X^\top\big(\mathbf{y} - X\boldsymbol{\theta}^{(t)}\big).$$

Interpretation and practice

- The term $\mathbf{y} - X\boldsymbol{\theta}^{(t)}$ are the *residuals* at step t.
- Multiplying by $X^\top$ accumulates the correlations between residuals and features.
- The update therefore adjusts each coefficient proportionally to how much its feature is still correlated with the current errors.

Why gradient descent matters

For the linear regression problem, gradient descent is not strictly necessary because we already have a closed-form solution. However:

- It scales better to very high-dimensional problems, where computing a matrix inverse is impractical.
- It generalizes to *non-linear models* (e.g. logistic regression, neural networks) where no closed-form solution exists.
- Variants like *stochastic gradient descent* (SGD) update parameters using only a subset (or a single point) of the data at each step, making them extremely efficient for large datasets.

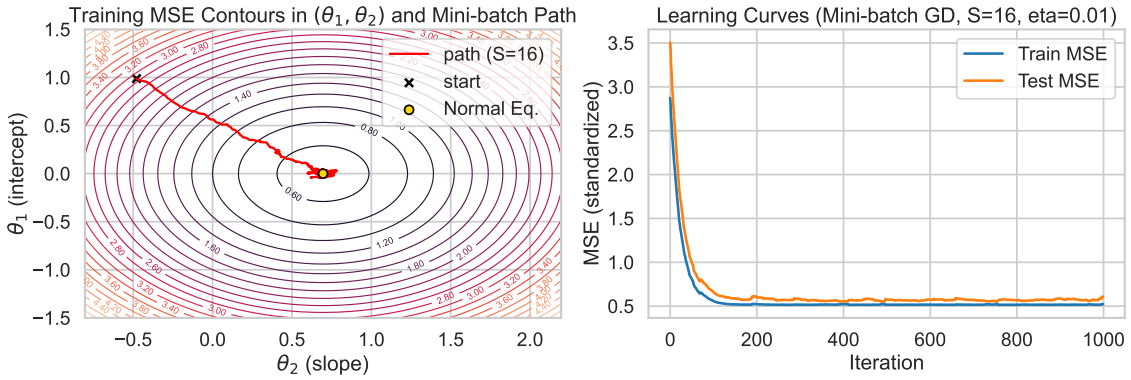


Figure 2.1.: Illustration of mini-batch gradient descent for fitting a linear model to predict house price from living area. **Left:** Contours of the training mean squared error (MSE) as a function of the parameters (θ_1, θ_2) (intercept and slope), with the path of the algorithm overlaid. The path shows how successive updates move the parameters toward the optimum. **Right:** Evolution of the training and test MSE as a function of the number of iterations, showing convergence of the algorithm and the generalization behavior.

Takeaway. Linear regression serves as the simplest playground to illustrate gradient descent, but the true power of the method emerges in complex, non-linear models where analytical solutions are impossible. Mastering gradient descent is therefore essential for modern machine learning.

2.6. Stochastic Gradient Descent (SGD)

Gradient descent requires evaluating the gradient of the loss over the *entire dataset* at each iteration:

$$\nabla \mathcal{L}(\boldsymbol{\theta}) \;=\; -\frac{2}{n} X^\top (\mathbf{y} - X\boldsymbol{\theta}).$$

When n is very large (millions of houses, billions of samples in modern applications), this computation can be prohibitively expensive per iteration.

Estimating the mean gradient

Note that the full gradient is the average of the per-sample gradients:

$$\nabla \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^{n} \nabla \ell_i(\boldsymbol{\theta}), \quad \text{where } \ell_i(\boldsymbol{\theta}) = \left(y_i - \tilde{\mathbf{x}}_i^\top \boldsymbol{\theta}\right)^2.$$

Thus, computing the gradient of the whole dataset is equivalent to taking a mean over n vectors. The key idea of **stochastic gradient descent (SGD)** is that we do not need to compute the full mean at every step: a good *estimate* of the mean is enough to move the parameters in a useful direction.

SGD with a single sample

In its simplest form, SGD picks one training point $(\tilde{\mathbf{x}}_i, y_i)$ uniformly at random and updates:

$$\boldsymbol{\theta}^{(t+1)} \;=\; \boldsymbol{\theta}^{(t)} - \eta \, \nabla \ell_i(\boldsymbol{\theta}^{(t)}),$$

where

$$\nabla \ell_i(\boldsymbol{\theta}) = -2\,\tilde{\mathbf{x}}_i\big(y_i - \tilde{\mathbf{x}}_i^\top \boldsymbol{\theta}\big).$$

This is an *unbiased* estimate of the true gradient, but it is noisy because it uses only one sample.

Bias and Variance of an Estimator

When we use a random sample (e.g. in stochastic gradient descent), the gradient we compute is a *random variable*. To understand its quality as an estimate of the true gradient, we introduce two key concepts.

Bias. Let $\hat{g}$ be an estimator of some true quantity g. The *bias* of $\hat{g}$ is

$$\mathrm{Bias}[\hat{g}] \;=\; \mathbb{E}[\hat{g}] - g.$$

An estimator is called *unbiased* if $\mathbb{E}[\hat{g}] = g$. This means that on average, over repeated samples, the estimator hits the correct value.

Variance. The *variance* of $\hat{g}$ quantifies how much the estimate fluctuates around its mean:

$$\mathrm{Var}[\hat{g}] \;=\; \mathbb{E}\big[\,(\hat{g} - \mathbb{E}[\hat{g}])^2\,\big].$$

High variance means the estimator is unstable: different samples may give very different estimates, even if it is unbiased.

Interpretation in SGD.

Mini-batch SGD

A common compromise is to use a *mini-batch* of m randomly selected points, with $1 \ll m \ll n$. The update is then

$$\boldsymbol{\theta}^{(t+1)} \;=\; \boldsymbol{\theta}^{(t)} - \eta \cdot \frac{1}{m} \sum_{i \in \mathcal{B}_t} \nabla \ell_i(\boldsymbol{\theta}^{(t)}),$$

where $\mathcal{B}_t$ is the set of indices in the mini-batch. This yields a gradient estimate with reduced variance compared to a single sample, but at a fraction of the cost of the full dataset.

Effect of learning rate and mini-batch size

- **Learning rate η:**
  - If η is too large, updates may overshoot and cause divergence.
  - If η is too small, convergence will be extremely slow.
  - Practical solutions include using *learning rate schedules* (decreasing η over time) or adaptive methods such as AdaGrad, RMSprop, and Adam.
- **Mini-batch size m:**
  - Small m (e.g. 1–32) produces noisy gradients but allows frequent updates and can escape shallow local minima.
  - Large m reduces noise but increases computational cost per update and may get stuck in narrow minima.

– In practice, m is chosen to balance statistical efficiency (low variance estimates) and computational efficiency (parallelization on GPUs).

Practical issues and remedies

- **Noisy updates:** Small batches introduce variance, which can hinder convergence. Remedies include increasing m or decaying η.
- **Convergence to sharp minima:** Large batches may find solutions that generalize poorly. Smaller batches encourage exploration and often yield flatter, more generalizable solutions.
- **Learning rate tuning:** Properly setting or adapting η is critical; adaptive methods (Adam, RMSprop) automatically adjust effective learning rates.

Takeaway. Stochastic gradient descent is the workhorse of modern machine learning. It replaces the exact but expensive full gradient with an approximate gradient computed on a small random subset of data. This stochasticity not only makes optimization scalable but can also improve generalization by preventing overfitting to training data.

Notes. Some authors insert $\frac{1}{2n}$ in $L(\theta)$ to cancel the factor 2 in the gradient. Here we keep the $\frac{1}{n}$ as in the lecture notes; both yield the same minimizer.

2.7. Evaluation Metrics

So far we have introduced linear regression as a model and described how to fit it. Once a model is trained, however, we must assess how well it performs on data that were not used for fitting. Evaluation metrics provide a quantitative way to measure predictive performance and to compare alternative models.

Mean Squared Error (MSE)

The most common metric for regression is the *mean squared error* (MSE):

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2,$$

where y_i are the observed values and $\hat{y}_i$ are the model predictions. MSE penalizes large deviations more strongly than small ones, due to the quadratic term, and has the convenient property of being minimized by the least-squares estimator.

Root Mean Squared Error (RMSE)

The square root of the MSE,

$$\text{RMSE} = \sqrt{\text{MSE}},$$

brings the error back to the same units as the target variable. RMSE is often easier to interpret, since it represents the typical magnitude of prediction errors.

Mean Absolute Error (MAE)

Another widely used metric is the *mean absolute error* (MAE):

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|.$$

MAE is more robust to outliers than MSE, because errors grow linearly instead of quadratically. It captures the average absolute deviation between predictions and true values.

Coefficient of Determination (R^2)

A different perspective is offered by the *coefficient of determination*, or R^2:

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2},$$

where $\bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i$ is the sample mean of the observed values. The denominator represents the total variation of the data around their empirical mean $\bar{y}$. In the population setting this would correspond to $\mu = \mathbb{E}[Y]$, but since μ is unknown we replace it with its observable estimate $\bar{y}$. Thus, R^2 quantifies the fraction of variance in the data (relative to the sample mean) that is captured by the model. An R^2 close to 1 indicates an excellent fit, while $R^2 = 0$ means the model is no better than predicting the constant $\bar{y}$.

Correlation coefficient (ρ)

Another useful metric is the (Pearson) correlation coefficient between the true and predicted values:

$$\rho(y, \hat{y}) = \frac{\sum_{i=1}^{n}(y_i - \bar{y})(\hat{y}_i - \bar{\hat{y}})}{\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}\sqrt{\sum_{i=1}^{n}(\hat{y}_i - \bar{\hat{y}})^2}},$$

where $\bar{y}$ and $\bar{\hat{y}}$ are the sample means of the observed and predicted values, respectively. The correlation coefficient lies in the interval $\rho \in [-1, 1]$ and measures the strength of the linear association between predictions and targets. Values of ρ close to 1 indicate a strong positive linear relationship, values close to 0 indicate little or no linear relationship, and values close to -1 indicate a strong negative linear relationship.

Sometimes we consider the squared correlation coefficient $\rho^2 = \big(\rho(y, \hat{y})\big)^2$, which lies in $[0, 1]$. This quantity represents the proportion of the variance in the observed values y that can be explained by a linear relationship with the predictions $\hat{y}$. It is closely related to R^2, though not identical: R^2 is based on comparing residual variance to total variance, while ρ^2 directly measures the strength of the linear correlation between y and $\hat{y}$.

Train vs. test performance

It is essential to compute these metrics not only on the training data, but also on a held-out *test set* that was not used for fitting. This provides an unbiased estimate of generalization performance and guards against overfitting. Cross-validation, discussed later, is a systematic way to evaluate models on multiple train/test splits.

Takeaway. For regression problems, MSE, RMSE, and MAE measure the size of the errors, R^2 quantifies performance relative to the baseline predictor $\bar{y}$, and explained variance measures how much of the variability in y remains in the residuals. The correlation coefficient

ρ assesses the strength and direction of the linear relationship between true and predicted values, while ρ^2 indicates the proportion of variance in y linearly explained by $\hat{y}$. Together, these metrics provide complementary insights into both the magnitude of prediction errors and the quality of the linear relationship between targets and predictions.

2.8. A Probabilistic View of Linear Regression

So far we have motivated linear regression from a geometric and optimization perspective: fit a linear function by minimizing the mean squared error. A deeper perspective emerges when we view the learning problem through the lens of probability.

From deterministic to probabilistic modeling

In the deterministic formulation we assumed

$$y_i \;=\; \tilde{\mathbf{x}}_i^\top \boldsymbol{\theta} + \varepsilon_i,$$

with "noise" terms ε_i capturing variability not explained by the linear model. In the probabilistic formulation, we model ε_i as a random variable. The most common choice is

$$\varepsilon_i \sim \mathcal{N}(0, \sigma^2),$$

i.e. Gaussian noise with mean zero and variance σ^2. This implies

$$y_i \,|\, \tilde{\mathbf{x}}_i \;\sim\; \mathcal{N}(\tilde{\mathbf{x}}_i^\top \boldsymbol{\theta}, \sigma^2).$$

Independent and identically distributed (i.i.d.) data

We now assume that the n data points $\{(\tilde{\mathbf{x}}_i, y_i)\}_{i=1}^n$ are *independent and identically distributed (i.i.d.)*:

- **Independent:** the outcome of one observation does not influence the others.
- **Identically distributed:** all observations are generated from the same distribution $p(y \,|\, \tilde{\mathbf{x}}, \boldsymbol{\theta}, \sigma^2)$.

This assumption allows us to write the likelihood of the entire dataset as a product of the individual probabilities:

$$p(\mathbf{y} \,|\, X, \boldsymbol{\theta}, \sigma^2) = \prod_{i=1}^{n} p(y_i \,|\, \tilde{\mathbf{x}}_i, \boldsymbol{\theta}, \sigma^2).$$

Likelihood under Gaussian noise

Using the Gaussian model for each y_i,

$$p(y_i \,|\, \tilde{\mathbf{x}}_i, \boldsymbol{\theta}, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2} \left(y_i - \tilde{\mathbf{x}}_i^\top \boldsymbol{\theta}\right)^2 \right),$$

the likelihood of the dataset is

$$p(\mathbf{y} \,|\, X, \boldsymbol{\theta}, \sigma^2) = \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2} \left(y_i - \tilde{\mathbf{x}}_i^\top \boldsymbol{\theta}\right)^2 \right).$$

From likelihood to MSE

It is easier to maximize the *log-likelihood*:

$$\log p(\mathbf{y} \mid X, \boldsymbol{\theta}, \sigma^2) = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}\left(y_i - \tilde{\mathbf{x}}_i^\top \boldsymbol{\theta}\right)^2.$$

The first term $-\frac{n}{2}\log(2\pi\sigma^2)$ depends on σ^2 but not on $\boldsymbol{\theta}$. Therefore, maximizing the log-likelihood with respect to $\boldsymbol{\theta}$ is equivalent to minimizing

$$\sum_{i=1}^{n}\left(y_i - \tilde{\mathbf{x}}_i^\top \boldsymbol{\theta}\right)^2.$$

If we fix $\sigma^2 = 1$ (or treat it as a constant prefactor), the log-likelihood optimization problem is exactly the same as minimizing the mean squared error (MSE). Thus, MSE has a probabilistic justification: it arises naturally from the Gaussian likelihood.

Power of the probabilistic formulation

The probabilistic perspective offers several advantages:

- **Uncertainty quantification:** Instead of producing only point estimates, we can quantify the uncertainty of predictions through the variance σ^2 or by using Bayesian extensions.
- **Principled model comparison:** Different noise models lead to different loss functions (e.g. absolute error for Laplace noise), allowing us to tailor regression to the data-generating process.
- **Connection to generative modeling:** By specifying a probability distribution for the data, we move from "just predicting" to building a *generative model* that can simulate new samples. This viewpoint extends naturally to modern generative AI, where the goal is to model complex high-dimensional data distributions (images, text, protein sequences).
- **Extensibility:** The same probabilistic principles apply to more complex settings such as classification (via logistic regression and probabilistic discriminative models), density estimation, and latent-variable models.

Takeaway. Viewing linear regression probabilistically explains why the squared error is a natural loss, deriving directly from the Gaussian likelihood under i.i.d. assumptions. This not only provides a theoretical foundation for MSE but also opens the door to more powerful extensions, including uncertainty quantification, alternative noise models, and the generative modeling framework underlying modern AI.

Gaussian distribution from the maximum entropy principle

Why do we so often assume Gaussian noise? One justification comes from the *maximum entropy principle*, which selects the least informative distribution consistent with known constraints.

Setup

The entropy of a continuous distribution with density $p(y)$ is

$$H[p] = -\int_{-\infty}^{\infty} p(y)\, \log p(y)\, dy.$$

We want to find the distribution $p(y)$ that maximizes entropy subject to the following constraints:

1. Normalization: $\int p(y)\, dy = 1$.
2. Fixed mean: $\int y\, p(y)\, dy = \mu$.
3. Fixed variance: $\int (y-\mu)^2\, p(y)\, dy = \sigma^2$.

Lagrangian formulation

Introduce multipliers $\lambda_0, \lambda_1, \lambda_2$ for these constraints. The functional to maximize is

$$\begin{aligned}\mathcal{L}[p] \;=\; &-\int p(y)\, \log p(y)\, dy + \lambda_0\Big(\int p(y)\, dy - 1\Big) \\ &+\lambda_1\Big(\int y\, p(y)\, dy - \mu\Big) + \lambda_2\Big(\int (y-\mu)^2 p(y)\, dy - \sigma^2\Big).\end{aligned}$$

Taking the functional derivative with respect to $p(y)$ gives

$$\frac{\delta \mathcal{L}}{\delta p(y)} = -\big(1 + \log p(y)\big) + \lambda_0 + \lambda_1 y + \lambda_2 (y-\mu)^2 = 0.$$

Solution

Rearranging yields

$$p(y) \propto \exp\big(\lambda_1 y + \lambda_2 (y-\mu)^2\big)\,.$$

The quadratic term in y ensures the density is Gaussian. Solving for the multipliers under the constraints leads to

$$p(y) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y-\mu)^2}{2\sigma^2}\right).$$

Interpretation

Thus, the Gaussian distribution is the maximum entropy distribution consistent with a specified mean and variance. In other words, among all possible distributions with given first and second moments, the Gaussian makes the fewest additional assumptions. This explains its privileged role as a natural noise model in regression and many other contexts.

3. Logistic Regression

3.1. Motivation: Why Linear Regression Fails for Classification

In many applications, the target variable y is not continuous but *categorical*. For simplicity we first consider the binary case $y \in \{0, 1\}$, where the goal is to decide between two possible classes (e.g., spam vs. not spam, malignant vs. benign, dog vs. cat).

A first idea might be to reuse the linear regression framework:

$$\hat{y} = \theta^\top x,$$

and interpret $\hat{y}$ as a prediction of the class label. This, however, presents several fundamental problems:

1. **Unbounded predictions.** Linear regression outputs $\hat{y} \in (-\infty, \infty)$, whereas probabilities must lie in the interval $[0, 1]$. There is no guarantee that $\hat{y}$ respects this range, making direct probabilistic interpretation impossible.

2. **Poor calibration.** Even if we threshold $\hat{y}$ at 0.5 to make class predictions, values outside $[0, 1]$ lose meaning: a prediction of -10 or $+25$ cannot be interpreted as a probability.

3. **Inappropriate loss.** Linear regression minimizes the mean squared error (MSE), which assumes Gaussian noise around the target. In classification problems the natural data-generating process is Bernoulli (success/failure), not Gaussian. Using MSE leads to inefficient estimation and biased decision boundaries.

These issues motivate a new model that directly respects the probabilistic structure of classification: mapping a linear predictor $\theta^\top x$ into the $[0, 1]$ interval, and deriving an appropriate likelihood for binary outcomes. This leads to the *logistic regression* model, based on the logistic (sigmoid) function introduced next.

3.2. The Logistic (Sigmoid) Function

To build a model suitable for binary classification, we need a way to map the real-valued linear predictor

$$z = \theta^\top x$$

into the interval $[0, 1]$, so that it can be interpreted as a probability. The function that accomplishes this is the *logistic*, or *sigmoid*, function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Properties

The logistic function has several important properties:

- **Range:** $\sigma(z) \in (0, 1)$ for all $z \in \mathbb{R}$.
- **S-shape:** It is monotonically increasing and has the characteristic "S" shape, smoothly transitioning from values close to 0 (when $z \to -\infty$) to values close to 1 (when $z \to +\infty$).
- **Symmetry:** $\sigma(-z) = 1 - \sigma(z)$.
- **Derivative:** The derivative has a convenient form:

$$\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big),$$

  which will be useful for optimization via gradient methods.

Interpretation

By composing the linear predictor with the logistic function, we define the conditional probability of class membership as

$$P(y = 1 \mid x) = \sigma(\theta^\top x).$$

This makes logistic regression a *probabilistic classifier*: predictions are not just hard labels, but probabilities that can be thresholded (e.g. at 0.5) or compared according to the application requirements.

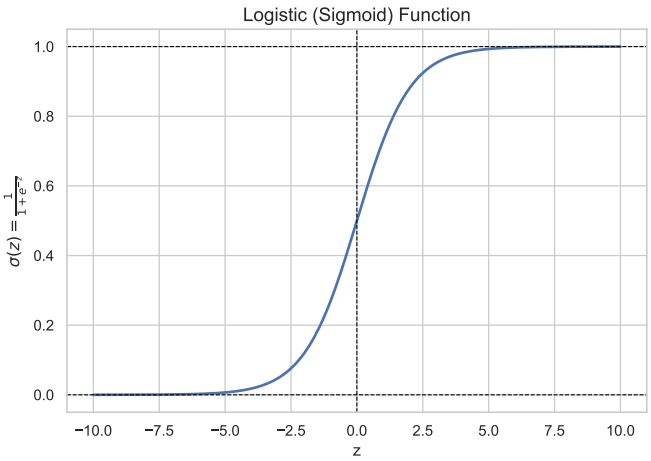


Figure 3.1.: The logistic (sigmoid) function $\sigma(z) = \frac{1}{1+e^{-z}}$. The curve maps real numbers $z \in (-\infty, \infty)$ into the probability interval $(0, 1)$.

3.3. Model Formulation

Logistic regression builds directly on the linear models we studied earlier, but interprets their output through the logistic function to obtain probabilities.

Linear predictor

We begin with a linear score function,

$$z = \theta^\top x = \theta_0 + \theta_1 x_1 + \cdots + \theta_p x_p,$$

where θ_0 is the intercept and $\theta_1, \ldots, \theta_p$ are the feature weights. The quantity z can be viewed as a measure of evidence in favor of class 1.

Probability model

Instead of treating z as a direct prediction, we pass it through the logistic function:

$$P(y = 1 \mid x) = \sigma(\theta^\top x) = \frac{1}{1 + e^{-\theta^\top x}},$$

so that the output is always between 0 and 1 and admits a natural probabilistic interpretation. By complementarity,

$$P(y = 0 \mid x) = 1 - \sigma(\theta^\top x).$$

This defines a conditional distribution $P(y \mid x; \theta)$ for binary labels.

Geometry of the decision boundary

The decision rule in logistic regression is to predict class 1 (malignant) whenever the model assigns probability at least 0.5:

$$\hat{y} = \begin{cases} 1 & \text{if } P(y = 1 \mid x) \geq 0.5, \\ 0 & \text{otherwise.} \end{cases}$$

Since the logistic function satisfies $\sigma(z) = 0.5$ exactly when $z = 0$, the *decision boundary* is defined by the set of points $x = (x_1, x_2, \ldots, x_p)$ that satisfy

$$\theta^\top x = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_p x_p = 0.$$

This boundary is always a linear surface in feature space:

- With one feature it reduces to a *threshold* on x_1.
- With two features it is a *straight line*.
- With three or more features it generalizes to a *hyperplane*.

For the case of two features, say $x_1 =$ `area_mean` and $x_2 =$ `smoothness_mean` from the breast cancer dataset, the boundary is given explicitly by

$$\theta_0 + \theta_1 x_1 + \theta_2 x_2 = 0,$$

or equivalently,

$$x_2 = -\frac{\theta_0}{\theta_2} - \frac{\theta_1}{\theta_2} x_1.$$

This is the equation of a straight line with slope $-\theta_1/\theta_2$ and intercept $-\theta_0/\theta_2$. Points on one side of the line correspond to $\theta^\top x > 0$, which implies $P(y = 1 \mid x) > 0.5$ and a malignant prediction. Points on the other side correspond to $\theta^\top x < 0$, giving $P(y = 1 \mid x) < 0.5$ and a benign prediction.

The figure below illustrates this geometry: the black line corresponds to the decision boundary $\theta^\top x = 0$, while the color shading represents the predicted probability of malignancy. Regions farther from the line correspond to more confident predictions.

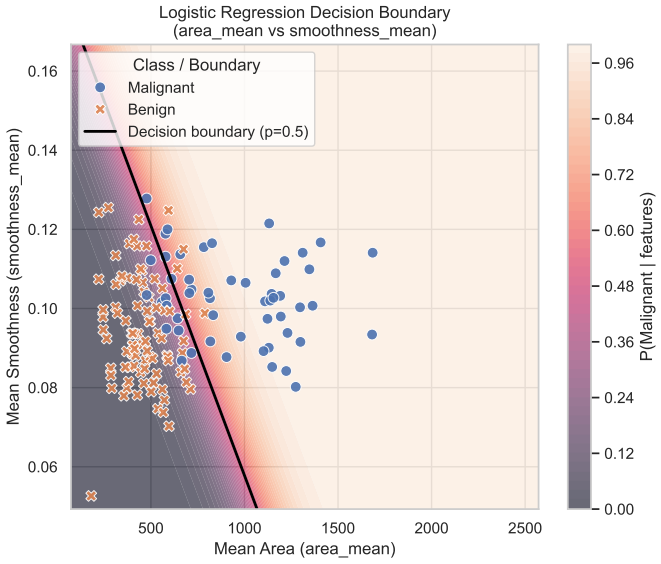


Figure 3.2.: Decision boundary for logistic regression trained on the breast cancer dataset using two features: mean area (x_1) and mean smoothness (x_2). The separating line corresponds to $\theta^\top x = 0$. Shading indicates predicted probability of malignancy.

3.4. Loss Function: Cross-Entropy

To estimate the parameters of a logistic regression model we need to define a suitable loss function. Because the outputs are binary ($y \in \{0, 1\}$), the natural probabilistic model is a Bernoulli distribution.

Bernoulli likelihood

Given a data point (x_i, y_i) with predicted probability $p_i = P(y = 1 \mid x_i) = \sigma(\theta^\top x_i)$, the probability of observing y_i is

$$P(y_i \mid x_i; \theta) = p_i^{y_i}(1 - p_i)^{1-y_i}.$$

Assuming n independent samples, the likelihood is

$$L(\theta) = \prod_{i=1}^{n} p_i^{y_i}(1 - p_i)^{1-y_i}.$$

Log-likelihood and cross-entropy

It is more convenient to maximize the log-likelihood:

$$\ell(\theta) = \log L(\theta) = \sum_{i=1}^{n} \big[y_i \log p_i + (1 - y_i) \log(1 - p_i)\big].$$

Instead of maximizing $\ell(\theta)$, we minimize its negative:

$$\mathcal{L}(\theta) = -\ell(\theta) = -\sum_{i=1}^{n} \big[y_i \log p_i + (1 - y_i) \log(1 - p_i)\big].$$

This expression is known as the *cross-entropy loss* for binary classification:

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^{n} \Big(- y_i \log p_i - (1 - y_i) \log(1 - p_i)\Big).$$

Comparison with MSE

One might wonder why not simply minimize the mean squared error (MSE), as in linear regression:

$$\text{MSE}(\theta) = \frac{1}{n}\sum_{i=1}^{n}(y_i - p_i)^2.$$

While MSE can be applied, it is not well aligned with the Bernoulli data-generating process and often leads to slower convergence and poorer decision boundaries. Cross-entropy, on the other hand:

- Arises directly from the principle of maximum likelihood under a Bernoulli model.
- Strongly penalizes confident but wrong predictions (e.g., $p_i \approx 0$ when $y_i = 1$).
- Provides well-calibrated probabilities and better optimization properties for classification.

For these reasons, cross-entropy (also called log-loss) is the standard objective function for logistic regression and many other classification models.

Why is it called cross-entropy?

In information theory, the *entropy* of a distribution $q(y)$ is

$$H(q) = -\sum_{y} q(y)\log q(y),$$

which measures the average information (in bits, if using $\log_2$) needed to encode samples from q. Given a true distribution $q(y)$ and a model distribution $p(y)$, the *cross-entropy* is defined as

$$H(q,p) = -\sum_{y} q(y)\log p(y).$$

In logistic regression, each label induces a one-hot true distribution: for a sample (x_i, y_i),

$$q(y) = \begin{cases} 1 & y = y_i, \\ 0 & \text{otherwise.} \end{cases}$$

The model predicts $p(y=1) = p_i$ and $p(y=0) = 1-p_i$. Plugging into the definition,

$$H(q,p) = -\Big[y_i \log p_i + (1-y_i)\log(1-p_i)\Big],$$

which is exactly the logistic regression loss for that data point.

Interpretation. Cross-entropy quantifies the inefficiency of encoding samples from the true distribution q using a code optimized for p. Minimizing cross-entropy therefore encourages the model distribution p to match the true labels as closely as possible.

3.5. Optimization with Gradient Descent

Unlike linear regression, logistic regression does not admit a closed-form solution for its parameters. The cross-entropy loss

$$\mathcal{L}(\theta) = -\sum_{i=1}^{n} \Big[y_i \log p_i + (1 - y_i) \log(1 - p_i) \Big], \quad \text{where } p_i = \sigma(\theta^\top x_i),$$

is a convex function of θ, but cannot be minimized by solving a system of linear equations as in the normal equations of Chapter 2.

Gradient of the log-likelihood

To minimize the loss we compute its gradient. For a single data point (x_i, y_i), the contribution to the log-likelihood is

$$\ell_i(\theta) = y_i \log p_i + (1 - y_i) \log(1 - p_i).$$

Differentiating with respect to θ gives

$$\nabla_\theta \ell_i(\theta) = (y_i - p_i)\, x_i.$$

Summing over the dataset, the gradient of the log-likelihood is

$$\nabla_\theta \ell(\theta) = \sum_{i=1}^{n} (y_i - p_i)\, x_i,$$

and the gradient of the loss (negative log-likelihood) is

$$\nabla_\theta \mathcal{L}(\theta) = -\sum_{i=1}^{n} (y_i - p_i)\, x_i.$$

Gradient descent update

We update the parameters by moving against the gradient:

$$\theta^{(t+1)} = \theta^{(t)} - \eta\, \nabla_\theta \mathcal{L}(\theta^{(t)}),$$

where $\eta > 0$ is the learning rate. This ensures that each step reduces the loss until convergence to the global optimum (guaranteed by convexity).

Connection to SGD

In practice, we often replace the full gradient with an estimate based on a subset (mini-batch) of the data:

$$\theta^{(t+1)} = \theta^{(t)} - \eta\, \frac{1}{m} \sum_{i \in B_t} (y_i - p_i)\, x_i,$$

where B_t is a mini-batch of size m. This is exactly the *stochastic gradient descent* (SGD) method introduced in Chapter 2, now applied to logistic regression. SGD makes optimization scalable to large datasets and often improves generalization.

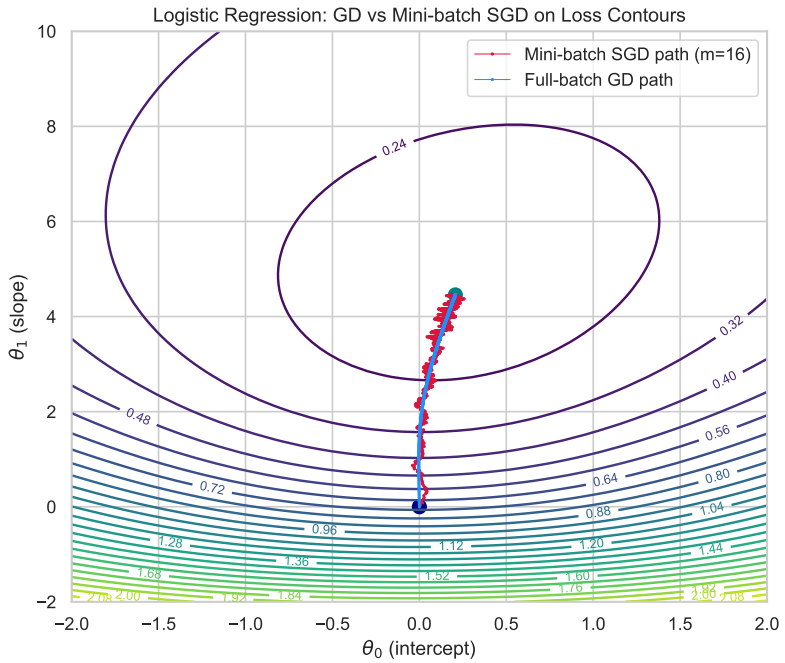


Figure 3.3.: Illustration of gradient descent for logistic regression. The convex surface of the cross-entropy loss ensures convergence to the unique global optimum. Mini-batch SGD follows a noisier path but reaches the same solution efficiently on large datasets.

3.6. Probabilistic Interpretation

Logistic regression is best understood as a *probabilistic discriminative model*. Rather than modeling how the data are generated, it directly models the conditional probability of the class label given the input features:

$$P(y \mid x; \theta) = \begin{cases} \sigma(\theta^\top x), & y = 1, \\ 1 - \sigma(\theta^\top x), & y = 0. \end{cases}$$

This means that logistic regression focuses solely on learning the *decision boundary* that separates classes, without making any assumptions about the distribution of the input features themselves.

Discriminative vs. generative models

To place logistic regression in context, it is useful to contrast it with generative classifiers:

- **Discriminative models** (e.g. logistic regression) model the conditional probability $P(y \mid x)$ directly. Their goal is to separate classes and provide accurate predictions of the labels given the features. They make fewer assumptions about the data distribution and often yield better predictive accuracy.
- **Generative models** (e.g. Linear Discriminant Analysis and Naive Bayes, which we will study later) model the joint distribution $P(x, y)$ by specifying $P(x \mid y)$ and $P(y)$. They then use Bayes' rule to compute

$$P(y \mid x) = \frac{P(x \mid y)P(y)}{P(x)}.$$

Generative approaches can be more interpretable in terms of the underlying data distribution, and they allow sampling or data generation, but they rely on stronger distributional assumptions (e.g. Gaussian features in LDA, conditional independence in Naive Bayes).

Takeaway

Logistic regression is a discriminative model that provides a simple, convex, and well-calibrated way to estimate class probabilities. It does not attempt to explain how the features are distributed, but rather optimizes directly for predictive performance. In the following chapters, we will see how this contrasts with generative classifiers such as LDA and Naive Bayes, which take a complementary perspective.

3.7. Evaluation Metrics

The performance of a classifier cannot be assessed solely by its training loss. Once we have trained a logistic regression model, we must evaluate how well it generalizes to unseen data. Logistic regression produces *probabilistic outputs*, and this enables a wide range of evaluation metrics beyond simple accuracy.

Confusion matrix

A binary classifier can be summarized by the confusion matrix shown in Figure 3.4. The matrix compares the predicted class labels with the true labels:

- **True Positive (TP):** the model correctly predicts the positive class (e.g. malignant tumor predicted as malignant).
- **False Negative (FN):** the model incorrectly predicts negative when the actual class is positive (e.g. malignant tumor predicted as benign). This corresponds to a *Type II error*.
- **False Positive (FP):** the model incorrectly predicts positive when the actual class is negative (e.g. benign tumor predicted as malignant). This corresponds to a *Type I error*.
- **True Negative (TN):** the model correctly predicts the negative class (e.g. benign tumor predicted as benign).

From these four basic outcomes we can define the main performance metrics.

Accuracy

The most common measure is *accuracy*, the proportion of correct predictions:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}.$$

Although accuracy is intuitive, it can be misleading in imbalanced datasets. For example, if only 5% of patients have cancer, predicting "healthy" for everyone yields 95% accuracy but zero clinical utility.

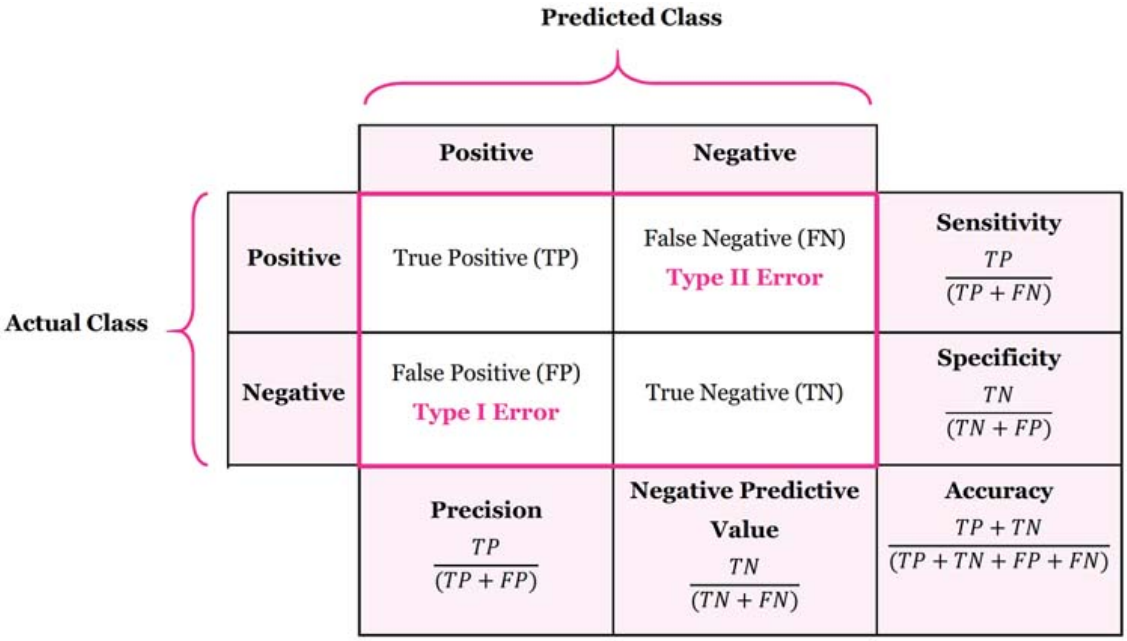


Figure 3.4.: Confusion matrix with associated evaluation metrics.

Precision and Recall

Two key metrics for the positive class are *precision* and *recall*:

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall (Sensitivity)} = \frac{TP}{TP + FN}.$$

- Precision answers: *Of all patients predicted malignant, how many truly are malignant?*
- Recall answers: *Of all truly malignant patients, how many did we detect?*

Often there is a trade-off: increasing recall (detecting more positives) may lower precision (increasing false alarms). A useful summary measure is the *F1-score*, the harmonic mean of precision and recall:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

ROC Curve and AUC

Because logistic regression produces probabilities, we are not limited to a fixed threshold such as 0.5. Varying the decision threshold gives rise to the *Receiver Operating Characteristic (ROC) curve*, which plots:

$$\text{True Positive Rate (Recall)} = \frac{TP}{TP + FN} \quad \text{vs.} \quad \text{False Positive Rate} = \frac{FP}{FP + TN}.$$

The ROC curve illustrates the trade-off between sensitivity and specificity across all thresholds. A random classifier produces a diagonal line (AUC ≈ 0.5), while a perfect classifier reaches the top-left corner (AUC $= 1$). The *Area Under the Curve (AUC)* is a scalar summary that corresponds to the probability that the classifier ranks a random positive example above a random negative one.

Probabilistic outputs as a strength

The strength of logistic regression lies in its probabilistic outputs. Rather than forcing a single hard classification, the model produces calibrated probabilities. These can be:

- thresholded differently depending on the application (e.g. prioritize recall in medical screening, prioritize precision in spam filtering),
- used to compute ROC curves and AUC values, which characterize performance independently of any particular threshold,
- or combined with cost-sensitive rules if the consequences of misclassifying positives and negatives are asymmetric.

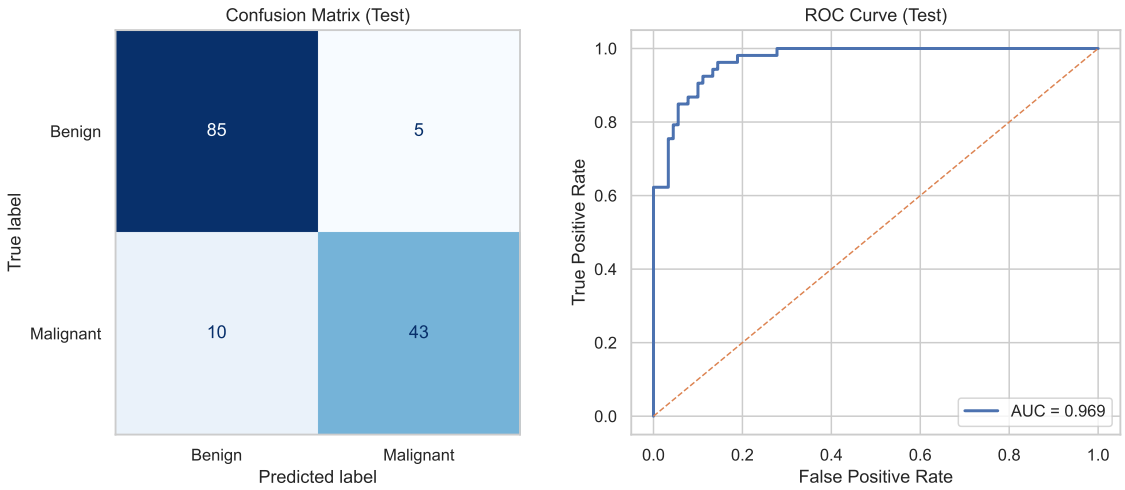


Figure 3.5.: Evaluation of logistic regression on the breast cancer dataset using two features (`area_mean` and `smoothness_mean`). The left panel shows the confusion matrix on the test set, with true positives, true negatives, false positives, and false negatives. The right panel displays the ROC curve, obtained by varying the classification threshold; the area under the curve (AUC) is 0.969, indicating excellent discriminative performance.

3.8. Extensions

Logistic regression, as presented so far, addresses binary classification problems. However, the framework can be extended in several important directions.

Multiclass classification via softmax

In problems with more than two classes, $y \in \{1, 2, \ldots, K\}$, a direct generalization of logistic regression is obtained using the *softmax function*. For each class k we define a linear predictor

$$z_k = \theta_k^\top x,$$

and map these scores to probabilities through

$$P(y = k \mid x) = \frac{\exp(z_k)}{\sum_{j=1}^{K} \exp(z_j)}.$$

This formulation ensures that all class probabilities are non-negative and sum to one. The model is trained by maximizing the multinomial log-likelihood, which generalizes the cross-entropy loss. The resulting classifier is often referred to as *multinomial logistic regression* or *softmax regression*.

Multiclass classification via softmax

In problems with more than two classes, $y \in \{1, 2, \ldots, K\}$, a direct generalization of logistic regression is obtained using the *softmax function*. For each class k we define a linear predictor

$$z_k = \theta_k^\top x,$$

and map these scores to probabilities through

$$P(y = k \mid x) = \frac{\exp(z_k)}{\sum_{j=1}^{K} \exp(z_j)}.$$

This formulation ensures that all class probabilities are non-negative and sum to one:

$$\sum_{k=1}^{K} P(y = k \mid x) = 1.$$

Multinomial log-likelihood

Suppose we have n independent training examples $\{(x_i, y_i)\}_{i=1}^n$, with $y_i \in \{1, \ldots, K\}$. The likelihood of the data under the model is

$$L(\Theta) = \prod_{i=1}^{n} \prod_{k=1}^{K} \Big[P(y = k \mid x_i)\Big]^{\mathbf{1}\{y_i = k\}},$$

where $\mathbf{1}\{y_i = k\}$ is an indicator function. Taking logs gives the multinomial log-likelihood:

$$\ell(\Theta) = \sum_{i=1}^{n} \sum_{k=1}^{K} \mathbf{1}\{y_i = k\} \log P(y = k \mid x_i),$$

where $\Theta = (\theta_1, \ldots, \theta_K)$ is the collection of parameter vectors.

This is equivalent to minimizing the *multiclass cross-entropy loss*, which generalizes the binary case.

Gradient for optimization

The loss function has no closed-form solution, so we use gradient-based optimization. Differentiating the log-likelihood with respect to the parameters θ_k gives:

$$\nabla_{\theta_k} \ell(\Theta) = \sum_{i=1}^{n} \Big[\mathbf{1}\{y_i = k\} - P(y = k \mid x_i)\Big] x_i.$$

Thus, the gradient takes the form of an "observed minus expected" count, scaled by the input vector. The parameters are updated using gradient ascent on the log-likelihood (or equivalently, gradient descent on the negative log-likelihood):

$$\theta_k^{(t+1)} = \theta_k^{(t)} + \eta \, \nabla_{\theta_k} \ell(\Theta^{(t)}),$$

where $\eta > 0$ is the learning rate. In practice, optimization is performed using stochastic or mini-batch gradient descent for efficiency.

This framework is commonly known as *multinomial logistic regression* or *softmax regression*, and it is the basis of the final layer in many neural networks for classification tasks.

Regularization

As in linear regression, logistic regression can suffer from overfitting, particularly when the number of features is large. A common solution is to add a *regularization term* to the loss function:

- **Ridge logistic regression (L2 penalty):**

$$\mathcal{L}(\theta) = -\sum_{i=1}^{n} \Big[y_i \log p_i + (1 - y_i) \log(1 - p_i) \Big] + \lambda \|\theta\|_2^2,$$

  where $\lambda > 0$ controls the strength of regularization. The L2 penalty shrinks coefficients toward zero, improving generalization while keeping all features in the model.

- **Lasso logistic regression (L1 penalty):**

$$\mathcal{L}(\theta) = -\sum_{i=1}^{n} \Big[y_i \log p_i + (1 - y_i) \log(1 - p_i) \Big] + \lambda \|\theta\|_1,$$

  where $\|\theta\|_1 = \sum_j |\theta_j|$. The L1 penalty encourages sparsity, effectively performing feature selection by driving some coefficients to exactly zero.

Regularization thus provides a powerful mechanism to handle high-dimensional datasets, prevent overfitting, and improve interpretability. In practice, both L1 and L2 regularization (or a combination, known as *elastic net*) are widely used.

These extensions demonstrate the flexibility of logistic regression: it can naturally handle multiclass classification through the softmax function, and it can be adapted to complex, high-dimensional settings via regularization. For these reasons, logistic regression remains a cornerstone of applied machine learning despite its conceptual simplicity.

4. Dimensionality Reduction and Clustering

4.1. Introduction

Up to now, we have focused on *supervised learning*, where the goal is to learn from labeled data. In regression and classification we had pairs (x_i, y_i) consisting of an input vector and a target label, and our task was to predict y from x. Supervision made evaluation straightforward: we could compare predictions to known answers and optimize accordingly.

In this chapter, we turn to the first models and algorithms of *unsupervised learning*. Here, we are given only the raw data $\{x_1, \ldots, x_n\}$, without labels or targets. The challenge is to uncover meaningful structure hidden in the data itself. This is inherently more difficult: there is no "ground truth" to guide the learning process, and many different explanations may be plausible. Nevertheless, unsupervised learning plays a central role in modern machine learning, allowing us to compress, visualize, and organize high-dimensional data, and to generate hypotheses in situations where no labels exist.

Why Unsupervised Learning is More Challenging

The difficulty of unsupervised learning comes from several sources:

- **No explicit target.** Unlike in supervised tasks, we cannot measure error against a label. Instead, we must decide what "structure" means (variance, distance, density, etc.).
- **Multiple plausible structures.** The same dataset may support several interpretations (e.g., grouping by shape or by color).
- **Evaluation without ground truth.** It is harder to decide objectively whether one solution is better than another. Often evaluation relies on indirect criteria such as interpretability, stability, or downstream predictive performance.

Despite these challenges, unsupervised learning is powerful because it helps reveal patterns that may not be apparent otherwise. To make progress, we often introduce the idea of *latent variables*.

Latent Variable Models: Continuous vs. Discrete Structure

A *latent variable model* assumes that the observed data are generated by some underlying hidden structure. The observed features are then explained as noisy or indirect manifestations of this structure. Two broad cases are particularly important: continuous latent variables and discrete latent variables. Each case leads to a different type of unsupervised method.

Continuous latent variables. When the hidden structure varies smoothly, data often lie close to a *low-dimensional manifold* embedded in a high-dimensional observation space. A

classical example comes from sequential or periodic processes, such as human gait or the progression of the cell cycle. Although each sample (an image of a walking person, or the transcriptome of a cell) has thousands of measured features, the essential variation can be described by a few continuous latent factors, such as the *phase* of the gait cycle or cell cycle.

The classical method for uncovering such structure is Principal Component Analysis (PCA). PCA seeks orthogonal directions (principal components) in the data that capture the largest variance. By projecting the high-dimensional observations onto the top principal components, we obtain a lower-dimensional representation that reveals the underlying continuous manifold. For instance, the cyclic trajectory of walking can appear as a loop in the first two principal components, making explicit the latent continuous variable (gait phase) that generates the observed images.

Discrete latent variables. In other situations, the hidden structure corresponds to categorical membership, where each data point belongs to one of several distinct groups or *clusters*. An example is provided by images of handwritten digits 0–9: although each digit image is high-dimensional, the essential latent variable is discrete, indicating which of the ten digits the image represents.

The most widely used method to capture this type of latent structure is **k-Means Clustering**. Here we assume that each observation is generated by first choosing a cluster index (a discrete latent variable), and then sampling features around a cluster centroid. The k-means algorithm alternates between assigning each point to the closest centroid (inferring the latent variable) and updating the centroids as the means of their assigned points. The result is a partition of the data into clusters that reflect the discrete hidden categories.

These two methods, PCA for continuous latent structure and k-means for discrete latent structure, illustrate the dual perspectives that underlie much of unsupervised learning. Although conceptually simple, they provide powerful tools for visualization, compression, and pattern discovery, and serve as building blocks for more advanced latent variable models.

4.2. Principal Component Analysis (PCA)

Geometric intuition

Imagine a cloud of points in two dimensions. The points are scattered around the origin, perhaps forming an elongated ellipse. The goal of PCA is to find a new axis, a direction through the origin, that best summarizes how the points are spread out.

Each point can be projected perpendicularly onto this axis, so that we can describe the data using only a single coordinate: how far along the axis the point falls. The projection captures the main trend of the data, while the distance between each point and its projection reflects how much information is lost. The best axis is the one that minimizes these perpendicular distances for all points, or equivalently, the one that captures as much of the overall spread of the cloud as possible.

In this way, PCA can be seen as a method that rotates our coordinate system so that its axes line up with the natural directions of greatest variation in the data. In two dimensions, this means aligning the first axis with the long axis of the ellipse-shaped cloud. In higher dimensions, the same idea applies: PCA finds a set of orthogonal directions that capture the dominant patterns of variation in the data.

One-dimensional PCA as MSE minimization

Setup. Let $x_i \in \mathbb{R}^p$ denote centered observations ($\bar{x} = 0$), and let $u \in \mathbb{R}^p$ be a unit vector ($\|u\|_2 = 1$) defining a 1D subspace (a line through the origin). The scalar projection (score) of x_i onto u is $z_i = u^\top x_i$. A linear reconstruction from the 1D representation is then $\hat{x}_i = z_i\, u$.

Objective (joint in u and z). We want the direction u and scores $\{z_i\}$ that minimize the average reconstruction error:

$$\mathcal{L}(u, \{z_i\}) \;=\; \frac{1}{n}\sum_{i=1}^{n} \big\| \, x_i - z_i u \, \big\|_2^2 \quad \text{subject to } \|u\|_2 = 1.$$

Step 1: Optimize over z for fixed u. For any fixed unit u, $\mathcal{L}$ is a quadratic function of each z_i. Expand the norm:

$$\|x_i - z_i u\|_2^2 = \|x_i\|_2^2 - 2z_i\, u^\top x_i + z_i^2 \underbrace{\|u\|_2^2}_{=1} = \|x_i\|_2^2 - 2z_i\, u^\top x_i + z_i^2.$$

Differentiate w.r.t. z_i and set to zero:

$$\frac{\partial}{\partial z_i}\Big(\|x_i\|_2^2 - 2z_i\, u^\top x_i + z_i^2\Big) = -2\, u^\top x_i + 2z_i \;=\; 0 \;\;\Longrightarrow\;\; z_i^\star \;=\; u^\top x_i.$$

Thus, for any fixed u, the optimal score is the *orthogonal projection*:

$$z_i^\star = u^\top x_i, \qquad \hat{x}_i^\star = (u^\top x_i)u = (uu^\top)x_i.$$

Step 2: Plug back and reduce to an optimization in u only.

We already found that the optimal score for each point is $z_i^\star = u^\top x_i$. Substituting this into the reconstruction $\hat{x}_i = z_i u$, the loss becomes

$$\mathcal{L}(u) = \frac{1}{n}\sum_{i=1}^{n} \| \, x_i - (u^\top x_i)u \, \|_2^2.$$

Step 2a: Write the squared norm as an inner product. By definition,

$$\|x_i - (u^\top x_i)u\|_2^2 = \big(x_i - (u^\top x_i)u\big)^\top \big(x_i - (u^\top x_i)u\big).$$

Step 2b: Expand the product. Expanding the parentheses gives

$$= x_i^\top x_i \;-\; 2(u^\top x_i)(u^\top x_i) \;+\; (u^\top x_i)^2 (u^\top u).$$

Since $u^\top u = 1$, this simplifies to

$$\|x_i - (u^\top x_i)u\|_2^2 = \|x_i\|_2^2 - (u^\top x_i)^2.$$

Step 2c: Average over all samples. The loss is therefore

$$\mathcal{L}(u) = \frac{1}{n}\sum_{i=1}^{n} \Big(\|x_i\|_2^2 - (u^\top x_i)^2\Big).$$

Step 2d: Rewrite in terms of the covariance matrix. Define the sample covariance matrix

$$\Sigma \;=\; \frac{1}{n}\sum_{i=1}^{n} x_i x_i^\top.$$

Two useful identities follow directly:

$$\frac{1}{n}\sum_{i=1}^{n}\|x_i\|_2^2 = \frac{1}{n}\sum_{i=1}^{n}\mathrm{tr}(x_i x_i^\top) = \mathrm{tr}\left(\frac{1}{n}\sum_{i=1}^{n} x_i x_i^\top\right) = \mathrm{tr}(\Sigma),$$

and

$$\frac{1}{n}\sum_{i=1}^{n}(u^\top x_i)^2 = \frac{1}{n}\sum_{i=1}^{n} u^\top(x_i x_i^\top)u = u^\top\left(\frac{1}{n}\sum_{i=1}^{n} x_i x_i^\top\right)u = u^\top\Sigma u.$$

Step 2e: Final expression. Therefore the loss can be written as

$$\boxed{\mathcal{L}(u) = \mathrm{tr}(\Sigma) - u^\top\Sigma u, \quad \|u\|_2 = 1.}$$

The first term, $\mathrm{tr}(\Sigma)$, is constant in u and represents the total variance of the data. Thus minimizing $\mathcal{L}(u)$ is the same as maximizing $u^\top\Sigma u$, which is the variance of the projections onto u. This optimization is exactly the *Rayleigh quotient*, whose maximizer is the eigenvector of Σ with the largest eigenvalue.

Step 3: Lagrange multipliers ⇒ eigenvalue problem. Set up $\mathcal{J}(u,\lambda) = u^\top\Sigma u - \lambda(u^\top u - 1)$. Stationarity in u gives

$$\nabla_u\mathcal{J} = 2\,\Sigma u - 2\,\lambda u = 0 \;\;\Longrightarrow\;\; \Sigma u = \lambda u,$$

so any maximizer must be an eigenvector of Σ. The Rayleigh quotient is maximized by the eigenvector associated to the largest eigenvalue λ_1:

$$\boxed{\Sigma u_1 = \lambda_1 u_1, \quad \|u_1\|_2 = 1, \quad z_i = u_1^\top x_i, \quad \hat{x}_i = z_i\,u_1.}$$

Consequences.

- The minimal 1D reconstruction error is

  $$\mathcal{L}(u_1) \;=\; \mathrm{tr}(\Sigma) - \lambda_1,$$

  i.e. we "lose" all variance except that captured along u_1.

- The variance of the projected data z equals the top eigenvalue:

  $$\mathrm{Var}(z) \;=\; \frac{1}{n}\sum_{i=1}^{n} z_i^2 \;=\; \frac{1}{n}\sum_{i=1}^{n}(u_1^\top x_i)^2 \;=\; u_1^\top\Sigma u_1 \;=\; \lambda_1.$$

- (Preview) The *explained variance ratio* (EVR) of the first component is $\mathrm{EVR}_1 = \lambda_1/\mathrm{tr}(\Sigma)$, the fraction of total variance preserved by the best 1D projection.

Deflation: subtract the first component and recompute

Let $X \in \mathbb{R}^{n\times p}$ be the *centered* data matrix (rows are $x_i^\top$), and

$$\Sigma \;=\; \frac{1}{n} X^\top X$$

its sample covariance. Compute the first principal component $u_1 \in \mathbb{R}^p$ and its eigenvalue λ_1 from

$$\Sigma u_1 = \lambda_1 u_1, \qquad \|u_1\|_2 = 1.$$

Rank-1 reconstruction and residual. Collect the *scores* in $z_1 = Xu_1 \in \mathbb{R}^n$ (one score per sample). The rank-1 reconstruction of X along u_1 is

$$\widehat{X}_1 \;=\; z_1 u_1^\top \;=\; X\, u_1 u_1^\top,$$

and the residual after removing the first component is

$$R_1 \;=\; X - \widehat{X}_1 \;=\; X\,(I_p - u_1 u_1^\top).$$

Thus, R_1 is obtained by projecting every row of X orthogonally onto the subspace orthogonal to u_1.

Deflated covariance. The covariance of the residual is

$$\Sigma^{(1)} \;=\; \frac{1}{n} R_1^\top R_1 \;=\; \frac{1}{n}\,(I_p - u_1 u_1^\top)\, X^\top X\,(I_p - u_1 u_1^\top) \;=\; (I_p - u_1 u_1^\top)\,\Sigma\,(I_p - u_1 u_1^\top).$$

Using $\Sigma u_1 = \lambda_1 u_1$ and $u_1^\top u_1 = 1$, this simplifies to the standard *deflation* form

$$\boxed{\;\Sigma^{(1)} \;=\; \Sigma \;-\; \lambda_1\, u_1 u_1^\top.\;}$$

(Equivalently, $R_1 = X(I_p - u_1 u_1^\top)$ implies $\frac{1}{n} R_1^\top R_1 = \Sigma - \lambda_1 u_1 u_1^\top$.)

Next component from the residual. Compute the second principal direction u_2 as the top eigenvector of the deflated covariance:

$$\Sigma^{(1)} u_2 \;=\; \lambda_2 u_2, \qquad u_1^\top u_2 = 0.$$

Scores and reconstruction for the second component are

$$z_2 = X u_2, \qquad \widehat{X}_2 = z_2 u_2^\top.$$

The cumulative rank-2 reconstruction and residual are

$$\widehat{X}_{1:2} \;=\; \widehat{X}_1 + \widehat{X}_2 \;=\; X\,(u_1 u_1^\top + u_2 u_2^\top), \qquad R_2 \;=\; X - \widehat{X}_{1:2} \;=\; X\Big(I_p - \sum_{j=1}^{2} u_j u_j^\top\Big).$$

General step k. Iterating,

$$\Sigma^{(k)} \;=\; \Sigma^{(k-1)} - \lambda_k u_k u_k^\top \;=\; \Sigma - \sum_{j=1}^{k} \lambda_j u_j u_j^\top,$$

and

$$\widehat{X}_{1:k} \;=\; X\Big(\sum_{j=1}^{k} u_j u_j^\top\Big), \qquad R_k \;=\; X - \widehat{X}_{1:k} \;=\; X\Big(I_p - \sum_{j=1}^{k} u_j u_j^\top\Big).$$

At each step, u_{k+1} is the top eigenvector of $\Sigma^{(k)}$, ensuring u_{k+1} is orthogonal to $\{u_1, \ldots, u_k\}$ and captures the *largest remaining variance*.

PCA via the Rayleigh quotient and eigenbasis expansion

The first principal component can be derived by seeking the unit vector u that maximizes the variance of the projected data, $\mathrm{Var}(u^\top x) = u^\top \Sigma\, u$.

Optimization problem (Rayleigh quotient). Let $\Sigma \in \mathbb{R}^{p\times p}$ be the (sample) covariance matrix, symmetric and positive semidefinite. The first principal direction solves

$$\max_{u\in\mathbb{R}^p} u^\top \Sigma\, u \quad \text{s.t. } \|u\|_2 = 1.$$

Equivalently, this maximizes the Rayleigh quotient $R(u) = \frac{u^\top \Sigma\, u}{u^\top u}$ over $u \neq 0$.

Eigen-decomposition. Since Σ is symmetric, it admits an orthonormal eigenbasis:

$$\Sigma = Q\Lambda Q^\top, \qquad Q = [q_1, \ldots, q_p], \quad \Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_p),$$

with eigenvalues ordered $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_p \geq 0$ and $Q^\top Q = I$.

Expand u in the eigenbasis. Any unit vector u can be expanded as

$$u \;=\; \sum_{j=1}^{p} \alpha_j q_j, \qquad \sum_{j=1}^{p} \alpha_j^2 = 1.$$

Then

$$u^\top \Sigma u = \Big(\sum_j \alpha_j q_j^\top\Big)\, \Sigma \,\Big(\sum_k \alpha_k q_k\Big) = \sum_{j,k} \alpha_j \alpha_k\, q_j^\top (\Sigma q_k) = \sum_j \alpha_j^2\, \lambda_j,$$

using $\Sigma q_k = \lambda_k q_k$ and $q_j^\top q_k = \delta_{jk}$.

Maximizer is an eigenvector. The expression $u^\top \Sigma u = \sum_j \alpha_j^2 \lambda_j$ is a *weighted average of eigenvalues* with nonnegative weights α_j^2 summing to 1. Therefore,

$$u^\top \Sigma u \;\leq\; \lambda_1,$$

with equality iff all the weight is placed on the top eigenspace, i.e., $u \in \mathrm{span}\{q_1\}$ when $\lambda_1 > \lambda_2$. Hence any maximizer is an eigenvector associated with λ_1 (unique up to sign if λ_1 is simple).

Scores and variance. With $u_1 \in \mathrm{argmax}_{\|u\|=1} u^\top \Sigma u$ chosen as a unit top eigenvector, the 1D scores are $z_i = u_1^\top x_i$, and

$$\mathrm{Var}(z) \;=\; u_1^\top \Sigma u_1 \;=\; \lambda_1.$$

Subsequent components (Courant–Fischer). The k-th principal direction u_k solves

$$\max_{\|u\|=1,\; u\perp\{u_1,\ldots,u_{k-1}\}} u^\top \Sigma u,$$

whose maximizers are eigenvectors associated with λ_k. This is the Courant–Fischer min–max principle: constraining u to be orthogonal to the first $k-1$ directions forces the next maximizer to lie in the orthogonal complement and attain $u^\top \Sigma u = \lambda_k$.

Matrix form and reconstruction. Let $U_k = [u_1, \ldots, u_k]$ and $\Lambda_k = \mathrm{diag}(\lambda_1, \ldots, \lambda_k)$. The k-dimensional projection and reconstruction of a centered x are

$$z = U_k^\top x, \qquad \hat{x} = U_k z = U_k U_k^\top x,$$

and the total preserved variance equals $\text{tr}(\Lambda_k) = \sum_{j=1}^{k} \lambda_j$. The explained variance ratio is $\text{EVR}(k) = \frac{\sum_{j=1}^{k} \lambda_j}{\sum_{j=1}^{p} \lambda_j}$.

Takeaway. No non-eigenvector can achieve higher variance than λ_1 under $\|u\| = 1$, because any non-eigen u mixes eigen-directions and yields a convex combination of eigenvalues strictly below the maximum unless all weight is on the top eigenspace.

Practical considerations

Although the definition of PCA is conceptually simple, its application to real datasets requires attention to several practical aspects.

Scaling of variables. PCA depends on the variances of the input features. If variables are measured in different units (e.g. centimeters vs. kilograms), those with larger numerical scales will dominate the principal components. To avoid this, it is common to *standardize* each variable by subtracting its mean and dividing by its standard deviation, so that all features have variance one. Standardization is recommended whenever features have different units or orders of magnitude. If all features are already comparable (e.g. pixel intensities in an image), raw units may be sufficient.

Numerical computation: SVD vs. eigen decomposition. Mathematically, PCA requires the eigen-decomposition of the covariance matrix $\Sigma = \frac{1}{n} X^\top X$. However, in practice it is more stable to compute the singular value decomposition (SVD) of the centered data matrix X:

$$X \;=\; U\,\Sigma_X\,V^\top.$$

The right singular vectors (columns of V) are the principal component directions, and the squared singular values divided by n give the eigenvalues of Σ. Modern software libraries typically implement PCA via SVD.

Centering and why it matters. Centering the data (subtracting the mean vector $\bar{x}$ from each observation) is essential in PCA. Without centering, the first principal component will often simply point toward the mean of the data cloud, rather than capturing the direction of maximum variation around the mean. Centering ensures that PCA describes the *shape* of the cloud, not its absolute position in space.

Computational cost for $n \ll p$ or $p \ll n$. The covariance matrix Σ is of size $p \times p$, where p is the number of features. Computing its eigen-decomposition costs $O(p^3)$ operations, which may be prohibitive when p is very large (e.g. in genomics or text data). Two useful observations:

- If $n \ll p$ (few samples, many features), it is cheaper to compute the eigen-decomposition of the $n \times n$ Gram matrix $XX^\top/n$ and then map back to feature space.
- If $p \ll n$ (many samples, few features), directly working with the $p \times p$ covariance matrix is usually efficient.

In practice, randomized or approximate SVD algorithms can scale PCA to datasets with millions of features or samples.

Choosing the Number of Principal Components

A central practical question is how many principal components k to retain. Keeping too few components risks discarding important signal, while keeping too many may reintroduce noise and defeat the purpose of dimensionality reduction. There is no universally

correct answer; instead, practitioners rely on a combination of quantitative criteria and interpretability.

Explained variance ratio (EVR) Each eigenvalue λ_j of the covariance matrix Σ represents the variance captured by the j-th principal component. The cumulative explained variance ratio is

$$\text{EVR}(k) \;=\; \frac{\sum_{j=1}^{k} \lambda_j}{\sum_{j=1}^{p} \lambda_j}.$$

A common rule is to choose the smallest k such that $\text{EVR}(k) \geq \tau$, with τ typically 0.90 or 0.95. This ensures that most of the total variance in the data is preserved. Note, however, that the threshold is arbitrary, and whether there exists an optimal one will be problem-dependent.

Eigenvalue plot and the elbow rule. One simple diagnostic is to plot the eigenvalues $\lambda_1, \lambda_2, \ldots$ of the covariance matrix in decreasing order. The first few eigenvalues are typically large, reflecting directions that capture most of the variance, while the remaining eigenvalues form a long, flat tail. The elbow rule suggests choosing the number of components k at the point where this curve shows a sharp bend and then levels off. Beyond this bend, each additional component explains only a small fraction of the variance. Although somewhat subjective, this visual heuristic provides a quick way to decide how many components to retain.

Cross-validation and downstream tasks In predictive settings, the best choice of k can be evaluated empirically by training a supervised model (e.g. regression or classification) on the first k PCs and comparing validation error across values of k. Here the optimal k is whichever balances predictive accuracy and model simplicity.

Domain knowledge and interpretability In exploratory analysis, interpretability may guide the choice. For example, in genomics, one may retain a handful of PCs that correspond to known biological processes, even if the explained variance is modest. In image compression, visual inspection of reconstructions can help decide how many components yield acceptable fidelity.

In practice, the choice of k is often based on a mixture of these approaches: start with the EVR threshold, inspect the eigenvalue plot, and validate against the task-specific goal (prediction, compression, visualization, or interpretation).

Applications of PCA

Beyond its theoretical interest, PCA is one of the most widely used tools in applied data analysis. Its main applications include:

Visualization. When data are high-dimensional, plotting them directly is impossible. PCA provides a linear projection into two or three dimensions that preserves as much variance as possible. Plotting the first two or three principal components often reveals hidden structure such as clusters, outliers, or continuous trajectories (e.g. in biology, the cell cycle can appear as a circular trajectory in the first two PCs).

Data compression. By keeping only the first k principal components (with $k \ll p$), we obtain a low-dimensional representation that captures most of the variability in the data. This reduces storage and computational costs while retaining the essential information. PCA has historically been used for tasks such as image compression, where reconstructions from a small number of components still look remarkably faithful.

Noise reduction / denoising. Noise often contributes to small-variance directions in the data. By discarding the principal components associated with the smallest eigenvalues, PCA can suppress noise while keeping signal. This idea is common in signal processing, image analysis, and genomics, where PCA serves as a simple denoising step.

Preprocessing for other models. PCA is frequently applied before supervised learning to reduce dimensionality and mitigate collinearity between features. For example, in regression problems with highly correlated predictors, replacing the original variables by their principal components (or a subset of them) can improve numerical stability. In classification tasks, PCA can reduce the input space to a compact representation that speeds up training.

Exploratory data analysis. PCA is also a tool for hypothesis generation. By examining which variables contribute most to each principal component (the loadings), analysts can identify groups of correlated variables, underlying factors, or hidden patterns that may not be obvious from the raw data.

In short, PCA is both a dimensionality reduction technique and a lens through which we can *understand*, *simplify*, and *visualize* high-dimensional data. Its practical importance cannot be overstated: PCA is often the very first tool applied when facing a new dataset.

Limitations of PCA

Despite its usefulness, PCA comes with important caveats that should guide how it is applied and interpreted.

Linearity. PCA finds a *linear* subspace. If the data lie near a curved (nonlinear) manifold, a low-dimensional linear projection may distort the structure (e.g., unrolling a spiral into overlapping directions). Nonlinear methods (kernel PCA, Isomap, UMAP, t-SNE) are often more appropriate in such cases.

Variance $\neq$ task relevance. PCA orders directions by *total variance*, not by their usefulness for a downstream task. High-variance components can be irrelevant (or even detrimental) for prediction or classification, while low-variance components may carry discriminative signal. Supervised dimensionality reduction (e.g., LDA, PLS) may be preferable when labels are available.

Sensitivity to scaling and centering. PCA is not scale-invariant across features: variables with larger variance dominate the components unless features are standardized. Likewise, failure to center the data typically points the first PC toward the mean rather than the shape around the mean.

Sensitivity to outliers. Because covariance (and least-squares) are influenced by extremes, a few outliers can rotate PCs substantially. Robust variants (e.g., robust covariance, M-estimators, Robust PCA via low-rank + sparse decomposition) mitigate this issue.

Interpretability of loadings. PCs are linear combinations of many variables; when loadings are dense, components can be hard to interpret. Sparse PCA and rotation techniques (e.g., varimax) can improve interpretability at the cost of optimal variance capture.

Non-uniqueness under ties and sign flips. If eigenvalues are equal (or nearly equal), the associated eigenspace is not uniquely determined: any orthonormal basis spanning that subspace is a valid set of PCs. Additionally, each PC is defined up to a global sign (flipping u_j leaves the model unchanged).

Instability under dataset shift. PC directions are estimated from the sample covariance

and can change when the dataset changes (e.g., different cohorts, batch effects). This can reduce reproducibility across studies unless careful harmonization is performed.

Global objective, local structure. By optimizing a global variance criterion, PCA may overlook fine-grained or cluster-specific structure (e.g., well-separated small clusters that contribute little to total variance).

Practical takeaway. Use PCA with standardized, centered data; check robustness to outliers; validate that retained components support the downstream objective; and consider task-aware (LDA/PLS), robust (Robust PCA), sparse (Sparse PCA), or nonlinear (kernel PCA, Isomap, UMAP, t-SNE) alternatives when PCA's assumptions are violated.

4.3. k-Means Clustering

Motivation and intuition

While PCA addresses *continuous* latent structure, clustering methods aim to uncover *discrete* latent structure: groups of points that are more similar to each other than to the rest of the dataset. The goal is to partition unlabeled data into coherent subsets called *clusters*, so that points within the same cluster are close together in feature space, while points in different clusters are farther apart.

A classical example comes from image data. Consider a collection of handwritten digits. Although each digit image is high-dimensional, each observation fundamentally belongs to one of ten categories (0–9). Clustering seeks to discover these categories automatically, without access to labels. Similar motivations appear in market segmentation (grouping customers by purchasing behavior), biology (grouping cells by expression profiles), or information retrieval (organizing documents by topic).

The most widely used clustering method is **k-means**. The intuition is simple:

- Assume that each cluster can be represented by a prototype or *centroid*.
- Assign each data point to the cluster whose centroid is closest.
- Recompute centroids as the average of the points assigned to them.

By repeating this process, the centroids move to positions that represent the data well, and the assignments stabilize into a partition of the dataset.

In geometric terms, k-means seeks to approximate the data cloud by k points (centroids) and represent each observation by its nearest centroid. This is equivalent to fitting the data with a piecewise-constant model: every point in a cluster is approximated by the same centroid. The quality of the clustering is then measured by the average squared distance between points and their assigned centroid.

Objective function and Optimization

Let $x_1, \dots, x_n \in \mathbb{R}^p$ be the data points to be clustered into k groups. Introduce binary indicator variables

$$c_{ik} = \begin{cases} 1 & \text{if point } x_i \text{ is assigned to cluster } k, \\ 0 & \text{otherwise,} \end{cases} \qquad \text{with } \sum_{k=1}^{K} c_{ik} = 1 \ \ \forall i.$$

Each cluster k is represented by a centroid $\mu_k \in \mathbb{R}^p$. The k-means objective function, known as the *within-cluster sum of squares* (WCSS), is

$$J(C, \{\mu_k\}) \;=\; \sum_{i=1}^{n} \sum_{k=1}^{K} c_{ik} \,\| \, x_i - \mu_k \, \|_2^2,$$

where $C = (c_{ik})$ is the assignment matrix of size $n \times K$.

Optimization over μ_k. For fixed assignments C, the minimizer of J with respect to μ_k is the mean of the points assigned to cluster k:

$$\mu_k \;=\; \frac{\sum_{i=1}^{n} c_{ik} x_i}{\sum_{i=1}^{n} c_{ik}}.$$

Optimization over c_{ik}. For fixed centroids $\{\mu_k\}$, the optimal assignment for point x_i is

$$c_{ik} \;=\; \begin{cases} 1 & \text{if } k = \arg\min_{j \in \{1,\dots,K\}} \|x_i - \mu_j\|_2^2, \\ 0 & \text{otherwise.} \end{cases}$$

Thus, minimizing J amounts to an alternating optimization: update the centroids given the assignments, then update the assignments given the centroids. This is exactly the procedure known as Lloyd's algorithm, which decreases (or leaves unchanged) the objective at each step and converges to a local minimum.

Algorithm: Lloyd's k-Means

Input: Data points $x_1, \dots, x_n \in \mathbb{R}^p$, number of clusters K.

Initialization: Choose initial centroids $\mu_1, \dots, \mu_K$ (e.g. randomly or with k-means++).

Repeat until convergence:

1. **Assignment step:** For each $i = 1, \dots, n$, assign x_i to the nearest centroid:

$$c_{ik} = \begin{cases} 1 & \text{if } k = \arg\min_{j \in \{1,\dots,K\}} \|x_i - \mu_j\|_2^2, \\ 0 & \text{otherwise.} \end{cases}$$

2. **Update step:** For each $k = 1, \dots, K$, update the centroid as the mean of its assigned points:

$$\mu_k = \frac{\sum_{i=1}^{n} c_{ik}\, x_i}{\sum_{i=1}^{n} c_{ik}}.$$

Output: Cluster assignments $C = (c_{ik})$ and centroids $\{\mu_k\}$.

Note: Each iteration decreases (or leaves unchanged) the objective function

$$J(C, \{\mu_k\}) \;=\; \sum_{i=1}^{n} \sum_{k=1}^{K} c_{ik} \, \|x_i - \mu_k\|_2^2,$$

so the algorithm is guaranteed to converge in finitely many steps to a *local minimum*.

Computational complexity. Although each update step of Lloyd's algorithm is simple, finding the *global minimum* of the k-means objective is computationally intractable in general. In fact, the problem is known to be NP-hard for $K \geq 2$ clusters in $\mathbb{R}^p$, even when $p = 2$. A problem is called *NP-hard* if it is at least as difficult as every problem in NP: formally, any NP problem can be transformed ("reduced") into it in polynomial time. This means that if we could solve k-means optimally in polynomial time, we would also be able to solve all problems in NP efficiently. Since no such algorithm is known, and is widely believed not to exist, practical methods such as Lloyd's algorithm settle for finding a *local minimum*, whose quality depends on the initialization of the centroids. This brings us to the question of initialization strategies.

Initialization issues

The quality of k-means clustering strongly depends on the choice of starting centroids. Since the objective function is nonconvex, different initializations can lead to very different local optima, and hence to very different final clusterings.

Random initialization. A simple strategy is to choose the initial centroids by sampling K points uniformly at random from the dataset. While fast and easy to implement, this approach can give poor results: if two centroids are initialized close together, one cluster may be empty, and the final partition may be far from optimal. Moreover, random seeding can make the algorithm unstable, producing different outcomes on different runs.

k-means++ initialization. A widely used improvement is the *k-means++* seeding algorithm. The idea is to spread out the initial centroids:

1. Choose the first centroid uniformly at random from the data points.
2. For each subsequent centroid, choose a new point x_i with probability proportional to its squared distance from the nearest already chosen centroid.
3. Repeat until K centroids have been chosen.

This procedure ensures that the initial centroids are well separated, which greatly improves the stability of the algorithm. In fact, it can be shown that k-means++ achieves an expected clustering cost within a logarithmic factor of the global optimum, and in practice it almost always outperforms random initialization.

Decision boundaries induced by k-means

An important geometric perspective on k-means clustering is that it partitions the feature space into regions of influence around each centroid. After the algorithm converges, every point in the dataset is assigned to the nearest centroid. This induces a *Voronoi tessellation* of the space: an intuitive way to think of this is that the space is divided into tiles, where each tile consists of all points that are closer to one centroid than to any other. Formally, the region corresponding to centroid μ_k is

$$\mathcal{R}_k = \{\, x \in \mathbb{R}^p \;:\; \|x - \mu_k\|_2 \leq \|x - \mu_j\|_2 \;\text{ for all } j \neq k \,\}.$$

Linear decision boundaries. The boundaries between clusters are formed by the loci of points that are equidistant between two centroids. These are affine hyperplanes in $\mathbb{R}^p$, so the decision boundaries of k-means are always linear. As a result, the clusters it recovers are necessarily convex regions of space.

Interpretation. This Voronoi perspective clarifies both the strengths and limitations of k-means. For roughly spherical, equally sized clusters, the linear boundaries approximate the true structure well. However, when clusters have irregular or non-convex shapes, k-means will impose straight-line splits that may not reflect the underlying data distribution.

Choosing the number of clusters k

A fundamental practical issue in clustering is how to choose the number of clusters k. Unlike supervised learning, there is no external label information to guide the choice, and the "right" number of clusters is often ambiguous. Several heuristics and criteria are commonly used in practice.

The elbow method. An approach analogous to PCA is to examine how the within-cluster sum of squares (WCSS) decreases as k increases. By definition, WCSS is nonincreasing in k, since more clusters always allow a better fit. However, the marginal improvement usually decreases after a certain point. Plotting WCSS versus k often reveals a sharp bend, or "elbow," beyond which additional clusters bring little gain. The elbow location is taken as a reasonable choice of k. This method is simple and widely used, but it can be subjective, especially if the curve does not show a clear bend.

Silhouette score. The silhouette score provides a quantitative measure of clustering quality that balances cohesion (points are close to their own cluster centroid) and separation (points are far from other cluster centroids). For a point x_i, let

$$a(i) = \text{average distance of } x_i \text{ to other points in the same cluster,} \tag{4.1}$$

$$b(i) = \min_{k' \neq c_i} \text{ average distance of } x_i \text{ to points in cluster } k'. \tag{4.2}$$

The silhouette value of x_i is then

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \in [-1, 1].$$

The overall silhouette score is the average of $s(i)$ across all points. A high silhouette score (close to 1) indicates well-separated, compact clusters, while values near 0 suggest overlapping clusters. The optimal k can be chosen as the value maximizing the silhouette score.

Davies–Bouldin index. Another internal validation measure is the Davies–Bouldin (DB) index, which quantifies the average similarity between each cluster and its most similar one. It is defined as

$$\text{DB} = \frac{1}{K} \sum_{k=1}^{K} \max_{j \neq k} \left(\frac{\sigma_k + \sigma_j}{d(\mu_k, \mu_j)} \right),$$

where σ_k is the average distance of points in cluster k to their centroid μ_k, and $d(\mu_k, \mu_j)$ is the distance between centroids. Lower DB values indicate better clustering (compact, well-separated clusters). The optimal k minimizes the DB index.

Application-driven choice. In many domains, the choice of k is guided by the application rather than by purely statistical criteria. For example, in market segmentation, the number of customer groups may be determined by business strategy; in biology, k may correspond to known cell types or experimental conditions. In exploratory analysis, different values of k can reveal structure at different levels of granularity. Ultimately, the "best" number of clusters depends on the balance between interpretability and the goals of the analysis.

Applications of k-means

Beyond its role as a simple and intuitive clustering algorithm, k-means has been widely adopted across many domains. Its appeal lies in its computational efficiency, scalability to large datasets, and ease of interpretation.

Market segmentation. In marketing and economics, k-means is often used to partition customers into groups with similar purchasing behavior, demographics, or preferences. These clusters provide insight into consumer heterogeneity and can guide targeted advertising or product design.

Image compression and vector quantization. Each pixel in an image can be represented by its color vector (e.g. RGB values). By clustering all pixel colors into k centroids, and then replacing each pixel by the nearest centroid, the image can be stored using only k representative colors. This reduces storage while retaining perceptual quality, and is a classical application of k-means in signal and image processing.

Document and text clustering. Representing documents as high-dimensional vectors (e.g. term frequency or embeddings), k-means can group documents by topic or style. This is useful in information retrieval, topic modeling, and organizing large corpora without labeled categories.

Biological data analysis. In genomics and transcriptomics, k-means is frequently used to cluster genes with similar expression profiles or to group cells with similar transcriptional states. These clusters can reveal functional modules or putative cell types, providing hypotheses for further biological investigation.

Preprocessing and initialization. Finally, k-means is not only used as a standalone method but also as a preprocessing step. For example, it can provide initial clusters for more complex mixture models, initialize centroids for Gaussian Mixture Models (EM algorithm), or generate visual codebooks in computer vision pipelines.

Limitations of k-means

Although k-means is simple and widely used, it comes with several important limitations that must be considered in practice.

Assumption of spherical clusters. The algorithm partitions space using Voronoi regions around the centroids. As a result, it implicitly assumes that clusters are roughly spherical (isotropic) and of similar size. When clusters have elongated shapes or very different densities, k-means often fails to capture the true structure.

Sensitivity to outliers. Because centroids are defined as means, a single extreme point can pull a centroid far away from the bulk of its cluster. This makes k-means sensitive to noise and outliers, which can significantly distort the clustering.

Difficulty with non-convex shapes. The decision boundaries induced by k-means are linear, so the algorithm cannot separate clusters with complex, non-convex geometries. For example, in a dataset shaped like two concentric circles, k-means will split the circles into wedges rather than recognizing them as distinct rings.

5. Model Performance, Generalization, and Regularization

5.1. Underfitting, Overfitting, and Generalization

In Chapter 1 we analyzed the task of modeling global temperature anomalies with polynomial regression of increasing degree (Figure 1.7). That example already illustrated one of the most fundamental phenomena in machine learning: the trade-off between *fitting the training data* and *generalizing to unseen data*. We now revisit that experiment to formalize the concepts of **underfitting**, **overfitting**, and **generalization**.

Motivating Example: Temperature Anomalies Revisited

Recall that we fitted polynomials of degree $d = 1, 2, \ldots, D$ to predict yearly global temperature anomalies $\hat{y}_t = f_d(x_t)$ from time x_t. When d was too small (e.g. $d = 1$), the model could only represent a straight trend and failed to capture nonlinear curvature—a case of *underfitting*. When d was too large (e.g. $d = 25$), the model matched the training points almost perfectly but produced spurious oscillations that did not correspond to physical reality—a case of *overfitting*. The intermediate degrees achieved the best predictive accuracy on the test data, striking a balance between the two extremes.

Capacity and Model Complexity

To reason about these effects systematically, we introduce the notion of *model capacity*. The capacity of a hypothesis space $\mathcal{H}$ measures its ability to represent complex functions. Formally, $\mathcal{H}$ is the set of predictors $\{f_\theta : \theta \in \Theta\}$ considered during training. A simple model class (e.g. linear functions) has low capacity and can only approximate simple relationships, whereas a richer class (e.g. high-degree polynomials or deep neural networks) has high capacity and can fit intricate patterns in the data.

Increasing capacity generally decreases the training error, since a larger space $\mathcal{H}$ contains more expressive functions. However, beyond a certain point, the additional flexibility allows the model to adapt to noise rather than signal, leading to poor performance on new data. Understanding this balance is central to the theory of *generalization*.

Summary

- **Underfitting** arises when the hypothesis space is too limited to capture the structure of the data (*high bias*).
- **Overfitting** occurs when the model is overly flexible and fits noise rather than the underlying pattern (*high variance*).
- **Generalization** reflects how well the model's predictions extend to unseen data drawn from the same distribution.

In the next section we will formalize this intuition mathematically through the *bias–variance decomposition* of the expected error, which provides a quantitative framework for understanding and balancing model complexity.

5.2. The Bias–Variance Trade-off

The concepts of underfitting and overfitting can be understood quantitatively through the **bias–variance decomposition**, which expresses the expected prediction error as the sum of three interpretable terms: squared bias, variance, and irreducible noise. This decomposition provides a mathematical foundation for the intuitive notion of model complexity discussed in the previous section.

Expected Prediction Error

Let (x, y) denote a random test point drawn from the true data-generating distribution $P(x, y)$, and let $\hat{f}(x)$ be the predictor obtained from a training sample $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$. Assume that the target variable follows the model

$$y = f^*(x) + \varepsilon, \quad \text{with} \quad \mathbb{E}[\varepsilon \mid x] = 0, \quad \text{Var}(\varepsilon \mid x) = \sigma^2.$$

Here $f^*(x)$ is the true underlying function and ε represents random noise that cannot be predicted, even with a perfect model.

The goal is to evaluate the *expected test error* at a given input x:

$$\mathbb{E}_{\mathcal{D},\varepsilon}\left[(\hat{f}(x) - y)^2\right], \tag{5.1}$$

where the expectation is taken over both the randomness of the training set (which affects $\hat{f}$) and the intrinsic data noise ε.

Derivation of the Bias–Variance Decomposition

We start by expanding the squared error in (5.1):

$$\mathbb{E}\Big[(\hat{f}(x) - y)^2\Big] = \mathbb{E}\Big[(\hat{f}(x) - f^*(x) - \varepsilon)^2\Big].$$

Expanding the square gives

$$\mathbb{E}\Big[(\hat{f}(x) - f^*(x))^2\Big] - 2\,\mathbb{E}\Big[(\hat{f}(x) - f^*(x))\,\varepsilon\Big] + \mathbb{E}[\varepsilon^2].$$

Because ε has zero mean and is independent of $\hat{f}$, the cross term vanishes, and we obtain:

$$\mathbb{E}\Big[(\hat{f}(x) - y)^2\Big] = \mathbb{E}\Big[(\hat{f}(x) - f^*(x))^2\Big] + \sigma^2.$$

The first term measures the expected squared deviation of our predictor from the true function $f^*(x)$, while σ^2 is the irreducible error caused by intrinsic noise in the data.

We now decompose $\mathbb{E}\Big[(\hat{f}(x) - f^*(x))^2\Big]$ into bias and variance components. Let $\bar{f}(x) = \mathbb{E}_{\mathcal{D}}[\hat{f}(x)]$ be the average prediction over all possible training sets. Then

$$\mathbb{E}_{\mathcal{D}}\Big[(\hat{f}(x) - f^*(x))^2\Big] = \underbrace{(\bar{f}(x) - f^*(x))^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}_{\mathcal{D}}\Big[(\hat{f}(x) - \bar{f}(x))^2\Big]}_{\text{Variance}}.$$

Combining these results, we arrive at the classical decomposition:

$$\mathbb{E}_{\mathcal{D},\varepsilon}\Big[(\hat{f}(x)-y)^2\Big] = \underbrace{(\bar{f}(x)-f^*(x))^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}_{\mathcal{D}}\Big[(\hat{f}(x)-\bar{f}(x))^2\Big]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Irreducible noise}}. \tag{5.2}$$

Averaging over all possible inputs $x \sim P(x)$ yields the global expected test mean squared error (MSE):

$$\mathbb{E}_{x,\mathcal{D},\varepsilon}\Big[(\hat{f}(x)-y)^2\Big] = \text{Bias}^2 + \text{Variance} + \sigma^2.$$

Interpretation

- **Bias** measures systematic error—how far the average prediction $\bar{f}(x)$ is from the true function $f^*(x)$. High bias means the model is too simple to capture the underlying structure (*underfitting*).
- **Variance** measures how much $\hat{f}(x)$ fluctuates across different training sets. High variance means the model is too sensitive to random noise or sample fluctuations (*overfitting*).
- **Irreducible noise** σ^2 represents randomness inherent in the data and cannot be reduced by any model.

Examples: Polynomial Degree and Tree Depth

In the polynomial regression example from Chapter 1, increasing the degree d reduces bias (the model captures more curvature) but increases variance (it becomes more sensitive to fluctuations in the data). The same pattern appears in decision trees: shallow trees exhibit high bias and low variance, while very deep trees can perfectly fit the training data but generalize poorly.

Regularization techniques (discussed in Section 5.4) and validation strategies (Section 5.5) are practical tools to control this trade-off and select models that achieve good generalization.

5.3. Regularization and Model Complexity Control

In the previous section, we saw that increasing model capacity reduces bias but increases variance. To achieve good generalization, we need a mechanism to control this capacity. **Regularization** provides such a mechanism by explicitly penalizing overly complex models during training. It modifies the optimization objective so that simpler models are preferred unless the data strongly support greater complexity.

Motivation: Controlling Complexity through Penalization

Consider a linear model

$$f_\theta(x) = \theta^\top x,$$

fitted by minimizing the empirical mean squared error (MSE):

$$R_n(\theta) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \theta^\top x_i)^2.$$

If the number of features p is large or if predictors are highly correlated, the unregularized least-squares estimator

$$\hat{\theta}_{\text{OLS}} = (X^\top X)^{-1} X^\top y$$

may have high variance and poor generalization. Regularization stabilizes the solution by adding a penalty term that discourages large or complex parameter values.

Ridge Regression (L2 Regularization)

Ridge regression introduces an ℓ_2 penalty on the weights:

$$\hat{\theta}_{\text{ridge}} = \arg\min_{\theta} \left\{ \frac{1}{n} \sum_{i=1}^{n} (y_i - \theta^\top x_i)^2 + \lambda \|\theta\|_2^2 \right\}, \tag{5.3}$$

where $\lambda \geq 0$ is the regularization strength. Large values of λ shrink all coefficients toward zero but never exactly to zero. The analytic solution is

$$\hat{\theta}_{\text{ridge}} = (X^\top X + n\lambda I)^{-1} X^\top y.$$

Bayesian interpretation. Ridge regression corresponds to a Gaussian prior on parameters:

$$P(\theta) = \mathcal{N}(0, \tau^2 I),$$

where the prior variance τ^2 controls how strongly we believe weights should be small. Maximizing the posterior probability

$$P(\theta \mid \mathcal{D}) \propto P(\mathcal{D} \mid \theta) P(\theta)$$

is equivalent to minimizing (5.3) with $\lambda = \frac{\sigma^2}{2n\tau^2}$. Thus, ridge regression can be seen as *MAP estimation under a Gaussian prior*. This prior smooths parameter estimates and reduces variance without introducing sparsity.

Lasso Regression (L1 Regularization)

The **Lasso** (Least Absolute Shrinkage and Selection Operator) replaces the ℓ_2 penalty with an ℓ_1 norm:

$$\hat{\theta}_{\text{lasso}} = \arg\min_{\theta} \left\{ \frac{1}{n} \sum_{i=1}^{n} (y_i - \theta^\top x_i)^2 + \lambda \|\theta\|_1 \right\}, \tag{5.4}$$

where $\|\theta\|_1 = \sum_{j=1}^{p} |\theta_j|$.

Unlike ridge regression, the Lasso can drive some coefficients exactly to zero, thereby performing automatic *feature selection*. The optimization problem is convex but non-differentiable at zero; efficient algorithms include coordinate descent and the LARS (Least Angle Regression) method.

Bayesian interpretation. The Lasso corresponds to a *Laplace prior* (double-exponential distribution) over parameters:

$$P(\theta_j) = \frac{\lambda}{2} \exp(-\lambda |\theta_j|).$$

This prior places more probability mass near zero and heavier tails than a Gaussian, encouraging sparsity: most coefficients are strongly shrunk, while a few remain large.

Elastic Net Regularization

Both Ridge and Lasso have advantages: ridge stabilizes correlated predictors, while Lasso selects features. The **Elastic Net** combines both penalties:

$$\hat{\theta}_{\text{EN}} = \arg\min_{\theta} \left\{ \frac{1}{n} \sum_{i=1}^{n} (y_i - \theta^\top x_i)^2 + \lambda \left[\alpha \|\theta\|_1 + (1-\alpha) \|\theta\|_2^2 \right] \right\}, \tag{5.5}$$

where $\alpha \in [0, 1]$ controls the balance between ℓ_1 and ℓ_2 penalties.

- $\alpha = 0 \Rightarrow$ ridge regression.
- $\alpha = 1 \Rightarrow$ lasso regression.
- Intermediate values interpolate between the two, often improving performance when predictors are highly correlated.

Geometric Interpretation

Figure **??** illustrates the geometry of regularization in the two-dimensional parameter space (θ_1, θ_2). Each regularizer defines a constraint region $\|\theta\|_p \leq t$ whose boundary intersects the ellipses of the MSE loss. The shape of this region determines the qualitative behavior of the estimator:

- Ridge (ℓ_2) penalty: circular constraint $\Rightarrow$ smooth shrinkage of all coefficients; no exact zeros.
- Lasso (ℓ_1) penalty: diamond-shaped constraint $\Rightarrow$ corners aligned with coordinate axes; solutions often lie on axes, producing sparse coefficients.
- Elastic Net: intermediate geometry combining shrinkage and sparsity.

Effect on Bias and Variance

Regularization introduces bias (coefficients are shrunk toward zero) but greatly reduces variance, leading to improved generalization. Choosing the regularization parameter λ controls this trade-off:

- Small λ: low bias, high variance (risk of overfitting).
- Large λ: high bias, low variance (risk of underfitting).

Optimal values are typically found through cross-validation.

Part 2.

Nonlinear and Non-Parametric Models

6. Kernel Methods

In the previous chapters, we developed linear models for regression and classification, studied their probabilistic interpretation, and explored how regularization controls model complexity. While linear models are elegant and often effective, their main limitation lies in their inability to represent nonlinear relationships between inputs and outputs. A natural question arises: *how can we extend the power of linear models to capture nonlinear structure, without abandoning their mathematical simplicity?*

This chapter introduces the framework of **kernel methods**, which provides a unified way to construct nonlinear models from linear ones. The central idea is to replace explicit nonlinear feature transformations by a kernel function that computes inner products in a high-dimensional feature space implicitly. This allows us to design flexible nonlinear models that remain linear in their parameters and computationally tractable.

6.1. Motivation: From Linear Models to Nonlinear Feature Spaces

To motivate kernel methods, let us recall the general form of a linear regression model:

$$\hat{y}(x) = \theta^\top \phi(x),$$

where $\phi(x)$ denotes a vector of basis functions (or *features*) that map the input $x \in \mathbb{R}^D$ into a feature space of dimension M, and $\theta \in \mathbb{R}^M$ are the model parameters. Nonlinearity enters the model through the choice of the mapping $\phi(x)$: by designing suitable basis functions, we can represent complex dependencies between inputs and outputs while preserving linearity in the parameters.

Polynomial regression as feature mapping. To illustrate this idea, consider a one-dimensional input $x \in \mathbb{R}$. A polynomial regression model of degree d can be written as

$$\hat{y}(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \cdots + \theta_d x^d,$$

which is nonlinear in x but linear in the coefficients θ_j. We can express it in vector form as

$$\hat{y}(x) = \theta^\top \phi(x), \quad \text{with} \quad \phi(x) = (1, x, x^2, \ldots, x^d)^\top.$$

Given a dataset of N pairs $\{(x_i, y_i)\}_{i=1}^N$, we collect the feature vectors into the *design matrix*

$$\Phi = \begin{pmatrix} \phi(x_1)^\top \\ \phi(x_2)^\top \\ \vdots \\ \phi(x_N)^\top \end{pmatrix}, \quad \text{and} \quad y = (y_1, \ldots, y_N)^\top.$$

The least-squares solution for the parameters is then

$$\theta = (\Phi^\top \Phi)^{-1} \Phi^\top y,$$

and the prediction for a new input x is

$$\hat{y}(x) = \phi(x)^\top \theta = \phi(x)^\top (\Phi^\top \Phi)^{-1} \Phi^\top y.$$

Both the estimated parameters and the predictions depend on the training data only through

$$\Phi^\top \Phi = \sum_{i=1}^{N} \phi(x_i)\phi(x_i)^\top,$$

that is, through *inner products* between feature vectors:

$$\phi(x_i)^\top \phi(x_j).$$

This dependence on feature inner products is a fundamental property of many linear models, including ridge regression, linear discriminant analysis, and principal component analysis.

The curse of dimensionality in feature expansions

The polynomial example above is transparent in one dimension, but the situation becomes rapidly more complex in higher dimensions. Let the input be a vector $x = (x_1, x_2, \ldots, x_D) \in \mathbb{R}^D$. A polynomial model of total degree d includes all monomials of the form

$$x_1^{p_1} x_2^{p_2} \cdots x_D^{p_D}, \quad \text{with} \quad p_1 + p_2 + \cdots + p_D \leq d,$$

where each exponent $p_i \geq 0$ is an integer. Each distinct combination of exponents $(p_1, \ldots, p_D)$ corresponds to one basis function.

The number of such monomials, and hence the dimensionality of the feature vector $\phi(x)$, is given by the number of non-negative integer solutions to

$$p_1 + p_2 + \cdots + p_D \leq d.$$

This classical combinatorial problem, known as the *stars and bars* problem, has the closed-form solution

$$M = \binom{D+d}{d}.$$

Combinatorial growth of polynomial features For $D = 1$ and $d = 3$, we have $M = 4$ basis functions: $1, x, x^2, x^3$. For $D = 2$ and $d = 2$, there are already $M = 6$ features:

$$1,\ x_1,\ x_2,\ x_1^2,\ x_1 x_2,\ x_2^2.$$

For $D = 3$ and $d = 3$, the number increases to $M = 20$, and for $D = 10$ and $d = 5$, we obtain $M = \binom{15}{5} = 3003$.

This explosive combinatorial growth is known as the *curse of dimensionality*. Even for moderate D and d, the number of basis functions becomes enormous, making explicit feature construction computationally expensive and statistically inefficient.

This observation motivates a change in perspective. Rather than expanding $\phi(x)$ explicitly, we can design algorithms that depend only on inner products between feature vectors,

$$\phi(x_i)^\top \phi(x_j).$$

In the next section, we will see that ridge regression can be rewritten entirely in terms of these inner products, leading to the **kernel trick**: a simple idea that allows us to perform nonlinear learning in high-dimensional spaces without ever computing the features explicitly.

The key idea: implicit feature spaces

If we can compute the inner product between two feature vectors without explicitly evaluating $\phi(x)$, we can effectively work in very high, even infinite-dimensional, feature spaces at negligible cost. This leads to the central concept of kernel methods.

Definition 6.1 (Kernel function)**.** A *kernel function* is any function $k : \mathbb{R}^D \times \mathbb{R}^D \to \mathbb{R}$ that can be expressed as an inner product in some feature space:

$$k(x, x') = \phi(x)^\top \phi(x'),$$

for a (possibly infinite-dimensional) mapping $\phi(\cdot)$.

Polynomial kernel. For the polynomial feature map

$$\phi(x) = (1, \sqrt{2}x, x^2),$$

the inner product between two points x and x' in the feature space is

$$\phi(x)^\top \phi(x') = 1 + 2xx' + (xx')^2 = (1 + xx')^2.$$

More generally, this yields the *polynomial kernel*

$$k(x, x') = (1 + x^\top x')^d,$$

which implicitly represents all polynomial terms of degree up to d without explicitly constructing them.

Kernel functions thus provide a practical and elegant way to build nonlinear models by implicitly defining the geometry of a high-dimensional feature space. Rather than specifying basis functions explicitly, we specify a function $k(x, x')$ that defines how similar two inputs are according to the model's geometry.

In the next section, we will revisit ridge regression and show that its dual formulation depends only on inner products between training points. By replacing these inner products with kernel functions, we obtain nonlinear versions of our linear models.

6.2. Dual Representation of Ridge Regression

We begin by revisiting the ridge regression model in its standard linear form and then extend it to arbitrary feature mappings. This will lead to an alternative formulation, the *dual representation*, that depends only on inner products between feature vectors. This dual form provides the mathematical foundation for kernel methods.

Ridge regression in matrix form. Consider a dataset of N observations $\{(x_i, y_i)\}_{i=1}^N$ with $x_i \in \mathbb{R}^D$. Let X be the design matrix that stacks all inputs as rows:

$$X = \begin{pmatrix} x_1^\top \\ x_2^\top \\ \vdots \\ x_N^\top \end{pmatrix} \in \mathbb{R}^{N \times D}, \quad \text{and} \quad y = (y_1, \ldots, y_N)^\top \in \mathbb{R}^N.$$

The ridge regression problem estimates parameters $\theta \in \mathbb{R}^D$ by minimizing

$$J(\theta) = \|X\theta - y\|^2 + \lambda \|\theta\|^2,$$

where $\lambda > 0$ controls the amount of regularization. The solution in the *primal form* is well known:

$$\theta = (X^\top X + \lambda I_D)^{-1} X^\top y.$$

The prediction for a new input x is

$$\hat{y}(x) = x^\top \theta = x^\top (X^\top X + \lambda I_D)^{-1} X^\top y.$$

This expression shows that predictions depend on the training data through the inner products $x_i^\top x_j$ that appear inside $X^\top X$. We will now make this dependence explicit by moving to the dual formulation.

Deriving the dual form. We begin by expressing θ as a linear combination of the training inputs:

$$\theta = X^\top \alpha,$$

where $\alpha \in \mathbb{R}^N$ is a vector of coefficients, one per training example. This representation is natural because θ appears in the loss function only through the product $X\theta$, which lies in the column space of X.

Substituting $\theta = X^\top \alpha$ into the ridge regression objective gives

$$J(\alpha) = \|XX^\top \alpha - y\|^2 + \lambda \|X^\top \alpha\|^2.$$

Let us define the *Gram matrix*

$$K = XX^\top, \quad \text{where} \quad K_{ij} = x_i^\top x_j.$$

The objective becomes

$$J(\alpha) = \|K\alpha - y\|^2 + \lambda\, \alpha^\top K \alpha.$$

Taking the derivative with respect to α and setting it to zero yields

$$(K + \lambda I_N) K\alpha = Ky,$$

which simplifies to

$$(K + \lambda I_N)\alpha = y.$$

Thus, the solution in the dual space is

$$\alpha = (K + \lambda I_N)^{-1} y.$$

The prediction for a new input x is obtained as

$$\hat{y}(x) = x^\top \theta = x^\top X^\top \alpha = k(x)^\top \alpha,$$

where

$$k(x) = \begin{pmatrix} x_1^\top x \\ x_2^\top x \\ \vdots \\ x_N^\top x \end{pmatrix}.$$

Substituting the solution for α, we obtain the dual prediction formula:

$$\hat{y}(x) = k(x)^\top (K + \lambda I_N)^{-1} y.$$

Extension to feature mappings. We can now generalize this result to nonlinear features by introducing a mapping $\phi : \mathbb{R}^D \to \mathbb{R}^M$, and define the feature matrix

$$\Phi = \begin{pmatrix} \phi(x_1)^\top \\ \phi(x_2)^\top \\ \vdots \\ \phi(x_N)^\top \end{pmatrix}.$$

The ridge regression problem becomes

$$J(\theta) = \|\Phi\theta - y\|^2 + \lambda\|\theta\|^2, \quad \text{with solution} \quad \theta = (\Phi^\top\Phi + \lambda I_M)^{-1}\Phi^\top y.$$

Proceeding as before, we let $\theta = \Phi^\top\alpha$, leading to the same dual solution but with a new kernel matrix

$$K = \Phi\Phi^\top, \quad \text{where} \quad K_{ij} = \phi(x_i)^\top\phi(x_j).$$

The prediction for a new input x becomes

$$\hat{y}(x) = k(x)^\top(K + \lambda I_N)^{-1}y, \quad \text{with} \quad k(x)_i = \phi(x_i)^\top\phi(x).$$

Interpretation of the Dual Coefficients

The coefficients $\alpha = (\alpha_1, \ldots, \alpha_N)^\top$ provide an alternative representation of the regression model in the space of training samples. In the primal form, the model parameters θ describe how each feature contributes to the prediction:

$$f(x) = \theta^\top x, \quad \theta \in \mathbb{R}^D.$$

In the dual form, we instead express θ as a linear combination of the training inputs:

$$\theta = X^\top\alpha = \sum_{i=1}^{N} \alpha_i x_i,$$

so that the prediction becomes

$$f(x) = \theta^\top x = \alpha^\top(Xx) = \sum_{i=1}^{N} \alpha_i \, x_i^\top x.$$

Geometric view. This reparametrization corresponds to a change of coordinates. The vector θ lives in the column space of $X^\top$, that is, the subspace spanned by the training samples $\{x_i\}$. Hence, α gives the coordinates of the same model in this basis of training samples. Each α_i specifies how much the training vector x_i contributes to the overall regression direction.

Functional view. From the prediction formula

$$f(x) = \sum_{i=1}^{N} \alpha_i \, x_i^\top x,$$

we see that α_i controls the influence of training point x_i on the prediction at x. A large positive value of α_i makes x_i pull the prediction upward in regions where $x_i^\top x$ is large (high similarity), while a negative value pushes it downward. Hence, each training sample acts as a *basis function* centered at x_i, and the coefficients α_i determine their amplitudes.

Kernel view. When using a nonlinear mapping $\phi(x)$, the model becomes

$$f(x) = \sum_{i=1}^{N} \alpha_i \, \phi(x_i)^\top \phi(x) = \sum_{i=1}^{N} \alpha_i \, k(x_i, x),$$

so that each α_i measures the contribution of the kernel function $k(x_i, \cdot)$ centered at x_i to the regression surface. The function $f(x)$ is thus a weighted combination of kernel functions, with α_i determining the strength and sign of each contribution.

Summary. The dual coefficients α_i can be interpreted as:

- Coordinates of the model in the subspace spanned by the training samples,
- Weights controlling the influence of each training point on predictions,
- Amplitudes of kernel basis functions centered at the data points.

They play the same conceptual role as the parameters θ in the primal formulation, but expressed in the geometry defined by the data and the kernel.

The dual formulation reveals that: i) Ridge regression depends only on inner products between training points, $\phi(x_i)^\top \phi(x_j)$; and, ii) The parameter vector θ never needs to be computed explicitly. This insight is the cornerstone of *kernel methods*: by replacing inner products with a kernel function $k(x_i, x_j)$, we can perform regression or classification in very high, even infinite-dimensional, feature spaces without ever evaluating the mapping $\phi(x)$ explicitly. In the next section, we will formalize this idea through the kernel trick.

6.3. The Kernel Trick

The dual representation of ridge regression revealed that predictions depend only on inner products between feature vectors. This insight allows us to extend linear models to nonlinear ones without explicitly constructing the feature mapping. The key idea is simple but powerful: *replace inner products by a kernel function that computes them implicitly in a high-dimensional space.* This is known as the kernel trick.

From linear to nonlinear inner products. Recall that in the dual form, the prediction for a new input x is given by

$$\hat{y}(x) = k(x)^\top (K + \lambda I_N)^{-1} y,$$

where the elements of $k(x)$ are inner products between the new point and each training point,

$$k(x)_i = \phi(x_i)^\top \phi(x),$$

and the matrix K contains all pairwise inner products between training samples,

$$K_{ij} = \phi(x_i)^\top \phi(x_j).$$

The kernel trick consists of replacing these inner products by the evaluations of a kernel function,

$$k(x_i, x_j) = \phi(x_i)^\top \phi(x_j),$$

without ever computing the feature vectors $\phi(x_i)$ explicitly. By doing so, we can operate as if we were working in a high-dimensional (or even infinite-dimensional) feature space, while all computations remain in the original input space.

The kernel matrix. For a given kernel function $k(x_i, x_j)$, the *kernel matrix* (or Gram matrix) is defined as

$$K = \begin{pmatrix} k(x_1, x_1) & k(x_1, x_2) & \dots & k(x_1, x_N) \\ k(x_2, x_1) & k(x_2, x_2) & \dots & k(x_2, x_N) \\ \vdots & \vdots & \ddots & \vdots \\ k(x_N, x_1) & k(x_N, x_2) & \dots & k(x_N, x_N) \end{pmatrix}.$$

This matrix is symmetric and positive semidefinite, reflecting the fact that it is composed of inner products in a feature space. Its entries measure how similar or "aligned" different samples are according to the geometry defined by the kernel function.

Kernelized ridge regression. Once we substitute inner products by kernel evaluations, the ridge regression prediction function becomes

$$\hat{y}(x) = \sum_{i=1}^{N} \alpha_i \, k(x_i, x), \quad \text{where} \quad \alpha = (K + \lambda I_N)^{-1} y.$$

This is the **kernelized ridge regression model**. It defines a nonlinear function in the input space, but remains linear in the coefficients α_i when expressed in terms of kernels.

Common examples of kernels. Several kernels are widely used in practice, each corresponding to a particular geometry or feature space:

- **Linear kernel:**
$$k(x, x') = x^\top x'.$$
This recovers the standard linear model.

- **Polynomial kernel:**
$$k(x, x') = (1 + x^\top x')^d.$$
This corresponds to an implicit feature space containing all monomials of degree up to d.

- **Gaussian (RBF) kernel:**
$$k(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2\sigma^2} \right).$$
This kernel corresponds to an infinite-dimensional feature space and measures smooth similarity that decays with distance.

Each kernel defines its own notion of similarity between points. For example, the Gaussian kernel measures similarity in terms of Euclidean distance, while the polynomial kernel measures it in terms of correlated feature magnitudes.

Conditions for valid kernels. Not every function of two inputs can serve as a kernel. A function $k(x, x')$ is a *valid kernel* if the corresponding kernel matrix K is symmetric and positive semidefinite for any finite set of inputs. This ensures that k can be interpreted as an inner product in some (possibly infinite-dimensional) Hilbert space, a result known as **Mercer's theorem**.

Summary. The kernel trick allows us to construct nonlinear models without explicitly defining or computing feature mappings. By choosing a kernel function, we implicitly determine:

- The feature space $\phi(x)$,
- The geometry (inner products and distances) of that space,
- The notion of similarity between inputs.

In the next section, we will explore examples of kernelized algorithms and examine how different kernels induce different geometries in the input space.

6.4. Geometric Interpretation of Kernels

Kernel methods may appear abstract at first, but their essence is geometric. They define a notion of similarity and geometry between inputs by implicitly embedding them into a (possibly high-dimensional) feature space. In this section, we provide an intuitive interpretation of how kernels shape the geometry of learning.

From parameter space to data space. In the primal formulation of ridge regression, the model is defined in terms of the parameter vector θ:

$$f(x) = \theta^\top x.$$

In the dual formulation, we express θ as a linear combination of the training samples:

$$\theta = X^\top \alpha,$$

which gives

$$f(x) = \theta^\top x = (X^\top \alpha)^\top x = \alpha^\top (Xx).$$

Here, Xx is the vector of inner products between the new point x and all training samples,

$$Xx = \begin{pmatrix} x_1^\top x \\ x_2^\top x \\ \vdots \\ x_N^\top x \end{pmatrix}.$$

Each entry $(x_i^\top x)$ measures how similar the new point x is to a training point x_i. The prediction $f(x)$ is thus a *weighted combination of similarities* between x and all samples, with weights α_i:

$$f(x) = \sum_{i=1}^{N} \alpha_i (x_i^\top x).$$

The coefficients α_i quantify the contribution of each training sample to the prediction. Training points that are more relevant (e.g. closer to the new input) will typically have larger influence.

Extension to feature mappings. If we map the data through a nonlinear feature transformation $\phi(x)$, the model becomes

$$f(x) = \theta^\top \phi(x) = \sum_{i=1}^{N} \alpha_i \, \phi(x_i)^\top \phi(x).$$

The vector

$$k(x) = \begin{pmatrix} \phi(x_1)^\top \phi(x) \\ \phi(x_2)^\top \phi(x) \\ \vdots \\ \phi(x_N)^\top \phi(x) \end{pmatrix}$$

collects the inner products between the new input and all training samples in the feature space. Replacing these inner products by a kernel function gives

$$f(x) = \sum_{i=1}^{N} \alpha_i \, k(x_i, x) = \alpha^\top k(x).$$

Hence, the kernel function defines the *geometry of similarity* used by the model.

Kernels define geometry. A kernel implicitly defines an embedding of inputs into a feature space where inner products correspond to kernel evaluations:

$$k(x, x') = \phi(x)^\top \phi(x').$$

From this, we can also derive the squared distance between two points in that space:

$$\|\phi(x) - \phi(x')\|^2 = k(x, x) + k(x', x') - 2k(x, x').$$

Thus, the kernel determines both inner products (angles, similarity) and distances in the feature space. For instance:

- The **linear kernel** $k(x, x') = x^\top x'$ preserves the original Euclidean geometry.
- The **polynomial kernel** $k(x, x') = (1 + x^\top x')^d$ corresponds to a space where nonlinear combinations of features become linear.
- The **Gaussian (RBF) kernel** $k(x, x') = \exp(-\|x - x'\|^2 / 2\sigma^2)$ corresponds to an infinite-dimensional feature space where proximity in input space translates into similarity in feature space.

Summary. The kernel function plays the role of an inner product in a (possibly infinite-dimensional) feature space. It defines the geometry of similarity between inputs, determines the notion of distance, and provides the building block for all kernelized algorithms. In the next section, we will apply this geometric viewpoint to extend other linear models, such as PCA and classification, into their kernelized counterparts.

6.5. Kernel Methods for Classification

So far, we have discussed kernel methods in the context of regression. However, the same principles apply to classification problems as well. By expressing linear classifiers in their dual form and replacing inner products by kernel evaluations, we can construct powerful nonlinear classifiers that operate in high-dimensional feature spaces while remaining computationally efficient.

Linear classification. In the simplest linear setting, we model the decision function as

$$f(x) = \theta^\top x + b,$$

and assign a label according to the sign of $f(x)$:

$$\hat{y} = \text{sign}(f(x)).$$

For binary classification with targets $y_i \in \{-1, +1\}$, the parameters θ and b can be estimated by minimizing a loss function that penalizes misclassified points, such as the logistic or hinge loss.

Introducing feature mappings. As before, we can replace the raw input x by a nonlinear feature vector $\phi(x)$, yielding

$$f(x) = \theta^\top \phi(x) + b.$$

The decision boundary is linear in the feature space, but generally nonlinear in the input space. If we again express θ as a linear combination of the training feature vectors,

$$\theta = \sum_{i=1}^{N} \alpha_i y_i \, \phi(x_i),$$

the model becomes

$$f(x) = \sum_{i=1}^{N} \alpha_i y_i \, \phi(x_i)^\top \phi(x) + b = \sum_{i=1}^{N} \alpha_i y_i \, k(x_i, x) + b.$$

Thus, the classifier depends on the training data only through the kernel function $k(x_i, x_j)$.

This expression has a clear geometric meaning: each training point $\phi(x_i)$ defines a direction in the feature space, and the coefficients α_i determine how strongly each of them contributes to the final decision boundary.

Kernel logistic regression. As a concrete example, consider **logistic regression**, where the conditional probability of the positive class is modeled as

$$p(y = 1 \mid x) = \sigma(f(x)) = \frac{1}{1 + \exp(-f(x))}, \quad \text{with} \quad f(x) = \theta^\top \phi(x).$$

In matrix form, for all training samples,

$$f = \Phi\theta, \quad \text{and} \quad \theta = \Phi^\top \alpha.$$

Substituting the dual form into the log-likelihood gives an objective that depends only on the kernel matrix $K = \Phi\Phi^\top$:

$$\mathcal{L}(\alpha) = \sum_{i=1}^{N} \Big[y_i \log \sigma((K\alpha)_i) + (1 - y_i) \log(1 - \sigma((K\alpha)_i)) \Big] - \frac{\lambda}{2} \alpha^\top K \alpha.$$

Optimizing this objective yields the dual coefficients α, and predictions for new inputs follow from

$$p(y = 1 \mid x) = \sigma\left(\sum_{i=1}^{N} \alpha_i \, k(x_i, x) \right).$$

This is the **kernel logistic regression model**, a nonlinear extension of logistic regression that can capture arbitrarily complex decision boundaries, depending on the choice of kernel.

Geometry of kernel classification. The kernel defines the geometry of the feature space in which the linear decision boundary is drawn:

$$f(x) = 0 \quad \Longleftrightarrow \quad \sum_i \alpha_i y_i k(x_i, x) + b = 0.$$

Different kernels produce different shapes of separating surfaces in the input space:

- The linear kernel produces hyperplanes.
- The polynomial kernel yields smooth curved surfaces.

- The Gaussian kernel produces flexible, localized decision boundaries.

Hence, by choosing the kernel, we choose the geometry of the decision boundary.

Summary. Kernel methods unify many classical algorithms—such as logistic regression, and SVMs, under a common framework:

$$f(x) = \sum_{i=1}^{N} \alpha_i y_i \, k(x_i, x) + b.$$

Once the kernel function is specified, all computations depend only on inner products between data points, allowing us to model complex nonlinear structures while retaining the simplicity and interpretability of linear methods.

6.6. Kernel Principal Component Analysis (Kernel PCA)

The kernel framework is not limited to supervised learning. It can also be applied to unsupervised methods such as Principal Component Analysis (PCA), enabling the discovery of nonlinear structures in data. This nonlinear generalization of PCA is known as **Kernel PCA**.

Recap: linear PCA. In ordinary PCA, we seek directions in feature space that maximize the variance of the projected data. Given centered data points $\{x_i\}_{i=1}^N$, we form the covariance matrix

$$\Sigma = \frac{1}{N} X^\top X,$$

and find eigenvectors u_k and eigenvalues λ_k satisfying

$$X^\top X u_k = N \lambda_k u_k.$$

The principal components correspond to the projections

$$z_{ik} = u_k^\top x_i,$$

which represent the coordinates of the samples along each principal direction u_k.

Feature mappings and nonlinear PCA. To capture nonlinear structure, we can first map the data into a higher-dimensional feature space via a nonlinear transformation $\phi(x)$. The goal of Kernel PCA is then to perform standard PCA in that feature space.

Let

$$\Phi = \begin{pmatrix} \phi(x_1)^\top \\ \phi(x_2)^\top \\ \vdots \\ \phi(x_N)^\top \end{pmatrix} \in \mathbb{R}^{N \times M},$$

where M may be very large or even infinite. The empirical covariance matrix in feature space is

$$C = \frac{1}{N} \Phi^\top \Phi.$$

We seek eigenvectors v_k and eigenvalues λ_k such that

$$C v_k = \lambda_k v_k.$$

However, directly computing v_k is infeasible when M is large. We therefore exploit the fact that all solutions lie in the span of the feature vectors $\phi(x_i)$.

Dual formulation. Let us express each eigenvector as a linear combination of training points in feature space:

$$v_k = \sum_{i=1}^{N} \alpha_{ik}\, \phi(x_i).$$

Substituting this expression into the eigenvalue equation and multiplying by $\phi(x_j)^\top$ gives

$$\frac{1}{N} \sum_{i=1}^{N} \alpha_{ik}\, \phi(x_j)^\top \phi(x_i) = \lambda_k \sum_{i=1}^{N} \alpha_{ik}\, \phi(x_j)^\top \phi(x_i).$$

Recognizing the inner products as kernel evaluations $k(x_j, x_i)$, we obtain the dual eigenvalue problem:

$$K\alpha_k = N\lambda_k \alpha_k,$$

where K is the $N \times N$ kernel (Gram) matrix with entries $K_{ij} = k(x_i, x_j)$.

Normalization and projection. The eigenvectors α_k must be normalized to ensure that the corresponding principal directions in feature space have unit norm:

$$\lambda_k = \frac{1}{N} \alpha_k^\top K \alpha_k, \quad \text{and} \quad \alpha_k^\top K \alpha_l = N\delta_{kl}.$$

Once the eigenvectors α_k are obtained, the projection of any data point x onto the k-th principal component is given by

$$z_k(x) = v_k^\top \phi(x) = \sum_{i=1}^{N} \alpha_{ik}\, k(x_i, x).$$

These coordinates $z_k(x)$ represent the nonlinear principal components of x, and can be used for visualization, dimensionality reduction, or anomaly detection.

Centering in feature space. As in standard PCA, it is important that the data be centered before computing the covariance matrix. In the feature space, centering can be achieved implicitly by adjusting the kernel matrix:

$$K_c = K - \mathbf{1}_N K - K\mathbf{1}_N + \mathbf{1}_N K \mathbf{1}_N,$$

where $\mathbf{1}_N$ is an $N \times N$ matrix with all entries equal to $1/N$. All computations in Kernel PCA should use the centered kernel matrix K_c in place of K.

Interpretation. Kernel PCA generalizes the notion of linear variance maximization to nonlinear manifolds. Instead of finding straight axes in the input space, it finds directions of maximal variance in a high-dimensional feature space defined by the kernel. Different kernels correspond to different geometries:

- The linear kernel $k(x, x') = x^\top x'$ reproduces standard PCA.
- The polynomial kernel reveals polynomial manifolds.
- The Gaussian kernel unfolds curved manifolds and nonlinear structures.

Summary. Kernel PCA performs linear PCA in an implicitly defined feature space using only kernel evaluations:

$$K\alpha_k = N\lambda_k\alpha_k, \quad z_k(x) = \sum_{i=1}^{N} \alpha_{ik}\, k(x_i, x).$$

By choosing an appropriate kernel, we can uncover nonlinear patterns in data while relying on the same elegant linear algebra that underlies standard PCA.

6.7. Discussion and Outlook

Kernel methods offer an elegant and powerful framework for building nonlinear models while preserving the mathematical simplicity of linear learning algorithms. By replacing explicit feature maps with kernel functions, we can implicitly operate in very high- or even infinite-dimensional spaces without ever computing those features directly. This geometric abstraction unifies many learning algorithms under a common view based on inner products and similarity.

Strengths of kernel methods. Kernel methods inherit several appealing properties from their linear foundations:

- **Conceptual simplicity.** Most kernel algorithms are derived from linear methods by replacing inner products with kernel evaluations, without changing the core mathematics.
- **Geometric interpretability.** The kernel function defines the notion of similarity and geometry in the feature space. The model remains linear in that space, which ensures convex optimization and a unique global solution for methods like ridge regression or SVMs.
- **Flexibility.** Choosing different kernels allows the model to adapt to complex structures in data, polynomial, periodic, smooth, or localized patterns, without explicitly designing features.
- **Non-parametric behavior.** The number of effective parameters grows with the number of training samples. This enables rich expressiveness without assuming a fixed functional form.

Limitations. Despite their elegance, kernel methods also have practical and conceptual limitations:

- **Computational cost.** The kernel matrix $K \in \mathbb{R}^{N\times N}$ must be computed and stored, which scales quadratically with the number of samples and requires cubic time to invert in standard algorithms. This makes kernel methods challenging for datasets with $N > 10^4$–10^5.
- **Choice of kernel and hyperparameters.** The kernel determines the geometry of the feature space, and choosing it well is crucial. There is no universal kernel: different problems may require different notions of similarity, and kernel parameters (e.g., the RBF bandwidth σ) must be tuned carefully.
- **Lack of explicit features.** While the kernel trick avoids computing features, it also obscures the structure learned by the model. Unlike neural networks or linear models, kernel methods do not provide direct access to interpretable latent representations.

- **Scalability and sparsity.** Kernel regression and kernel PCA typically involve all training points in prediction, making inference costly for large datasets. Approximations such as random Fourier features, Nyström methods, or sparse kernels are often required in practice.

Connections and extensions. Kernel methods are deeply connected to several modern approaches in machine learning:

- **Gaussian Processes (GPs).** Kernel ridge regression and Gaussian process regression are two sides of the same coin: the kernel matrix K plays the role of a covariance matrix in GPs. From a Bayesian viewpoint, GPs place a Gaussian prior over functions with covariance defined by the kernel, leading to identical predictions to kernel ridge regression under specific assumptions.
- **Neural Networks and Deep Kernels.** Infinite-width neural networks converge to kernel machines described by the *Neural Tangent Kernel (NTK)*. This bridges kernel methods and deep learning, showing that kernels can approximate certain neural architectures in the limit of infinite hidden units.
- **Large-scale approximations.** Modern variants such as random Fourier features, inducing-point methods, and scalable GP models address the computational bottlenecks of kernel methods, extending their reach to large datasets.

Outlook. Kernel methods illustrate a central theme in machine learning: the balance between expressiveness and tractability. By defining models through their geometry rather than their parameters, they achieve great flexibility while retaining convex optimization and interpretability. However, their limitations in scalability and feature interpretability have motivated newer paradigms, such as deep neural networks and probabilistic models, that generalize or complement kernel ideas.

In the next chapters, we will revisit kernels from two perspectives:

- as *covariance functions* in Gaussian Process regression, providing a probabilistic interpretation of function spaces;
- and as *limits of neural networks*, revealing deep connections between geometry, representation learning, and inference.

7. Neural Networks

In this chapter we introduce neural networks as a new way to model complex relationships between inputs and outputs. Building on the ideas developed for linear regression, logistic regression, and kernel methods, we will view neural networks as flexible, non-linear parametric models for function approximation. We will first revisit the learning problem from this perspective and explain why linear models and kernels, despite their strengths, can be limiting. We will then define the mathematical structure of a neural network and describe its basic components, discuss the universal approximation theorem and its intuition in one dimension, and present a simple derivation of the backpropagation algorithm on a toy example with just three neurons. Finally, we will see how neural networks can be used in practice for regression, classification, and dimensionality reduction through autoencoders.

7.1. Introduction: function approximation and the need for neural networks

Throughout this course we have treated supervised learning as a problem of function approximation. Given training examples

$$\{(x_i, y_i)\}_{i=1}^n, \qquad x_i \in \mathbb{R}^d,\ y_i \in \mathcal{Y},$$

we seek a function

$$f : \mathbb{R}^d \to \mathcal{Y}$$

such that $f(x_i) \approx y_i$ on the observed data and, more importantly, such that f generalizes well to new, unseen inputs.

Our first approach was to assume that f belongs to a simple parametric family of linear functions,

$$f_\theta(x) = \theta^\top x,$$

with parameter vector $\theta \in \mathbb{R}^d$. Linear regression and logistic regression both fit into this framework, differing only in the choice of output transformation and loss function. In linear regression the model $f_\theta(x) = \theta^\top x$ is used directly and fitted by minimizing a squared error loss. In logistic regression the quantity $\theta^\top x$ is passed through a fixed non-linearity (the sigmoid function) and the parameters are fitted by maximizing the likelihood, or equivalently by minimizing a cross-entropy loss.

These models are attractive because they are simple, their number of parameters is fixed and relatively small, and the associated optimization problems are convex under standard choices of loss. They are also often easy to interpret: each component of θ can be read as the contribution of the corresponding feature. However, they are fundamentally limited by their linearity. The decision boundaries of a linear classifier are hyperplanes, and the predictions of a linear regressor vary linearly along any direction in the input space. When the true relationship between x and y is strongly non-linear, no amount of data will allow a single linear model to capture it accurately.

To go beyond this limitation, we introduced kernel methods. Instead of working directly in the input space, we imagined a feature map $\phi : \mathbb{R}^d \to \mathcal{H}$ into a (possibly high-dimensional) Hilbert space and learned a linear function there:

$$f(x) = w^\top \phi(x).$$

By using a positive definite kernel $k(x, x') = \langle \phi(x), \phi(x') \rangle$, we were able to work with inner products in $\mathcal{H}$ without computing $\phi(x)$ explicitly. This led to flexible non-linear models such as kernel ridge regression and support vector machines, in which the effective capacity of the model grows with the number of training points and the choice of kernel.

Kernel methods thus provide a non-linear, non-parametric approach to function approximation: they can represent very complex functions and rest on a solid theoretical foundation. However, they also have practical limitations. The cost of training and prediction typically grows at least quadratically with the number of training examples, which is problematic for large datasets; their performance depends sensitively on the choice of kernel and its hyperparameters; and, once a kernel has been fixed, the structure of the model is largely determined and difficult to modify.

These issues motivate the search for a different kind of model class. Ideally we would like a family of functions that is parametric, like linear regression, so that we have a finite set of parameters and a clear notion of model size, but also rich and non-linear, like kernel methods, so that it can approximate complicated input–output relationships while scaling more gracefully with data size. Neural networks provide one such family: by composing simple building blocks—affine transformations followed by non-linear activation functions—into multiple layers, they define a parametric function f_θ whose complexity is controlled by architectural choices such as the number and width of layers. In the remainder of this chapter we will make this idea precise, study the expressive power of neural networks via the universal approximation theorem, and see how their parameters can be learned efficiently from data using gradient-based methods.

7.2. Neural networks: definition and basic components

Neural networks are built by composing simple transformations of the input. Each transformation consists of an affine (linear plus bias) mapping, followed by a fixed non-linear function applied componentwise. By stacking several such transformations, we obtain a flexible parametric family of functions f_θ from inputs to outputs.

Neurons and activation functions

The basic building block is the *neuron* (or unit). A neuron takes an input vector $x \in \mathbb{R}^d$, computes a weighted sum of its components plus a bias, and then applies a non-linear activation function. If $w \in \mathbb{R}^d$ is the weight vector and $b \in \mathbb{R}$ is the bias, the neuron computes

$$z = w^\top x + b, \qquad a = \sigma(z),$$

where $\sigma : \mathbb{R} \to \mathbb{R}$ is the activation function and a is the output (or activation) of the neuron.

Typical choices for σ include the sigmoid function $\sigma(z) = (1 + e^{-z})^{-1}$, the hyperbolic tangent $\sigma(z) = \tanh(z)$, and the rectified linear unit (ReLU) $\sigma(z) = \max\{0, z\}$. The activation introduces non-linearity into the model; without it, a network of neurons would reduce to a single linear transformation.

It is useful to notice that logistic regression fits exactly into this framework. There the model computes $z = \theta^\top x$, applies the sigmoid $\sigma(z)$, and interprets the result as a probability. From this point of view, a neural network is simply a generalization in which many such neurons are organized in layers and connected together.

Layers: input, hidden and output

In practice, neurons are grouped into *layers*. A layer takes a vector of inputs and produces a vector of outputs by applying an affine transformation followed by a non-linearity. If a layer has d_{in} inputs and d_{out} outputs, we collect the weights of all its neurons into a matrix $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and the biases into a vector $b \in \mathbb{R}^{d_{\text{out}}}$. Given an input vector $a^{(\ell-1)} \in \mathbb{R}^{d_{\text{in}}}$, the layer computes

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}, \qquad a^{(\ell)} = \sigma^{(\ell)}(z^{(\ell)}),$$

where $\sigma^{(\ell)}$ is an activation function applied componentwise. The superscript (ℓ) denotes the layer index.

A *feedforward neural network* (or multilayer perceptron) with L layers is obtained by composing several such layers. The input vector x is viewed as the activation of a layer 0,

$$a^{(0)} = x,$$

and for each layer $\ell = 1, \dots, L-1$ we compute

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}, \qquad a^{(\ell)} = \sigma^{(\ell)}(z^{(\ell)}).$$

The final layer L produces the network output,

$$z^{(L)} = W^{(L)} a^{(L-1)} + b^{(L)}, \qquad \hat{y} = a^{(L)} = \sigma^{(L)}(z^{(L)}),$$

where the choice of $\sigma^{(L)}$ depends on the task. For regression one often takes $\sigma^{(L)}$ to be the identity (a purely linear output), while for classification one may use, for example, a sigmoid or a softmax function. We will return to these choices later.

By convention, the input vector $a^{(0)} = x$ is said to belong to the *input layer*. The intermediate layers $1, \dots, L-1$ are called *hidden layers*, because their activations are not directly observed in the data. The last layer L is the *output layer*, producing the prediction $\hat{y}$.

The parameters of the network are the collection of all weight matrices and bias vectors,

$$\theta = \left\{ W^{(\ell)}, b^{(\ell)} \;\middle|\; \ell = 1, \dots, L \right\}.$$

For a fixed architecture (number and size of layers, choice of activation functions), a neural network defines a parametric function

$$f_\theta : \mathbb{R}^d \to \mathcal{Y}, \qquad f_\theta(x) = \hat{y},$$

obtained by carrying out these successive affine and non-linear transformations. Changing the parameters θ changes the function f_θ within this family. The learning problem for neural networks consists in choosing θ so that $f_\theta(x_i)$ matches the observed outputs y_i as well as possible, according to a suitable loss function.

7.3. Activation functions

Activation functions play a central role in neural networks. At each layer, the network computes an affine transformation

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}$$

and then applies a non-linear function componentwise,

$$a^{(\ell)} = \sigma^{(\ell)}(z^{(\ell)}).$$

If all $\sigma^{(\ell)}$ were linear, the composition of layers would still be a linear function of the input, and we would gain no expressive power over a single linear model. Non-linear activation functions are therefore essential: they allow the network to represent complex, non-linear relationships between inputs and outputs. They also influence the behavior of the gradients during learning.

In this section we briefly review a few common activation functions and discuss their main advantages and disadvantages.

Sigmoid

The sigmoid activation is defined by

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

It produces outputs in the interval $(0, 1)$ and has an "S"-shaped curve. For large negative z, $\sigma(z)$ is close to 0; for large positive z, it is close to 1; and around $z = 0$ it has the steepest slope.

The sigmoid has a natural probabilistic interpretation: it can be viewed as mapping a real-valued score to a probability. This is why it appears in logistic regression and in the output layer of binary classification networks.

However, the sigmoid also has important drawbacks. Its derivative $\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big)$ becomes very small when $|z|$ is large, so gradients can vanish as signals propagate backward through many layers. In addition, its outputs are always positive, which can make optimization more difficult (activations are not centered around zero).

Hyperbolic tangent

The hyperbolic tangent activation is given by

$$\sigma(z) = \tanh(z).$$

It is also S-shaped, but its range is $(-1, 1)$ instead of $(0, 1)$. Like the sigmoid, it saturates for large $|z|$, but its outputs are approximately zero-centered: negative inputs map to negative outputs, and positive inputs to positive outputs.

This zero-centering can make optimization somewhat easier than with the sigmoid, because the activations entering the next layer tend to have mean closer to zero. Nevertheless, tanh still suffers from saturation and vanishing gradients for large $|z|$, so very deep networks with tanh activations can be hard to train.

ReLU and variants

The rectified linear unit (ReLU) is defined by

$$\sigma(z) = \max\{0, z\}.$$

For negative inputs the output is 0; for positive inputs it grows linearly with slope 1. The derivative is 0 for $z < 0$ and 1 for $z > 0$ (ignoring the point $z = 0$, where it is undefined but can be set arbitrarily for practical purposes).

ReLU has several attractive properties. It is simple to compute, and it does not saturate for large positive inputs, so gradients in the positive region do not vanish as quickly as with sigmoid or tanh. In practice, ReLU activations often lead to faster training and better performance in deep networks. Moreover, ReLU produces sparse activations (many zeros), which can be beneficial both computationally and statistically.

On the other hand, ReLU can suffer from the so-called "dying ReLU" problem: if the input to a unit becomes negative for all training examples, its output is always zero and its gradient is zero as well, so the unit effectively stops learning. Variants such as the leaky ReLU try to mitigate this by allowing a small slope in the negative region, for example

$$\sigma(z) = \begin{cases} z & \text{if } z \geq 0, \\ \alpha z & \text{if } z < 0, \end{cases}$$

with a small constant $\alpha > 0$.

Output activations

The activation function in the output layer is chosen according to the task. For regression it is common to use a linear output, i.e. $\sigma^{(L)}$ is the identity function and the network directly outputs real values. For binary classification one often uses a single sigmoid unit to produce a number in $(0, 1)$ that can be interpreted as a probability. For multi-class classification with K classes, a standard choice is the softmax function, which maps a vector $z \in \mathbb{R}^K$ to a probability vector $\sigma(z) \in (0, 1)^K$ with components

$$\sigma(z)_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad k = 1, \ldots, K.$$

In summary, activation functions determine both the expressiveness of the network and the behavior of gradients during learning. Classical choices such as sigmoid and tanh are smooth and have clear probabilistic interpretations, but can lead to vanishing gradients in deep networks. Piecewise linear functions such as ReLU and its variants are less smooth but often make optimization easier and are widely used in modern deep learning architectures.

7.4. Universal approximation and function representation

One of the most important theoretical results about neural networks is the *universal approximation theorem*. It states, roughly, that even very simple neural network architectures are capable of approximating a large class of functions arbitrarily well, provided they have enough hidden units.

Informal statement of the theorem

Consider a feedforward neural network with a single hidden layer and a fixed activation function σ. For a scalar input $x \in \mathbb{R}$, such a network computes functions of the form

$$f_\theta(x) = \sum_{j=1}^{m} v_j \, \sigma(w_j x + b_j) + c,$$

where m is the number of hidden units, w_j, b_j, v_j are real-valued parameters, and c is an output bias. The universal approximation theorem says that, under mild conditions on σ (for example, σ is continuous, bounded, and not a polynomial), this simple family is extremely rich: for any continuous function f on a compact interval $[a, b]$ and any $\varepsilon > 0$, there exists some number of hidden units m and a choice of parameters θ such that

$$\sup_{x \in [a,b]} |f(x) - f_\theta(x)| < \varepsilon.$$

In words, one-hidden-layer neural networks can approximate any continuous function on a bounded interval, as closely as we wish, by increasing the number of hidden units.

This result is *existential*: it guarantees that suitable parameters exist, but it does not tell us how to find them. Still, it gives an important insight into the expressive power of neural networks.

Intuition in one dimension: building rectangles

To get an intuitive picture of why such a theorem might be true, let us focus on the one-dimensional case, where we want to approximate a continuous function $f : [a, b] \to \mathbb{R}$. A classical idea in analysis is to approximate f by a sum of simple *rectangle* (or step) functions, much like a Riemann sum. Neural networks with a single hidden layer can implement such rectangles using pairs of neurons.

7.5. Learning neural networks: backpropagation on a toy three-neuron chain

Defining a flexible parametric family f_θ is only part of the learning problem. We still need a way to choose the parameters θ so that the network fits the data. As in previous chapters, we do this by minimizing an empirical loss function using gradient descent or one of its variants. The key technical step is to compute the gradient $\nabla_\theta J(\theta)$ of the loss with respect to all parameters. For neural networks this is done by the *backpropagation* algorithm, which is essentially a systematic application of the chain rule.

To illustrate how backpropagation works, we consider the simplest non-trivial network: a chain of three neurons, consisting of an input neuron (which simply outputs the input), one hidden neuron, and one output neuron.

Network definition and forward pass

We take a scalar input $x \in \mathbb{R}$ and a scalar target $y \in \mathbb{R}$. The network has one hidden unit with parameters w_1, b_1 and one output unit with parameters w_2, b_2. We use the same activation function σ in both neurons. The computation proceeds as follows:

$$a^{(0)} = x,$$

$$z^{(1)} = w_1 a^{(0)} + b_1, \qquad a^{(1)} = \sigma\big(z^{(1)}\big),$$
$$z^{(2)} = w_2 a^{(1)} + b_2, \qquad a^{(2)} = \hat{y} = \sigma\big(z^{(2)}\big).$$

The parameters of the network are $\theta = (w_1, b_1, w_2, b_2)$.

For a single training example (x, y) we consider the squared error loss

$$L(\theta) = \frac{1}{2}\big(\hat{y} - y\big)^2 = \frac{1}{2}\big(a^{(2)} - y\big)^2.$$

Our goal is to compute the partial derivatives of L with respect to w_1, b_1, w_2, b_2.

Gradients at the output neuron

We start from the output and move backwards. First note that

$$\frac{\partial L}{\partial a^{(2)}} = a^{(2)} - y = \hat{y} - y.$$

Since $a^{(2)} = \sigma(z^{(2)})$, the chain rule gives

$$\frac{\partial L}{\partial z^{(2)}} = \frac{\partial L}{\partial a^{(2)}} \frac{\partial a^{(2)}}{\partial z^{(2)}} = (\hat{y} - y)\, \sigma'\big(z^{(2)}\big).$$

It is convenient to introduce the *error signal* at the output neuron,

$$\delta^{(2)} = \frac{\partial L}{\partial z^{(2)}} = (\hat{y} - y)\, \sigma'\big(z^{(2)}\big).$$

The pre-activation $z^{(2)}$ depends on w_2 and b_2 via $z^{(2)} = w_2 a^{(1)} + b_2$. Thus

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial z^{(2)}} \frac{\partial z^{(2)}}{\partial w_2} = \delta^{(2)}\, a^{(1)},$$

$$\frac{\partial L}{\partial b_2} = \frac{\partial L}{\partial z^{(2)}} \frac{\partial z^{(2)}}{\partial b_2} = \delta^{(2)}.$$

Gradients at the hidden neuron

Next we propagate the error one step backwards, from the output neuron to the hidden neuron. The loss depends on $z^{(1)}$ through the chain

$$z^{(1)} \;\to\; a^{(1)} \;\to\; z^{(2)} \;\to\; a^{(2)} \;\to\; L.$$

Applying the chain rule,

$$\frac{\partial L}{\partial z^{(1)}} = \frac{\partial L}{\partial a^{(1)}} \frac{\partial a^{(1)}}{\partial z^{(1)}}.$$

Because $z^{(2)} = w_2 a^{(1)} + b_2$,

$$\frac{\partial L}{\partial a^{(1)}} = \frac{\partial L}{\partial z^{(2)}} \frac{\partial z^{(2)}}{\partial a^{(1)}} = \delta^{(2)}\, w_2.$$

Since $a^{(1)} = \sigma(z^{(1)})$,

$$\frac{\partial a^{(1)}}{\partial z^{(1)}} = \sigma'\big(z^{(1)}\big).$$

Combining these expressions,

$$\frac{\partial L}{\partial z^{(1)}} = \delta^{(2)}\, w_2\, \sigma'\big(z^{(1)}\big).$$

We define the error signal at the hidden neuron as

$$\delta^{(1)} = \frac{\partial L}{\partial z^{(1)}} = w_2\, \delta^{(2)}\, \sigma'\big(z^{(1)}\big).$$

Finally, the pre-activation $z^{(1)}$ depends on w_1 and b_1 via $z^{(1)} = w_1 a^{(0)} + b_1 = w_1 x + b_1$. Therefore

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial z^{(1)}}\,\frac{\partial z^{(1)}}{\partial w_1} = \delta^{(1)}\, a^{(0)} = \delta^{(1)}\, x,$$

$$\frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial z^{(1)}}\,\frac{\partial z^{(1)}}{\partial b_1} = \delta^{(1)}.$$

Summary and generalization

Even in this tiny network, the pattern of backpropagation is visible:

- In a *forward pass* we compute all pre-activations and activations $(z^{(1)}, a^{(1)}, z^{(2)}, a^{(2)})$ from the input x.
- In a *backward pass* we start from the loss, compute the error signal at the output $\delta^{(2)}$, then propagate it backwards to obtain $\delta^{(1)}$.
- At each neuron, the gradient with respect to its weights is given by the product of its error signal and its input activation; the gradient with respect to its bias is the error signal itself.

For networks with many layers and many units per layer, the same ideas apply, with scalars replaced by vectors and matrices. Error signals $\delta^{(\ell)} = \partial L/\partial z^{(\ell)}$ are propagated backward through the layers using the chain rule, and parameter gradients are obtained locally from these signals and the activations $a^{(\ell-1)}$. Modern automatic differentiation libraries implement this procedure efficiently, so in practice we specify the network architecture and the loss function, and the gradients are computed automatically by backpropagation.

7.6. Neural networks for regression and classification

So far we have defined neural networks as flexible parametric functions $f_\theta : \mathbb{R}^d \to \mathcal{Y}$, obtained by composing affine transformations and non-linear activations. To use these models in practice we must adapt their architecture, and in particular their *output layer* and *loss function*, to the nature of the prediction task. In this section we focus on two basic cases: regression and classification.

Regression

In regression problems the target is real-valued,

$$y \in \mathbb{R}^k,$$

for example a single scalar ($k = 1$) or a small vector of continuous quantities. In this setting it is natural for the network to output real numbers without constraining them to a bounded interval. This is achieved by using a *linear* output layer, that is, an output layer without non-linearity.

Suppose the network has L layers. The hidden layers $\ell = 1, \dots, L-1$ use non-linear activations as before,

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}, \qquad a^{(\ell)} = \sigma^{(\ell)}\big(z^{(\ell)}\big),$$

with $a^{(0)} = x$. The output layer L is defined by

$$z^{(L)} = W^{(L)} a^{(L-1)} + b^{(L)}, \qquad a^{(L)} = \hat{y} = z^{(L)}.$$

Here the activation in the last layer is the identity function: we take the pre-activation $z^{(L)}$ directly as the network prediction $\hat{y}$.

Given training examples $\{(x_i, y_i)\}_{i=1}^n$, a standard choice of loss is the mean squared error,

$$J(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell\big(\hat{y}_i, y_i\big) = \frac{1}{n} \sum_{i=1}^{n} \frac{1}{2} \big\|\hat{y}_i - y_i\big\|^2,$$

where $\hat{y}_i = f_\theta(x_i)$ is the output of the network for input x_i. This generalizes linear regression: instead of a single affine map $x \mapsto \theta^\top x$, we now have a deep composition of affine and non-linear transformations culminating in a linear output.

Binary classification

In binary classification the target takes values in $\{0, 1\}$ or $\{-1, +1\}$. It is convenient to represent the target as $y \in \{0, 1\}$ and to interpret the network output as an estimate of the conditional probability $\mathbb{P}(Y = 1 \mid X = x)$. This suggests using a single output neuron with a sigmoid activation.

The hidden layers are defined as before. The output layer has one unit,

$$z^{(L)} = W^{(L)} a^{(L-1)} + b^{(L)} \in \mathbb{R},$$

$$a^{(L)} = \hat{p}(x) = \sigma\big(z^{(L)}\big) = \frac{1}{1 + e^{-z^{(L)}}},$$

where $\hat{p}(x) \in (0, 1)$ is interpreted as the predicted probability that $y = 1$ given x. A hard classification decision can be obtained by thresholding $\hat{p}(x)$, for example predicting class 1 if $\hat{p}(x) \geq 1/2$ and class 0 otherwise.

For this probabilistic interpretation, the natural loss function is the cross-entropy (negative log-likelihood). For a single training example (x, y) with $y \in \{0, 1\}$, the loss is

$$\ell\big(\hat{p}(x), y\big) = -\Big[y \log \hat{p}(x) + (1 - y) \log\big(1 - \hat{p}(x)\big)\Big].$$

For a dataset $\{(x_i, y_i)\}_{i=1}^n$, the empirical loss is

$$J(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell\big(\hat{p}(x_i), y_i\big).$$

This is precisely the logistic regression loss, but with $\hat{p}(x)$ produced by a deep network instead of a single linear model.

Multi-class classification

In multi-class classification the target belongs to a finite set of classes, $\mathcal{C} = \{1, \dots, K\}$. It is convenient to represent the target as a one-hot vector $y \in \mathbb{R}^K$, where $y_k = 1$ if the true class is k and $y_j = 0$ for $j \neq k$. The network should output a vector of class probabilities $\hat{p}(x) \in (0,1)^K$ with components $\hat{p}_k(x) \approx \mathbb{P}(Y = k \mid X = x)$ and $\sum_{k=1}^K \hat{p}_k(x) = 1$.

To achieve this, the output layer has K units and uses the softmax activation. The hidden layers are as before, and the last layer computes

$$z^{(L)} = W^{(L)} a^{(L-1)} + b^{(L)} \in \mathbb{R}^K,$$

$$a^{(L)} = \hat{p}(x) = \operatorname{softmax}\big(z^{(L)}\big),$$

with components

$$\hat{p}_k(x) = \frac{\exp\big(z_k^{(L)}\big)}{\sum_{j=1}^K \exp\big(z_j^{(L)}\big)}, \qquad k = 1, \dots, K.$$

The appropriate loss is again the cross-entropy between the true distribution (represented by the one-hot vector y) and the predicted distribution $\hat{p}(x)$. For a single example,

$$\ell\big(\hat{p}(x), y\big) = -\sum_{k=1}^K y_k \, \log \hat{p}_k(x).$$

If the true class is c, then $y_c = 1$ and $y_k = 0$ for $k \neq c$, so this simplifies to

$$\ell\big(\hat{p}(x), y\big) = -\log \hat{p}_c(x).$$

For a dataset, the empirical loss is

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n \ell\big(\hat{p}(x_i), y_i\big).$$

Summary

In summary, the architecture of a neural network is largely determined by the prediction task through the design of the output layer and the choice of loss function. For regression we use a linear output and a squared error loss; for binary classification, a sigmoid output and a cross-entropy loss; for multi-class classification, a softmax output and a multi-class cross-entropy loss. The hidden layers can be shared across these tasks; what changes is how the final activations are interpreted and how the discrepancy between predictions and targets is measured.

7.7. Autoencoders and nonlinear dimensionality reduction

In previous chapters we introduced *dimensionality reduction* as the task of representing high-dimensional data $x \in \mathbb{R}^d$ using a smaller number of variables that capture most of the relevant structure. The main example was Principal Component Analysis (PCA), which finds a low-dimensional linear subspace that best approximates the data in a least-squares sense. In this section we describe *autoencoders*, a class of neural network models that can be seen as a nonlinear generalization of PCA.

Architecture: encoder, bottleneck and decoder

An autoencoder is a neural network trained to reproduce its input at the output. Given an input $x \in \mathbb{R}^d$, the network computes a *latent representation* (or code) $z \in \mathbb{R}^r$, with $r < d$, through an *encoder* network, and then reconstructs x from z through a *decoder* network:

$$x \xrightarrow{\text{encoder}} z \xrightarrow{\text{decoder}} \hat{x}.$$

More concretely, the encoder is a function

$$\text{enc}_\phi : \mathbb{R}^d \to \mathbb{R}^r, \qquad z = \text{enc}_\phi(x),$$

implemented by a neural network with parameters ϕ, and the decoder is a function

$$\text{dec}_\psi : \mathbb{R}^r \to \mathbb{R}^d, \qquad \hat{x} = \text{dec}_\psi(z),$$

implemented by another neural network with parameters ψ. The full autoencoder is the composition

$$f_{\phi,\psi}(x) = \text{dec}_\psi\big(\text{enc}_\phi(x)\big) = \hat{x}.$$

If we denote by $a^{(0)} = x$ the input activations and apply a sequence of layers in the encoder,

$$z = a^{(L_{\text{enc}})} = \text{enc}_\phi(x),$$

the vector z is the activation at the *bottleneck layer*. Its dimension r is chosen smaller than d, so the network is forced to compress the information in x into a lower-dimensional code before reconstructing it.

Training objective and reconstruction loss

The parameters ϕ and ψ are learned by minimizing a *reconstruction loss* that measures how close the output $\hat{x}$ is to the input x. For real-valued data, a standard choice is the mean squared error:

$$J(\phi, \psi) = \frac{1}{n} \sum_{i=1}^{n} \ell\big(\hat{x}_i, x_i\big) = \frac{1}{n} \sum_{i=1}^{n} \frac{1}{2} \big\|\hat{x}_i - x_i\big\|^2,$$

where $\hat{x}_i = f_{\phi,\psi}(x_i)$ is the reconstruction of x_i by the autoencoder. For binary or bounded data, other losses such as cross-entropy can be used.

From the optimization point of view, training an autoencoder is similar to training any other neural network: we define the architecture (encoder and decoder), choose a loss function, and apply backpropagation with gradient descent to minimize the empirical loss. What distinguishes autoencoders is the symmetry between input and output and the presence of a low-dimensional bottleneck.

Once trained, the encoder $\text{enc}_\phi(x)$ provides a low-dimensional representation z of the data, which can be used for visualization, clustering, or as input features for other learning tasks.

Linear autoencoders and PCA

To understand the connection with PCA, consider a very simple autoencoder with only one hidden layer, linear activations, and no additional constraints. The encoder and decoder are given by

$$z = W_{\text{enc}} x, \qquad \hat{x} = W_{\text{dec}} z = W_{\text{dec}} W_{\text{enc}} x,$$

where $W_{\text{enc}} \in \mathbb{R}^{r \times d}$ and $W_{\text{dec}} \in \mathbb{R}^{d \times r}$ are matrices, and we use a squared reconstruction loss. In this purely linear setting, the autoencoder is seeking a rank-r matrix $M = W_{\text{dec}} W_{\text{enc}}$ that best approximates the identity transformation on the data:

$$x \approx Mx, \qquad \text{rank}(M) \leq r.$$

It can be shown that, under mild conditions (for example, when we restrict W_{enc} and W_{dec} appropriately), the optimal solution spans the same subspace as the top r principal components found by PCA. Thus, a linear autoencoder with a bottleneck layer and squared reconstruction error essentially recovers PCA as its dimensionality reduction method.

This observation justifies the view of autoencoders as a generalization of PCA: when all activations are linear, we fall back to PCA; when we introduce non-linear activation functions and deeper architectures, we obtain nonlinear extensions.

Nonlinear autoencoders as a generalization of PCA

In practice, the power of autoencoders comes from using nonlinear activation functions (such as ReLU or tanh) and multiple hidden layers in the encoder and decoder. The encoder then maps the data to a low-dimensional manifold in $\mathbb{R}^r$ in a nonlinear way, and the decoder learns how to map this manifold back to the original space.

From the point of view of dimensionality reduction, the latent code $z = \text{enc}_\phi(x)$ plays the role of a low-dimensional coordinate system, analogous to the principal component scores in PCA, but adapted to capture nonlinear structure in the data. Where PCA searches for a linear subspace that minimizes reconstruction error, a nonlinear autoencoder searches for a nonlinear manifold parameterized by z, together with a nonlinear projection onto that manifold and a reconstruction map.

Because autoencoders are trained by gradient-based optimization on a non-convex objective, there is no guarantee of finding a global optimum, and the learned representations can depend on initialization and architectural choices. Nonetheless, in many situations nonlinear autoencoders can discover low-dimensional representations that capture complex structures—such as curved manifolds or clusters—that PCA, constrained to linear subspaces, cannot easily represent.

In summary, autoencoders extend the idea of PCA from linear subspaces to nonlinear manifolds, using neural networks as flexible function approximators for both the encoding and decoding maps. They illustrate how the tools developed in this chapter—neural architectures, activation functions and backpropagation—can be used not only for prediction, but also for learning compact, informative representations of data.

7.8. Practical aspects: training stable and generalizing networks

So far we have focused on the definition of neural network architectures, their expressive power, and the basic learning mechanism via backpropagation. In practice, however, the performance of a neural network depends not only on its mathematical form, but also on a collection of design and training choices: data preprocessing, parameter initialization, regularization, and the tuning of hyperparameters such as the learning rate. In this section we briefly review a few practical aspects that are important for training stable networks that generalize well.

Data preprocessing and data splits

Before training a neural network, it is standard to preprocess the data. For tabular data with numerical features, a simple and effective step is to center each feature and scale it to have unit variance. This makes the scale of the inputs more homogeneous, which helps gradient-based optimization and makes initialization rules behave as intended.

As in previous chapters, we assume that the available data are split into at least two parts: a *training set*, used to fit the parameters by minimizing the loss; and a *test set*, used only once at the end to evaluate performance. In addition, it is common to set aside a *validation set* that is used to monitor training (for example to decide when to stop) and to tune hyperparameters such as the learning rate or the strength of regularization. These hyperparameters should be chosen based on the validation set, not the test set.

In stochastic gradient descent and its variants, the training set is typically shuffled and divided into small *mini-batches*. Each update step uses the gradient of the loss computed on one mini-batch. This introduces some noise into the gradient estimates, which often helps escape poor local minima and saddle points, and allows us to train efficiently on large datasets.

Initialization of weights and biases

The way in which the parameters are initialized has a strong influence on the dynamics of learning. If all weights are set to zero, all units in a given layer behave identically and remain symmetric, so the network does not exploit its full capacity. If the initial weights are too large, the activations $z^{(\ell)}$ and their gradients can explode; if they are too small, signals may shrink as they propagate through the layers, leading to vanishing activations and gradients.

Weights: A simple and effective strategy is to initialize the weights at each layer with independent random values of zero mean and a variance that depends on the size of the layer. For example, suppose layer ℓ has n_{in} inputs and n_{out} outputs. Initialization schemes such as Glorot (or Xavier) initialization choose the entries of $W^{(\ell)}$ with variance proportional to $1/(n_{\text{in}} + n_{\text{out}})$, which tends to keep the scale of activations and gradients approximately constant across layers for smooth activations like tanh. For ReLU activations, He initialization uses a variance proportional to $1/n_{\text{in}}$, reflecting the fact that roughly half of the activations are zero.

Biases: Biases $b^{(\ell)}$ are often initialized to zero or to small constants. In practice, modern deep learning libraries implement sensible default initializations, but it is useful to be aware that these rules are designed to avoid exploding or vanishing signals at the start of training.

Regularization and control of model capacity

Neural networks can have a very large number of parameters and therefore very high capacity. Without some form of regularization, they may fit the training data extremely well but fail to generalize to new examples. Several techniques are commonly used to control this behavior.

Penalty-based regularization: A first approach is to add a penalty on the size of the weights to the loss function. For example, *weight decay* corresponds to adding an L_2 penalty

$$J_{\text{reg}}(\theta) = J(\theta) + \lambda \sum_{\ell} \|W^{(\ell)}\|_F^2,$$

where $J(\theta)$ is the original empirical loss, $\|\cdot\|_F$ is the Frobenius norm, and $\lambda > 0$ controls the strength of the penalty. This encourages small weights and tends to produce smoother functions that are less sensitive to small variations in the input.

Early stopping: A second, very practical technique is *early stopping*. During training we monitor the loss on the validation set as well as on the training set. Often the training loss continues to decrease, while the validation loss reaches a minimum and then starts to increase, indicating overfitting. Early stopping simply means that we stop training when the validation performance stops improving, and keep the parameters corresponding to the best validation loss.

Dropout layer: Another widely used regularization method is *dropout*. During training, for each mini-batch and for each layer, we randomly set a fraction of the activations to zero:

$$a^{(\ell)} \leftarrow m^{(\ell)} \odot a^{(\ell)},$$

where $m^{(\ell)}$ is a random mask with entries equal to 0 or 1, and $\odot$ denotes elementwise multiplication. Intuitively, dropout prevents the network from relying too heavily on any single unit, and can be seen as averaging over an ensemble of many slightly different networks. At test time, we use the full network but rescale the activations to match the expected values during training.

Other forms of regularization exist, including architectural choices such as reducing the number of units or layers, or constraining the norms of the weights in specific ways. The overarching goal of regularization methods remains the same: to reduce overfitting by limiting the effective complexity of the functions that the network can represent.

Optimization hyperparameters

Gradient-based optimization methods for neural networks involve several hyperparameters. The most important one is the *learning rate*, which controls the size of the steps taken in parameter space. If the learning rate is too small, training progresses very slowly; if it is too large, the loss may oscillate or diverge. In practice, one usually tries a few candidate values on a validation set and sometimes decreases the learning rate during training according to a schedule.

The *mini-batch size* is another important choice. Small mini-batches provide noisy gradient estimates, which can help escape shallow local minima and saddle points, and can be more memory-efficient. Large mini-batches produce smoother gradients and can take advantage of parallel hardware, but may require more careful tuning of the learning rate.

The *number of epochs* (full passes over the training set) determines how long we train. If we stop too early, the network may underfit; if we train for too long without sufficient regularization, it may overfit. Early stopping can be viewed as a way of choosing the number of epochs based on validation performance.

Beyond plain stochastic gradient descent, many variants exist (such as momentum methods or Adam) that adapt the step sizes during training. In this course we will mostly treat them as black boxes and focus on the conceptual aspects of optimization.

Hyperparameter tuning and diagnostics

Finally, the architecture and training process of a neural network involve hyperparameters that cannot be learned directly by gradient descent, such as: the number of layers and units per layer, the choice of activation functions, the learning rate, the batch size,

the regularization strength λ, or the dropout rate. These choices are typically made by *hyperparameter tuning*.

A simple and effective approach is to define a set of candidate values for each hyperparameter (for example, several learning rates on a logarithmic scale, several network widths, and a few values of λ), and to evaluate a small number of combinations on the validation set. The combination with the best validation performance is then selected, and a final model is trained (possibly on the union of training and validation data) before evaluating on the test set.

The evolution of the training and validation losses during learning provides useful diagnostics. If both losses remain high, the model may be underfitting, and we might need a larger network or a higher learning rate. If the training loss becomes very small while the validation loss increases, the model is overfitting, and we may need stronger regularization, fewer epochs, or a simpler architecture. These patterns connect back to the bias–variance tradeoff discussed in earlier chapters, but now in the context of neural networks.

8. Tree-Based Methods

In the previous chapters we have developed several families of models for prediction and classification: linear and logistic regression, dimensionality reduction and clustering methods, regularized linear models, kernel methods, and neural networks. These approaches provide powerful tools to approximate complex input–output relationships, but they also come with limitations. Linear models struggle to capture strong interactions between features unless these are carefully engineered, kernel methods can become computationally expensive on large datasets, and neural networks often require substantial tuning and are less transparent in their decision-making process.

In many practical applications, however, the data are *tabular*: a moderate number of heterogeneous features (numerical, categorical, ordinal) describing each observation, with no particular spatial or temporal structure. In this setting, **tree-based methods** offer an attractive alternative. They can naturally handle mixed data types, automatically discover nonlinear interactions between features, and often provide human-readable decision rules. Moreover, by combining many trees into ensembles such as random forests or boosting methods, we obtain some of the most accurate and robust models in modern machine learning practice.

In this chapter we introduce decision trees as models that partition the input space into simple regions and make predictions based on the training examples within each region. We will then see how to turn a single, unstable tree into a strong predictor by averaging many trees (bagging and random forests) or by building them sequentially to correct each other's errors (boosting and gradient boosting). Throughout, we will connect these ideas to the concepts of bias, variance, and regularization developed earlier in the course.

8.1. Motivation and Overview

To motivate tree-based methods, let us start from the general supervised learning problem. We observe pairs (x_i, y_i), where $x_i \in \mathbb{R}^p$ is a feature vector and y_i is a target (real-valued for regression or categorical for classification). Our goal is to learn a function

$$f(x) \approx \mathbb{E}[Y \mid X = x],$$

which can be used to make accurate predictions for new inputs.

In linear regression we assumed that this function could be approximated by a linear combination of the components of x,

$$\hat{y}(x) = \theta^\top x,$$

and we saw how regularization controls the complexity of this model. Kernel methods extend this idea by implicitly mapping x to a high-dimensional feature space, and neural networks replace the fixed feature map by a flexible composition of learned nonlinear transformations. All these approaches, however, still rely on representing f as a relatively smooth function of the inputs.

Decision trees adopt a completely different viewpoint. Instead of trying to fit a single global function over the whole input space, they *recursively partition* the space into smaller

and smaller regions and approximate f by a *simple* model (in fact, a constant) within each region. Concretely, a decision tree builds a sequence of tests of the form

$$x_\alpha \leq t \quad \text{or} \quad x_\alpha > t,$$

for some feature index α and threshold t. Each test splits the data into two groups, and by repeatedly applying such splits we obtain a collection of disjoint regions $R_1, R_2, \ldots, R_M$ that cover the input space. The prediction for a new point x is simply the average (for regression) or majority class (for classification) of the training examples that fall in the same region as x.

This construction has several important consequences:

- The resulting model is **highly nonlinear**: the decision boundary is piecewise constant and can approximate complex shapes by combining many axis-aligned splits.
- The model naturally captures **interactions between features**. A path from the root of the tree to a leaf corresponds to a conjunction of conditions involving different features (for example, "if `age` ≤ 30 and `income` $>$ 40k then predict class 1").
- The sequence of tests along a path is easy to read and explain, which gives decision trees a degree of **interpretability** that is often lacking in kernel methods or neural networks.

On the other hand, a single decision tree is typically a **high-variance** model: small changes in the training data can lead to different splits and a different structure of the tree, and a very deep tree can overfit the training set badly. In the rest of the chapter we will address these issues in two stages. First, we will study how to construct and regularize individual trees through appropriate splitting criteria and pruning strategies. Then we will introduce *ensemble* methods such as bagging, random forests, and boosting, which aggregate the predictions of many trees to obtain models that are both powerful and stable.

8.2. Decision Trees for Regression and Classification

We now describe decision trees more formally, and introduce the notation that we will use throughout the rest of the chapter. We consider a supervised learning setting with training data

$$\{(x_i, y_i)\}_{i=1}^n,$$

where each input vector $x_i \in \mathbb{R}^p$ has components

$$x_i = (x_{i1}, x_{i2}, \ldots, x_{ip}).$$

From now on, we will use *Latin* indices $i, j, k, \ldots$ to denote *observations* and *Greek* indices $\alpha, \beta, \gamma, \ldots$ to denote *features*. Thus we will write the α-th component of the i-th observation as $x_{i\alpha}$, with $\alpha = 1, \ldots, p$.

A decision tree recursively partitions the input space into a finite collection of disjoint regions $R_1, R_2, \ldots, R_M$, and then associates a simple prediction to each region. For regression, this prediction is typically the average of the target values in that region; for classification, it is the most frequent class among the training examples that fall into the region.

Trees as Recursive Partitioning

Each internal node of a decision tree corresponds to a *test* on a single feature of the form

$$x_{i\alpha} \leq t \quad \text{or} \quad x_{i\alpha} > t,$$

for some feature index α and threshold $t \in \mathbb{R}$. The test splits the data at that node into two subsets, which are sent to the left and right child nodes respectively. By starting from a root node containing all training points and recursively applying such tests, we obtain a binary tree in which each terminal node (or *leaf*) defines a region R_m in the input space. The collection of leaves $\{R_m\}_{m=1}^{M}$ forms a partition of the space covered by the training data:

$$R_m \cap R_{m'} = \emptyset \quad \text{for } m \neq m', \qquad \bigcup_{m=1}^{M} R_m = \text{input space}.$$

For a new input x, prediction proceeds by sending x down the tree: starting at the root, we evaluate the test at each internal node and follow the corresponding branch until we reach a leaf. The prediction $\hat{f}(x)$ is then determined by the training examples that belong to the same region as x.

Regression trees. In regression, a natural choice for the prediction in region R_m is the average of the target values of the training points that fall in that region. Let

$$R_m = \{i \in \{1, \ldots, n\} : x_i \in R_m\}$$

also denote the set of indices of training examples in region m. Then the prediction for any $x \in R_m$ is

$$\hat{f}(x) = \bar{y}_m \quad \text{with} \quad \bar{y}_m = \frac{1}{|R_m|} \sum_{i \in R_m} y_i.$$

Thus a regression tree defines a piecewise-constant approximation to the regression function: $\hat{f}$ is constant on each region R_m and can approximate complex functions by using many small regions.

Classification trees. In classification, suppose that the target variable Y takes values in a finite set of classes $\mathcal{C} = \{1, \ldots, K\}$. For each region R_m we can estimate the class probabilities as

$$\hat{\pi}_{mc} = \frac{1}{|R_m|} \sum_{i \in R_m} \mathbb{I}(y_i = c), \qquad c = 1, \ldots, K,$$

where $\mathbb{I}(\cdot)$ denotes the indicator function. The simplest prediction rule assigns to any $x \in R_m$ the most likely class in that region:

$$\hat{f}(x) = \arg\max_{c \in \mathcal{C}} \hat{\pi}_{mc}.$$

More generally, the vector $(\hat{\pi}_{m1}, \ldots, \hat{\pi}_{mK})$ provides an estimate of the conditional class probabilities in region R_m.

Impurity Measures and Split Criteria

To grow a tree we need a rule to decide, at each node, which feature α and which threshold t to choose for the next split. Intuitively, we would like each split to produce child nodes that are "as homogeneous as possible" in terms of the target variable. This notion of homogeneity is formalized through a node *impurity* measure.

Regression: variance reduction. For regression trees, a natural impurity measure for a region R_m is the mean squared deviation of the targets from their region mean:

$$Q(R_m) = \frac{1}{|R_m|} \sum_{i \in R_m} \big(y_i - \bar{y}_m\big)^2.$$

A good split is one that leads to a large reduction in this impurity. Suppose we are considering splitting a node corresponding to region R into two child regions R_{left} and R_{right} using a test of the form $x_{i\alpha} \leq t$. The impurity after the split is the weighted average

$$Q_{\text{split}} = \frac{|R_{\text{left}}|}{|R|} Q(R_{\text{left}}) + \frac{|R_{\text{right}}|}{|R|} Q(R_{\text{right}}),$$

and the *impurity decrease* (or *variance reduction*) is

$$\Delta Q = Q(R) - Q_{\text{split}}.$$

The training algorithm will search over candidate features α and thresholds t and select the split that maximizes ΔQ.

Classification: Gini impurity and entropy. For classification trees, we define impurity in terms of the class proportions $\hat{\pi}_{mc}$ in a region R_m. Two popular impurity measures are:

- The **Gini impurity**:

$$G(R_m) = \sum_{c=1}^{K} \hat{\pi}_{mc}(1 - \hat{\pi}_{mc}) = 1 - \sum_{c=1}^{K} \hat{\pi}_{mc}^2.$$

  This quantity is small when one class dominates in the node, and attains its maximum when all classes are equally likely.

- The **entropy**:

$$H(R_m) = -\sum_{c=1}^{K} \hat{\pi}_{mc} \log \hat{\pi}_{mc},$$

  with the convention that $0 \log 0 = 0$. This is the same entropy function that appears in information theory and in logistic regression.

Given a parent region R and a candidate split into R_{left} and R_{right}, we compute the impurity of each node (using Gini or entropy) and form the weighted average

$$I_{\text{split}} = \frac{|R_{\text{left}}|}{|R|} I(R_{\text{left}}) + \frac{|R_{\text{right}}|}{|R|} I(R_{\text{right}}),$$

where I denotes either G or H. The *information gain* of the split is then

$$\Delta I = I(R) - I_{\text{split}}.$$

The training algorithm selects the feature index α and threshold t that maximize ΔI. In practice, Gini impurity and entropy typically lead to very similar trees; Gini is slightly cheaper to compute and is the default in many software libraries.

Greedy Top-Down Tree Growing

In principle, one could try to choose all splits in a tree jointly so as to minimize a global loss function on the training data. In practice, this would be computationally intractable. Indeed, finding the globally optimal tree is an NP-hard problem. Instead, decision tree algorithms follow a *greedy* top-down approach, also known as *recursive binary splitting*.

Starting from the root node containing all training examples, the algorithm repeats the following steps:

1. For the current node with region R, consider a set of candidate splits of the form $x_{i\alpha} \leq t$, where α ranges over the available features and t ranges over possible thresholds. For each candidate, compute the impurity decrease ΔQ (regression) or information gain ΔI (classification).
2. Select the feature index α^* and threshold t^* that achieve the largest impurity decrease.
3. Split the node using the test $x_{i\alpha^*} \leq t^*$, create the corresponding left and right child nodes, and assign each training example in R to one of the children according to its value of $x_{i\alpha^*}$.
4. Repeat the procedure recursively on each child node until a stopping condition is met.

Typical stopping conditions include:

- A maximum depth of the tree (maximum number of splits along any path from the root to a leaf).
- A minimum number of observations $|R_m|$ required to further split a node.
- A minimum impurity decrease: if the best possible split at a node does not reduce impurity by at least a given threshold, we stop splitting.

The result of this greedy procedure is a tree that is adapted to the structure of the data: dense regions with complex relationships between features and the target will tend to be partitioned more finely, while simpler regions will remain coarse. However, because the algorithm is greedy and local, the final tree is not guaranteed to be globally optimal. Moreover, as we will see in the next section, trees grown in this way can easily overfit the training data if we allow them to become too deep, which motivates the need for appropriate regularization and pruning strategies.

8.3. Overfitting and Regularization of Trees

As we have seen, decision trees construct a piecewise-constant approximation to the regression or classification function by recursively partitioning the input space. This flexibility allows trees to adapt closely to the training data, but it also makes them prone to overfitting. In this section we connect the behavior of trees to the bias–variance trade-off introduced earlier in the course and discuss strategies to control their complexity.

Bias–Variance Behavior of Trees

Recall that the prediction error of a model at a point x can be decomposed as

$$\mathbb{E}\big[(Y - \hat{f}(x))^2\big] = \underbrace{\big(\mathbb{E}[\hat{f}(x)] - f(x)\big)^2}_{\text{bias}^2} + \underbrace{\mathbb{E}\big[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2\big]}_{\text{variance}} + \underbrace{\sigma^2}_{\text{irreducible noise}},$$

where the expectation is over the randomness in the training sample and $\sigma^2 = (Y \mid X = x)$ is the noise variance.

A very *shallow* tree, with only a few splits, partitions the input space into a small number of large regions. In each region, the model can only make a coarse approximation to the underlying function $f(x)$, which leads to high bias but low variance: different training samples will typically produce similar trees.

At the opposite extreme, a very *deep* tree can create many small regions containing only a handful of observations. In the limit, a tree could keep splitting until most leaves contain just one training point, achieving zero training error. In this case the model has very low bias but extremely high variance: small changes in the training data (adding, removing, or slightly perturbing some points) can lead to very different splits and predictions.

If we plot the training error and the test error of a tree as a function of its depth, we typically observe the same general pattern as for other flexible models:

- The **training error** decreases monotonically with tree depth, since additional splits can only improve (or leave unchanged) the fit on the training data.
- The **test error** typically decreases at first, as the model becomes more flexible and captures structure in the data, but then starts to increase when the tree becomes too deep and begins to overfit. This gives rise to the familiar U-shaped test error curve as a function of model complexity.

Because of this high variance behavior, a single deep tree is often a poor predictor on its own. The next sections discuss how we can control the complexity of a single tree (regularization) and, later in the chapter, how we can combine many trees into ensembles that achieve low variance without increasing bias too much.

Pre-Pruning: Stopping Tree Growth Early

The basic greedy algorithm described above will keep splitting nodes as long as it can find some split that reduces impurity. If left unchecked, this will usually produce a very large tree with many leaves and a high risk of overfitting. A simple way to regularize trees is to stop their growth early by imposing constraints during training, often called *pre-pruning*.

Typical pre-pruning strategies include:

- **Maximum depth.** Limit the depth of the tree, that is, the number of splits along any path from the root to a leaf. For example, a tree of depth 1 has a single split (also called a *stump*), a tree of depth 2 can have up to four leaves, and so on.
- **Minimum number of observations per leaf.** Impose a lower bound on $|R_m|$, the number of training points in each region. This prevents the algorithm from creating leaves that are based on only a few observations, which would be highly unstable.
- **Minimum impurity decrease.** Require that a split produces an impurity decrease ΔQ (regression) or information gain ΔI (classification) above a certain threshold. If no candidate split at a node achieves this, we stop splitting that node.

These hyperparameters play a role similar to regularization parameters in linear models: they control the effective complexity of the tree. Smaller maximum depth, larger minimum leaf size, or larger minimum impurity decrease all lead to simpler trees with higher bias and lower variance. In practice, their values are chosen by cross-validation, just as we did for ridge and lasso regression.

Post-Pruning and Cost-Complexity Regularization

An alternative strategy is to first grow a relatively large tree that fits the data well and then *prune* it back to a smaller subtree. This is known as *post-pruning*. The idea is that, although the greedy algorithm may make suboptimal choices in the early splits, it is often easier in practice to start from a rich model and then simplify it.

One principled way to formalize pruning is through a regularized training criterion that trades off goodness of fit against tree complexity. Let T denote a tree and let $|T|$ denote the number of leaves (terminal nodes) in T. For a given tree we can define its empirical risk on the training data as

$$R(T) = \frac{1}{n} \sum_{i=1}^{n} L\big(y_i, \hat{f}_T(x_i)\big),$$

where L is an appropriate loss function (squared loss for regression, zero–one loss or cross-entropy for classification) and $\hat{f}_T$ denotes the predictor defined by tree T. We then introduce a *cost-complexity functional*

$$C_\alpha(T) = R(T) + \alpha\,|T|,$$

where $\alpha \geq 0$ is a tuning parameter that controls how strongly we penalize the number of leaves. For $\alpha = 0$ we recover the unpenalized training error; as α increases, the criterion favors smaller trees.

Searching over all possible trees to minimize $C_\alpha(T)$ would be computationally intractable. However, for a fixed large tree T_0 obtained by recursive splitting, there is an efficient dynamic programming algorithm to find, for each value of α, the *best subtree* of T_0 in terms of $C_\alpha(T)$. This procedure produces a finite sequence of nested trees

$$T_0 \supset T_1 \supset T_2 \supset \cdots \supset T_M,$$

ranging from the largest tree down to the root node. We can then use cross-validation to select the value of α, or equivalently the subtree T_m, that yields the lowest estimated prediction error.

In practice, many software libraries implement simpler variants of pruning or rely mostly on pre-pruning constraints. The important conceptual message is that tree complexity can and should be controlled by explicitly penalizing or limiting the size of the tree.

Interpretability and Regularization

One attractive feature of decision trees is that they can be relatively easy to interpret. Each path from the root to a leaf corresponds to a sequence of tests on the features, for example:

$$\text{if } x_{i\alpha_1} \leq t_1 \text{ and } x_{i\alpha_2} > t_2 \text{ and } x_{i\alpha_3} \leq t_3 \text{ then predict class 1.}$$

Such rules are often easier to communicate to non-technical stakeholders than coefficients in a linear model or the internal parameters of a neural network.

However, interpretability itself also depends on regularization: a very deep tree with hundreds of leaves is difficult to visualize and understand, even if it can be represented as a collection of explicit rules. By constraining the tree depth or pruning back to a small subtree, we not only reduce variance and improve generalization, but also obtain models that are more compact and interpretable.

In summary, decision trees are flexible models with the potential to overfit badly if left unconstrained. By controlling their depth, minimum leaf size, and overall size through pre-pruning and post-pruning strategies, we can obtain trees that strike a better balance between bias and variance and that remain useful as interpretable models. In the next sections we will see how these ideas extend to *ensembles* of trees, such as bagging, random forests, and boosting, which typically achieve much better predictive performance than any single tree.

8.4. Tree Ensemble Methods: Bagging and Random Forests

In the previous sections we have seen that individual decision trees are flexible but unstable models: small changes in the training data can lead to very different trees and predictions. One powerful idea to reduce this *variance* is to average the predictions of many trees, each trained on a slightly different version of the data. This leads to *bagging* (from *bootstrap aggregating*) and, with an additional ingredient, *random forests*.

Bootstrap Aggregating (Bagging)

Suppose we have a learning algorithm that, given the training data $\{(x_i, y_i)\}_{i=1}^n$, produces a predictor $\hat{f}(x)$. We assume that the algorithm is *unstable*: if we train it on a different sample from the same population, we may obtain quite different predictors. Decision trees are a canonical example of such an algorithm.

The idea of bagging is to generate many perturbed versions of the training set using the *bootstrap*, train a separate tree on each perturbed dataset, and then average their predictions.

Bootstrap samples. From the original training set of size n, we generate B new datasets

$$\mathcal{D}^{(1)}, \mathcal{D}^{(2)}, \ldots, \mathcal{D}^{(B)},$$

each of size n, by sampling observations (x_i, y_i) *with replacement*. Concretely, to build $\mathcal{D}^{(b)}$ we draw indices $i_1^{(b)}, \ldots, i_n^{(b)}$ independently and uniformly from $\{1, \ldots, n\}$, and set

$$\mathcal{D}^{(b)} = \big\{(x_{i_1^{(b)}}, y_{i_1^{(b)}}), \ldots, (x_{i_n^{(b)}}, y_{i_n^{(b)}})\big\}.$$

Some original observations will appear multiple times in a bootstrap sample, and some will not appear at all.

Bagged predictor. We then train a separate decision tree $\hat{f}^{(b)}(x)$ on each bootstrap sample $\mathcal{D}^{(b)}$, using the same tree-growing procedure as before. The *bagged* predictor is the average of their outputs:

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^{B} \hat{f}^{(b)}(x) \qquad \text{(regression)},$$

or, for classification, we can average the class probability estimates or take a majority vote over the predicted classes.

Bagging is particularly effective when the base learner (here, a decision tree) has low bias but high variance. Intuitively, averaging many such predictors cancels out part of their variability while preserving their ability to capture complex patterns.

Variance reduction by averaging. To make this idea more precise, consider a simplified setting where we have B predictors $\hat{f}^{(1)}(x), \ldots, \hat{f}^{(B)}(x)$, each with the same variance $(\hat{f}^{(b)}(x)) = \tau^2$, and with pairwise correlation ρ. The variance of their average is

$$\big(\hat{f}_{\text{avg}}(x)\big) = \left(\frac{1}{B}\sum_{b=1}^{B}\hat{f}^{(b)}(x)\right) = \frac{1}{B^2}\left(B\tau^2 + B(B-1)\rho\tau^2\right) = \frac{\tau^2}{B}\big(1 + (B-1)\rho\big).$$

When the predictors are uncorrelated ($\rho \approx 0$), the variance decreases roughly as $1/B$. When they are strongly correlated ($\rho \approx 1$), averaging brings much less benefit. Bagging reduces the variance of a single tree by averaging over trees trained on different bootstrap samples, but the trees can still be fairly correlated, especially if there are very strong predictor variables that dominate the splitting decisions. This observation motivates the additional randomness used in random forests.

Out-of-Bag Error Estimation

An appealing feature of bagging is that it provides a built-in estimate of prediction error, without the need for a separate validation set. The key observation is that each bootstrap sample $\mathcal{D}^{(b)}$ leaves out, on average, about a fraction $\exp(-1) \approx 0.37$ of the original observations.

For each training observation (x_i, y_i), we can collect the set of trees for which i was *not* included in the corresponding bootstrap sample. Let $\mathcal{B}_i$ denote this set of bootstrap indices:

$$\mathcal{B}_i = \{b \in \{1, \ldots, B\} : (x_i, y_i) \text{ does not appear in } \mathcal{D}^{(b)}\}.$$

We then define the *out-of-bag* (OOB) prediction for x_i as the average of the predictions of these trees:

$$\hat{f}_{\text{OOB}}(x_i) = \frac{1}{|\mathcal{B}_i|}\sum_{b \in \mathcal{B}_i}\hat{f}^{(b)}(x_i).$$

The OOB error is simply the average loss between y_i and $\hat{f}_{\text{OOB}}(x_i)$ over all training points. For regression we can use squared loss,

$$\text{OOB MSE} = \frac{1}{n}\sum_{i=1}^{n}\big(y_i - \hat{f}_{\text{OOB}}(x_i)\big)^2,$$

and for classification we can use the fraction of misclassified points or the cross-entropy loss. In practice, the OOB error provides a reliable estimate of the test error and can be used to tune hyperparameters such as the number of trees B or the complexity of the individual trees.

Random Forests: Decorrelating Trees by Random Feature Selection

Random forests build upon bagging by adding another source of randomness: at each split in each tree, instead of considering all features $\alpha = 1, \ldots, p$, we randomly select a subset of features and search for the best split only among those.

More precisely, when considering a node with region R during the growth of tree b, we proceed as follows:

1. Randomly select (without replacement) a subset $\mathcal{A} \subset \{1, \dots, p\}$ of feature indices, typically of fixed size $m_{\text{try}} \ll p$.
2. Among all candidate splits of the form $x_{i\alpha} \leq t$ with $\alpha \in \mathcal{A}$ and possible thresholds t, find the split that maximizes the impurity decrease (regression) or information gain (classification).
3. Split the node according to this best split and continue the recursive procedure.

The resulting ensemble of trees is called a *random forest*. The random feature selection at each split has two important effects:

- It forces different trees to explore different splitting structures, even if there are very strong predictors in the data. This reduces the correlation ρ between the trees.
- By lowering the correlation while keeping the variance of each individual tree relatively high, random forests increase the variance reduction achieved by averaging, leading to better generalization performance compared to plain bagging of trees.

In practice, random forests are among the most effective off-the-shelf predictors for tabular data. They require little feature engineering, can handle mixed types of features, and are relatively robust to outliers.

Hyperparameters of random forests. The main hyperparameters of a random forest are:

- The number of trees B. In general, more trees reduce variance (up to a point) at the cost of increased computation.
- The number of features m_{try} considered at each split. Smaller values of m_{try} increase the diversity of the trees (lower correlation) but may miss strong predictors at some splits. Typical defaults are $m_{\text{try}} = \sqrt{p}$ for classification and $m_{\text{try}} = p/3$ for regression, but the optimal choice is problem-dependent.
- The complexity of the individual trees: maximum depth, minimum number of observations per leaf, minimum impurity decrease, and so on. Often, the trees in a random forest are grown with little or no pruning (very deep trees), relying on the averaging effect of the ensemble to control variance.

These hyperparameters can be tuned by cross-validation or using the OOB error estimate.

Variable Importance

Although a random forest is less interpretable than a single small tree, it still provides useful information about the *importance* of each feature. One common measure of variable importance is based on the total impurity reduction attributed to splits using a given feature index α.

For each tree in the forest, we can compute, for every internal node that splits on feature α, the impurity decrease ΔI (or ΔQ in regression) produced by that split. Summing these decreases over all nodes and over all trees, and normalizing appropriately, yields an importance score for feature α. Features that are frequently used in splits that substantially reduce impurity will receive higher scores.

An alternative, more model-agnostic approach is *permutation importance*: we randomly permute the values of feature α among the training observations (breaking any association between that feature and the target), recompute the OOB error, and measure how much the error increases compared to the original forest. A large increase in OOB error indicates that the feature was important for prediction.

To summarize, bagging and random forests transform a single, high-variance decision tree into a powerful and stable predictor by averaging many trees trained on bootstrapped datasets and by introducing additional randomness in the choice of features at each split. In the next section we will see a complementary approach, *boosting*, which builds an ensemble of trees in a sequential manner to reduce bias as well as variance.

8.5. Boosting and Gradient Boosting

Bagging and random forests reduce the variance of decision trees by averaging many models trained in parallel on perturbed versions of the training data. *Boosting* takes a complementary approach: instead of training many trees independently and averaging them, it builds an ensemble *sequentially*, where each new tree focuses on the examples that were poorly predicted by the current ensemble.

Intuitively, boosting starts from a simple model and gradually increases its complexity by adding new trees that correct the errors of the previous ones. This typically leads to low-bias, low-variance predictors that perform very well on a wide range of problems, especially when using trees as base learners.

Boosting as Stagewise Additive Modeling

Formally, boosting constructs a predictor $f(x)$ as a sum of M simple base learners:

$$f_M(x) = \sum_{m=1}^{M} \gamma_m \, h_m(x),$$

where each $h_m(x)$ is a base learner (for us, a shallow decision tree), and γ_m is a scalar weight. The base learners are chosen *stagewise*: at stage m, we have already built

$$f_{m-1}(x) = \sum_{k=1}^{m-1} \gamma_k \, h_k(x),$$

and we add a new term $\gamma_m h_m(x)$ to obtain a better fit.

Given a loss function $L(y, f(x))$ (for example, squared loss for regression, or logistic loss for classification), the ideal goal would be to choose γ_m and h_m so as to minimize the empirical risk

$$R(f) = \frac{1}{n} \sum_{i=1}^{n} L\big(y_i, f(x_i)\big)$$

over all functions f of the form $f_M(x) = \sum_{m=1}^{M} \gamma_m h_m(x)$. This is still a difficult optimization problem because the space of possible trees h_m is huge. Boosting simplifies it by using a greedy, stagewise procedure that at each step tries to move in the direction that most reduces the loss, in analogy with gradient descent.

Boosting for Regression: Fitting Residuals

To see how boosting works in a simple setting, consider regression with squared loss

$$L(y, f(x)) = \big(y - f(x)\big)^2.$$

Suppose we have a current predictor $f_{m-1}(x)$. The gradient of the loss with respect to the prediction at point x_i is

$$\frac{\partial}{\partial f(x_i)} \big(y_i - f(x_i)\big)^2 = -2\big(y_i - f(x_i)\big).$$

Thus, apart from a constant factor, the negative gradient is just the *residual*

$$r_i^{(m)} = y_i - f_{m-1}(x_i).$$

Gradient boosting for regression with squared loss proceeds as follows:

1. Initialize the model with a constant prediction, for example the mean of the targets:

$$f_0(x) = \bar{y} = \frac{1}{n} \sum_{i=1}^{n} y_i.$$

2. For $m = 1, 2, \ldots, M$:
   a) Compute the residuals

$$r_i^{(m)} = y_i - f_{m-1}(x_i), \qquad i = 1, \ldots, n.$$

   b) Fit a regression tree $h_m(x)$ to the data $\{(x_i, r_i^{(m)})\}_{i=1}^n$, using squared loss on the residuals.
   c) Choose a step size γ_m, for example by solving a one-dimensional least-squares problem along the direction $h_m(x)$, or by using a fixed *learning rate* $\nu > 0$, $\gamma_m = \nu$.
   d) Update the model:

$$f_m(x) = f_{m-1}(x) + \gamma_m \, h_m(x).$$

In words, at each stage we fit a new tree to the residuals of the current model, and then add a small multiple of this tree to reduce those residuals. This is exactly gradient descent in function space: the residuals play the role of the negative gradient of the loss with respect to the predictions.

Gradient Boosting for General Loss Functions

The approach above can be generalized to other loss functions beyond squared loss. Let $L(y, f(x))$ be a differentiable loss with respect to the prediction $f(x)$. At iteration m, given the current predictor f_{m-1}, we compute the *pseudo-residuals*

$$r_i^{(m)} = - \left. \frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right|_{f = f_{m-1}}, \qquad i = 1, \ldots, n.$$

These pseudo-residuals are the negative gradients of the loss with respect to the predictions. We then fit a regression tree $h_m(x)$ to the data $\{(x_i, r_i^{(m)})\}$ and update

$$f_m(x) = f_{m-1}(x) + \gamma_m h_m(x),$$

where γ_m is chosen (approximately) to minimize the loss along the direction h_m, for example by a one-dimensional line search:

$$\gamma_m \approx \arg\min_{\gamma} \sum_{i=1}^{n} L\big(y_i, f_{m-1}(x_i) + \gamma h_m(x_i)\big).$$

This procedure is known as *gradient boosting*. When the base learners h_m are decision trees, the resulting models are called *gradient boosted decision trees* (GBDT), and are implemented in popular libraries such as XGBoost, LightGBM, and others.

Classification with logistic loss. A common choice for binary classification is the logistic (cross-entropy) loss. Let the binary labels be encoded as $y_i \in \{0, 1\}$, and let $f(x)$ denote the logit,

$$p(x) = \mathbb{P}(Y = 1 \mid X = x) \approx \sigma\big(f(x)\big) = \frac{1}{1 + \exp(-f(x))}.$$

The logistic loss for a single observation is

$$L(y, f(x)) = -\big[y \log \sigma(f(x)) + (1 - y) \log(1 - \sigma(f(x)))\big].$$

The negative gradient with respect to $f(x_i)$ turns out to be

$$r_i^{(m)} = y_i - p_{m-1}(x_i),$$

where $p_{m-1}(x_i) = \sigma\big(f_{m-1}(x_i)\big)$ is the current predicted probability. Thus, for logistic loss, the pseudo-residuals are simply the differences between the true labels and the current predicted probabilities, and gradient boosting again fits trees to these residuals.

Regularization in Boosting

Boosting algorithms are very powerful and can easily overfit if we make the ensemble too flexible. Several regularization mechanisms are therefore crucial for good performance:

- **Learning rate (shrinkage).** Instead of taking $\gamma_m = 1$, we often use a small learning rate $\gamma_m = \nu$ with $0 < \nu \ll 1$ (for example $\nu = 0.1$ or $\nu = 0.01$). Smaller learning rates require more trees but can improve generalization by making each stagewise update more conservative.
- **Number of trees.** The total number of stages M controls the overall complexity of the model. Too few trees lead to underfitting; too many trees can lead to overfitting, especially if the learning rate is not small. In practice, M is often chosen by monitoring the validation error and stopping when it stops improving (*early stopping*).
- **Tree depth and structure.** The base learners $h_m(x)$ are usually chosen to be shallow trees (for example, depth 1–5), often called *weak learners*. This limits their individual complexity and helps to prevent overfitting. Boosting combines many such weak learners to obtain a strong predictor.
- **Subsampling.** Some implementations of gradient boosting (stochastic gradient boosting) use only a random subset of observations at each iteration. This introduces additional randomness that can reduce variance and speed up training, similar in spirit to random forests.

The hyperparameters in boosting (learning rate, number of trees, tree depth, subsample size) play a similar role to regularization parameters in linear models or pre-pruning constraints in trees, and are typically tuned by cross-validation.

Boosting vs. Random Forests

Both random forests and gradient boosting build ensembles of decision trees, but they emphasize different aspects of the bias–variance trade-off:

- Random forests use **many deep, unpruned trees**, grown on bootstrapped samples with random feature selection, and average their predictions. They primarily reduce *variance*.
- Gradient boosting uses **many shallow trees** added sequentially, each focusing on the residuals or negative gradients of the current model. It can reduce both *bias* and variance, often achieving higher accuracy but with more sensitivity to hyperparameters.

In practice, random forests are simple, robust baseline models that work well with little tuning, while gradient boosted trees can yield state-of-the-art performance on many tabular datasets when carefully tuned. Together, these tree-based ensemble methods form an essential part of the modern machine learning toolbox.

8.6. Practical Guidance and Summary

In this chapter we have seen how decision trees and their ensembles provide flexible and powerful models for regression and classification. Before moving on, it is useful to summarize when tree-based methods are particularly appropriate, how they compare to other models we have studied, and how one might use them in practice on a new dataset.

When to Use Tree-Based Methods

Tree-based methods are especially well suited to *tabular* data: datasets in which each observation is described by a moderate number of features of possibly different types, and where there is no particular spatial, temporal or grid structure (unlike images or time series). Typical applications include:

- **Prediction from heterogeneous features.** Many real-world problems involve mixtures of numerical, categorical, and ordinal features (for example, age, income, education level, country). Trees handle such data naturally, often without requiring extensive preprocessing.
- **Nonlinear effects and interactions.** Trees can capture strong nonlinear relationships and complex interactions between features by recursively splitting the input space. There is no need to manually create interaction terms as in linear models.
- **Moderate dataset sizes.** Random forests and gradient boosted trees work very well for datasets of small to medium size (from a few hundred to a few hundred thousand observations, depending on the implementation and hardware).

On the other hand, tree-based methods are not always the best choice:

- For high-dimensional structured data such as images, audio, or text, neural networks or kernel methods are typically more effective.

- When interpretability in terms of linear effects (for example, estimating the effect of one feature while controlling for others) is the primary goal, simple linear or generalized linear models may be more appropriate.
- For extremely large datasets with millions of observations, the training cost of very large ensembles may become prohibitive, and simpler models or approximate methods may be preferred.

A Practical Modeling Workflow

When facing a new supervised learning problem with tabular data, a reasonable workflow using tree-based methods might look like this:

1. **Start with a simple baseline.** Fit a regularized linear or logistic regression model. This provides a useful reference point, is easy to interpret, and helps to detect obvious issues with the data.
2. **Train a random forest.** Use a random forest with default hyperparameters (or modest tuning) as a strong, robust baseline. Monitor the out-of-bag (OOB) error or a validation error to assess performance.
3. **Inspect variable importance.** Use impurity-based or permutation-based importance measures to identify which features are most influential in the random forest. This can guide further feature engineering or data collection.
4. **Try gradient boosting if higher accuracy is needed.** If the random forest performance is good but you need the best possible accuracy, train a gradient boosted tree model (for example, using shallow trees and a small learning rate). Tune the number of trees, learning rate, and tree depth using a validation set or cross-validation.
5. **Control overfitting.** For both random forests and boosting, monitor the validation error as a function of the number of trees and other hyperparameters. Use early stopping (for boosting) or limit tree depth and minimum leaf size to avoid overfitting.
6. **Interpret and communicate results.** For small trees, it may be possible to visualize the entire tree and extract human-readable rules. For ensembles, use variable importance, partial dependence plots, or other post-hoc interpretation tools to explain model behavior.

This workflow highlights an important fact: in many applications, random forests or gradient boosted trees serve as high-quality default models for tabular data, often outperforming more complex neural networks with less tuning effort.

Comparison with Other Methods

It is useful to situate tree-based methods within the broader landscape of models covered in this course:

- **Linear and logistic regression.** These models are simple, fast, and easy to interpret. They work best when the relationship between features and target is approximately linear (possibly after a simple transformation). Trees and their ensembles are more flexible, and can model complex nonlinear patterns, at the cost of reduced interpretability.

- **Kernel methods.** Kernel regression and support vector machines can represent very complex functions and perform well in moderate dimensions, but their computational cost typically scales poorly with the number of observations. Tree ensembles, by contrast, often scale better to larger datasets and require less feature engineering.
- **Neural networks.** Deep networks are extremely powerful for high-dimensional structured data such as images and text, but they require careful design and tuning, and their predictions are often harder to interpret. For tabular data, tree ensembles frequently match or outperform neural networks with much less effort.

In summary, tree-based methods occupy an important niche: they are flexible nonlinear models that work very well for many practical problems, especially with tabular data, and they are often the first serious models to try after simple linear baselines.

Summary

Let us briefly recap the main ideas of this chapter:

- A **decision tree** recursively partitions the input space into regions using axis-aligned splits on individual features. It predicts by averaging the target values (regression) or taking the majority class (classification) within each region.
- The tree is grown by a **greedy top-down algorithm** that chooses at each node the split that maximizes impurity reduction (variance reduction in regression, Gini or entropy decrease in classification).
- Individual trees are flexible but **high-variance** models: if grown too deep they can overfit badly. Their complexity is controlled by *pre-pruning* (constraints on depth, minimum leaf size, minimum impurity decrease) and *post-pruning* (cost-complexity pruning of a large initial tree).
- **Bagging** (bootstrap aggregating) reduces variance by averaging the predictions of many trees trained on bootstrap samples of the data. The out-of-bag error provides an internal estimate of test error.
- **Random forests** add further randomness by selecting a random subset of features at each split, which decorrelates the trees and increases the variance reduction achieved by averaging. They provide strong baselines and measures of variable importance.
- **Boosting** builds an ensemble of trees sequentially, where each new tree is trained to correct the errors of the current model, often using the negative gradients (pseudo-residuals) of a loss function. Gradient boosted trees can achieve state-of-the-art accuracy on many tabular datasets when properly regularized.
- In practice, tree-based methods form a crucial part of the modern machine learning toolbox. They complement linear models, kernel methods, and neural networks, and are often the method of choice for predictive modeling with heterogeneous tabular data.

In the next chapters we will return to the general theme of dimensionality reduction and the geometry of data, and see how ideas such as principal component analysis can be extended to more general nonlinear settings.

A. Exercises

Chapter 2: Linear Regression

I. Core Exercises

Q1. Understanding the Linear Model
Write the equation of a simple linear regression model with one input feature x. Explain what each term represents and what assumptions the model makes about the relationship between x and y.

Q2. Least Squares Loss
Define the **Mean Squared Error (MSE)** loss function for a linear regression model with parameters θ_0, θ_1. Explain in words what this loss measures and why minimizing it gives a good fit to the data.

Q3. Geometric Interpretation
In linear regression, explain what the fitted line represents in geometric terms with respect to the observed data points.

Q4. Model Evaluation
List two commonly used metrics to evaluate linear regression models. Briefly describe what each metric indicates about model performance.

II. More Advanced Exercises

Q5. Batch vs. Stochastic Gradient Descent
Explain the difference between **Batch Gradient Descent** and **Stochastic Gradient Descent (SGD)**. Discuss one advantage and one disadvantage of each method.

Q6. Feature Scaling
Why can feature scaling or normalization be important when training a linear regression model with gradient descent? Illustrate your answer by referring to the geometry of the cost function surface.

Q7. Interpreting Coefficients
Suppose a multiple linear regression model is fitted to predict house prices using features: size (m^2), number of rooms, and distance to city center. Explain how you would interpret the sign and magnitude of each estimated coefficient.

III. Very Challenging Exercises

Q8. Collinearity and Identifiability
You are fitting a linear regression model with two highly correlated predictors x_1 and x_2.

(a) Explain why this situation causes instability in the estimated coefficients.

(b) Propose one strategy to detect and one strategy to mitigate this issue.

Q9. Connecting Bias–Variance and Regularization
Consider the regularized cost function

$$J(\theta) = \frac{1}{2m}\sum_{i=1}^{m}(\hat{y}^{(i)} - y^{(i)})^2 + \lambda\sum_{j=1}^{p}\theta_j^2.$$

(a) Explain qualitatively how increasing λ affects bias and variance.

(b) Discuss how this modification relates to the probabilistic interpretation of linear regression.

Chapter 3: Logistic Regression

I. Core Exercises

Q1. Motivation for Logistic Regression
Explain why **linear regression** is not suitable for binary classification tasks. Describe one concrete issue that occurs when using a linear model to predict categorical labels.

Q2. Decision Boundary
Given the logistic model $\hat{y} = \sigma(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$, derive the equation of the decision boundary in terms of x_1 and x_2. What kind of curve does it represent?

Q3. Evaluation Metrics
List two commonly used performance metrics for binary classification and explain what aspect of performance each measures.

II. More Advanced Exercises

Q4. Gradient of the Loss
Show that the derivative of the cross-entropy loss with respect to the model parameters θ for a single sample is proportional to

$$(\hat{y} - y)x.$$

Explain why this leads to an efficient gradient descent update rule.

Q5. Multi-Feature Decision Boundary
A logistic regression model is trained on two features: exam score (x_1) and study hours (x_2). The fitted parameters are $\theta_0 = -10$, $\theta_1 = 0.05$, $\theta_2 = 0.1$. Find the equation of the decision boundary and describe in words which students are more likely to be admitted.

Q6. Evaluating Imbalanced Data
Explain why accuracy can be misleading when evaluating a logistic regression model trained on highly imbalanced data (e.g., 95% of examples belong to one class). Suggest two alternative metrics that would be more informative in this setting.

III. Very Challenging Exercises

Q7. Nonlinear Decision Boundaries
Suppose your data are not linearly separable in the original feature space. Explain how you could modify the logistic regression model to capture nonlinear relationships between features and the output, without changing the loss function itself. Provide one example of such a feature transformation.

Q8. Connection to Maximum Likelihood
Assume a logistic regression model with Bernoulli likelihood for $y \in \{0, 1\}$.

(a) Write the likelihood function for m independent observations.

(b) Show that maximizing this likelihood with respect to θ is equivalent to minimizing the cross-entropy loss.

(c) Explain intuitively why this probabilistic perspective justifies using cross-entropy as the objective function.

Chapter 4: Dimensionality Reduction and Clustering

I. Core Exercises

Q1. Motivation for Dimensionality Reduction
Explain why dimensionality reduction is often useful in machine learning. Give two examples of problems that may arise when working with very high-dimensional data.

Q2. Variance Maximization in PCA
In Principal Component Analysis (PCA), what does each principal component represent? Explain why the first principal component is chosen as the direction of maximum variance.

Q3. Covariance Matrix
Given a dataset X with n samples and p standardized features, write the expression for the covariance matrix. What do the diagonal and off-diagonal elements of this matrix represent?

Q4. Projection onto Principal Components
After computing the eigenvectors of the covariance matrix, how do we project the original data into a lower-dimensional space? Explain geometrically what this projection does.

Q5. k-Means Overview
List the main steps of the **k-Means** algorithm in order. Briefly describe what happens in the *assignment* and *update* phases.

II. More Advanced Exercises

Q6. PCA Computation Example
You are given two standardized features x_1 and x_2 with covariance matrix

$$\Sigma = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}.$$

(a) Compute the eigenvalues and eigenvectors of Σ.

(b) Determine the proportion of variance explained by the first principal component.

Q7. Effect of Scaling
Suppose one feature in your dataset has values in the range $[0, 1000]$ and another in $[0, 1]$. Explain how this affects PCA if the data are not standardized, and why scaling before PCA is essential.

Q8. Choosing the Number of Clusters
Describe one quantitative and one qualitative method to choose the appropriate number of clusters k in k-Means. What trade-off is involved when selecting a higher or lower value of k?

Q9. Comparing PCA and k-Means
PCA and k-Means are both unsupervised methods. Compare their goals and outputs. Under what circumstances might PCA be a useful preprocessing step before applying k-Means?

III. Very Challenging Exercises

Q10. Interpreting PCA Directions
Suppose you perform PCA on gene expression data and find that the first two principal components explain 85% of the total variance.

(a) Explain why a high explained variance ratio does not necessarily imply that the first two components correspond to biologically meaningful factors.

(b) Propose a strategy to interpret or validate the meaning of these components.

Q11. Connecting PCA and k-Means Geometrically
Consider a dataset of points forming three well-separated clusters in $\mathbb{R}^3$.

(a) Describe qualitatively what the first two principal components would capture in this dataset.

(b) Explain why the first two principal components often align with directions that best separate clusters, even though PCA does not use any label information.

(c) Discuss one situation where PCA might fail to reveal the cluster structure effectively.

Chapter 5: Model Performance, Generalization, and Regularization

I. Core Exercises

Q1. Underfitting and Overfitting
Define *underfitting* and *overfitting*. For each of the following situations, indicate whether it corresponds to underfitting, overfitting, or good generalization, and briefly justify:

(a) The model has very high training error and high test error.

(b) The model has very low training error but high test error.

(c) The model performs well on both training and test data.

Q2. Bias–Variance Trade-Off
Explain in simple terms what is meant by the **bias–variance trade-off**. Illustrate with an example of how increasing model complexity affects bias and variance.

Q3. Model Complexity and Generalization
Sketch (conceptually) how training error and test error typically behave as model complexity increases. Explain why the test error eventually rises even though the training error keeps decreasing.

II. More Advanced Exercises

Effect of Regularization on Coefficients
Consider two features, x_1 and x_2, that are highly correlated. Explain how Ridge and Lasso regression behave differently in this situation, and which one tends to produce sparse solutions.

Selecting the Regularization Parameter
Describe one practical method to select the optimal value of λ in Ridge or Lasso regression. Explain why this method helps balance bias and variance.

Regularization and Feature Scaling
Explain why feature scaling is important before applying Ridge or Lasso regularization. What could happen if one feature has values much larger in magnitude than the others?

III. Very Challenging Exercises

Q4. Bias–Variance in Regularized Models
Consider the regularized regression cost function

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^{m} (\hat{y}^{(i)} - y^{(i)})^2 + \lambda \|\theta\|_2^2.$$

(a) Explain qualitatively how increasing λ shifts the balance between bias and variance.

(b) Sketch or describe how the expected test error behaves as λ increases from 0 to a very large value.

Q5. Geometry of Regularization
In two dimensions, the constraint regions for Ridge and Lasso regression can be represented as contours in parameter space.

(a) Describe the geometric shape of the constraint region for Ridge and for Lasso.

(b) Explain why the shape of the Lasso constraint leads to coefficients becoming exactly zero, while Ridge coefficients rarely do.

Chapter 6: Kernel Methods

I. Core Exercises

Q1. Motivation for Kernel Methods
Explain why linear models may fail to capture complex relationships between input

features and output labels. Give one example of a dataset where a linear decision boundary would perform poorly and explain why.

Q2. Feature Space Mapping
In kernel methods, we map input data x from the original space to a higher-dimensional feature space via a transformation $\phi(x)$. Explain in words what the goal of this transformation is, and how it allows a linear algorithm to model non-linear relationships.

Q3. Definition of a Kernel Function
What is a kernel function? What are its properties? What its function in machine learning?

II. More Advanced Exercises

Q4. The Kernel Trick
Explain in your own words what the **kernel trick** is. Why does replacing inner products $\langle \phi(x_i), \phi(x_j) \rangle$ by a kernel function $K(x_i, x_j)$ avoid explicit computation in the high-dimensional feature space?

Q5. Kernel Ridge Regression
The dual prediction formula for kernel ridge regression is

$$\hat{y}(x) = \sum_{i=1}^{n} \alpha_i k(x, x_i),$$

where $\alpha_i = (K + \lambda I)^{-1} y_i$, $k(.,.)$ is the kernel function and K is the kernel matrix. Explain what each term represents and how it λ influences the smoothness of the fitted function. How would you fit λ? Draw a graph with the expected result.

Q6. Kernel Choice and Model Behavior
Discuss how the choice of kernel (e.g., linear vs. polynomial vs. Gaussian RBF) affects the flexibility and generalization of the model. What risk is associated with using a kernel that is too flexible?

III. Very Challenging Exercises

Q7. Constructing Valid Kernels
A student proposes the following functions as potential kernels:

$$K_1(x, x') = (1 + x^T x')^2,$$
$$K_2(x, x') = e^{-|x - x'|},$$
$$K_3(x, x') = x^T x' - 2.$$

(a) Which of these are valid positive semidefinite kernels?

(b) Explain briefly how you can determine kernel validity without computing the full feature mapping $\phi(x)$.

Q8. Geometry and Intuition of the RBF Kernel
The Gaussian RBF kernel is defined as

$$K(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2\sigma^2} \right).$$

(a) Explain intuitively what the parameter σ controls.

(b) Discuss how varying σ affects the bias–variance trade-off in kernel methods.

(c) Describe what happens to the feature space representation as $\sigma \to 0$ and as $\sigma \to \infty$.

Bibliography